**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ**

**ІМЕНІ ТАРАСА ШЕВЧЕНКА**

ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

**До захисту допущено**

**Завідувач кафедри ІСТ**

_____Володимир ДРУЖИНІН

(підпис) (ім'я, ПРІЗВИЩЕ)

"____"_____2026р.

# КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

Спеціальності 126 «Інформаційні системи та технології» освітньої програми «Програмні технології інтернет речей»

на тему: «ІОТ СИСТЕМА ЗАБЕЗПЕЧЕННЯ ПЕРЕВЕЗЕНЬ МОРСЬКИМ ТРАНСПОРТОМ»

Виконав: студент 4 курсу, групи ІР-41

(шифр групи)

Дмитро БЕЗНОСЮК

(Ім'я, ПРІЗВИЩЕ) (підпис)

Керівник к. т. н., доцент Ольга КРАВЧЕНКО

(посада, науковий ступінь, вчене звання, Ім'я, ПРІЗВИЩЕ) (підпис)

Консультант нормо контроль к.т.н, Ірина БОРИСЕНКО

(назва розділу) (посада, вчене звання, науковий ступінь, Ім'я, ПРІЗВИЩЕ) (підпис)

Рецензент _____ _____

(посада, науковий ступінь, вчене звання, науковий ступінь, Ім'я, ПРІЗВИЩЕ) (підпис)

Засвідчую, що у пояснювальній записці немає запозичень з праць інших авторів без відповідних посилань.

Здобувач освіти _____

(підпис)

Київ – 2026

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

ІМЕНІ ТАРАСА ШЕВЧЕНКА

Факультет інформаційних технологій

Кафедра Інформаційні системи та технології

Освітній рівень Бакалавр

Спеціальність 126 Інформаційні системи та технології

Освітня програма Програмні технології інтернет речей

ЗАТВЕРДЖУЮ

Завідувач кафедри,

д.т.н., професор

Володимир ДРУЖИНІН

_____

«__» _____ 2026 року

ЗАВДАННЯ

НА ВИКОНАННЯ КВАЛІФІКАЦІЙНОЇ РОБОТИ БАКАЛАВРА

Студент: Дмитро БЕЗНОСЮК

Група: ІР-41

1. Тема кваліфікаційна робота бакалавра: «ІОТ система забезпечення перевезень морським транспортом» Затверджена протоколом засідання кафедри ІСТ № 6_25/26 від 21 листопада 2025 року.

2. Строк подання студентом готової роботи - «15» червня 2026 р.

3. Вихідні дані до роботи: програмне й апаратне забезпечення для розробки системи міжнародної торгівлі.

4. Зміст роботи:

РОЗДІЛ 1. АНАЛІЗ ТА ОПИС СФЕРИ ЗАСТОСУВАННЯ. ПОСТАНОВКА ЗАДАЧІ

РОЗДІЛ 2. ПРОЄКТУВАННЯ КОНЦЕПТУАЛЬНОЇ, ЛОГІЧНОЇ ТА ФІЗИЧНОЇ МОДЕЛЕЙ БАЗИ ДАНИХ

РОЗДІЛ 3. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

РОЗДІЛ 4. ТЕСТУВАННЯ, ПЛАНУВАННЯ ТА ЕКОНОМІЧНЕ

ОБҐРУНТУВАННЯ ПРОЄКТУ

5. Календарний план виконання роботи:

| Етап виконання кваліфікаційної роботи бакалавра | Термін виконання | Результат виконання |
|---|---|---|
| 1. Вибір тематики кваліфікаційної роботи бакалавра | до 01.11.2025 | Виконано |
| 2. Наказ про затвердження тем кваліфікаційної роботи бакалавра та призначення керівників | 21.11.2025 | Виконано |
| 3. Розробка плану кваліфікаційної роботи бакалавра і його погодження з керівником | 15.01.2026 | Виконано |
| 4. Написання I - IV розділу кваліфікаційної роботи | 11.05.2026 | Виконано |
| 5. Підготовка висновків і пропозицій | 11.05.2026 | Виконано |
| 6. Попередній захист кваліфікаційної роботи | 13.05.2026 | Виконано |
| 7. Перевірка на плагіат | до 21.05.2026 | Виконано |
| 8. Нормоконтроль | до 08.06.2026 | Виконано |
| 9. Рецензування кваліфікаційної роботи бакалавра і представлення роботи на кафедру у друкованому вигляді | до 15.06.2026 | Виконано |
| 10. Захист кваліфікаційної роботи бакалавра | 23.06.2026 | Виконано |

Дата видачі завдання «21» листопада 2025 р.

Керівник роботи: к.т.н., доц. Ольга КРАВЧЕНКО _____

(підпис)

Завдання прийняв до виконання:

Здобувач освіти на освітньому рівні «бакалавр» 4-го курсу групи ІР-41

Дмитро БЕЗНОСЮК _____

(Ім’я, ПРІЗВИЩЕ) (підпис)

# АНОТАЦІЯ

Пояснювальна записка до кваліфікаційної роботи бакалавра на тему «IoT система забезпечення перевезень морським транспортом» містить 4 розділи, список використаних джерел та додатки.

Робота присвячена розробці цифрової платформи для морської контейнерної логістики, яка об'єднує оренду контейнерів, IoT-моніторинг стану вантажів і фінансовий облік в єдиному середовищі. Аналіз ринку показав, що існуючі рішення не надають клієнту повного контролю над замовленням — ця платформа усуває таку залежність і відкриває можливість замовнику самостійно керувати орендою від першої заявки до завершення перевезення.

У першому розділі досліджено предметну область, проаналізовано конкурентні рішення та описано вісім типів IoT-датчиків для контролю температури, вологості, удару, нахилу, стану дверей, освітлення, газів і геопозиції. У другому розділі спроєктовано архітектуру системи через діаграми, розроблено реляційну модель бази даних із партиціонуванням таблиці телеметрії та криптографічним захистом IoT-аудиту. У третьому розділі реалізовано платформу на Laravel і Vue.js з розгортанням через Docker, описано алгоритм багатокомпонентного розрахунку вартості оренди, інтерфейс і інструкцію користувача. У четвертому розділі виконано тестування, побудовано WBS і діаграму Ганта, розраховано бюджет та підтверджено економічну ефективність впровадження.

Практична цінність платформи полягає в тому, що клієнт самостійно обирає контейнер, розраховує вартість з повною розбивкою, відстежує маршрут і бачить телеметрію датчиків у реальному часі. Усі дані захищені криптографічно і не можуть бути змінені після збереження, що важливо для страхових і митних процедур.

Оригінальність роботи — у поєднанні криптографічного захисту телеметрії, прозорого ціноутворення та повноцінного IoT-моніторингу в рамках єдиної готової до впровадження платформи.

**КЛЮЧОВІ СЛОВА:** IOT, МОРСЬКА ЛОГІСТИКА, ОРЕНДА КОНТЕЙНЕРІВ, LARAVEL, VUE.JS, POSTGRESQL, ТЕЛЕМЕТРІЯ, МОНІТОРИНГ ВАНТАЖІВ, ВЕБ-ПЛАТФОРМА.

# ANNOTATION

The explanatory note for the bachelor's qualification thesis "IoT System for Maritime Cargo Transportation" contains 4 chapters, a list of references, and appendices.

This work presents the development of the platform for maritime container logistics, combining container rental, IoT cargo monitoring, and financial management in a single environment. Market analysis showed that existing solutions do not give clients full control over their orders — this platform eliminates that gap, allowing customers to independently manage the entire rental process from the first request to delivery completion.

The first chapter examines the domain, analyzes competitive solutions, and describes eight IoT sensor types covering temperature, humidity, shock, tilt, door status, illumination, gas levels, and GPS. The second chapter covers system architecture through IDEF0, IDEF3, DFD, and UML diagrams, and a relational database with telemetry partitioning and a cryptographic IoT audit chain. The third chapter presents the platform implementation using Laravel and Vue.js with Docker deployment, the multi-component rental cost algorithm, user interface, and user manual. The fourth chapter covers testing, WBS and Gantt chart construction, budget calculation, and economic efficiency analysis confirming short-term return on investment.

The practical value of the platform lies in clients being able to independently select a container, calculate costs with a full breakdown, and monitor sensor telemetry in real time. All data is cryptographically protected and cannot be altered after recording, which is critical for insurance and customs procedures.

The original contribution is the combination of cryptographically protected telemetry, transparent multi-component pricing, and full IoT monitoring within a single deployment-ready platform.

**KEYWORDS:** IOT, MARITIME LOGISTICS, CONTAINER RENTAL, LARAVEL, VUE.JS, POSTGRESQL, TELEMETRY, CARGO MONITORING, WEB PLATFORM.

**ЗМІСТ**

**ВСТУП** ..... 8

**РОЗДІЛ 1. АНАЛІЗ ТА ОПИС СФЕРИ ЗАСТОСУВАННЯ. ПОСТАНОВКА ЗАДАЧІ** ..... 9

**1.1 Характеристика об'єкта дослідження** ..... 9

**1.2 Організаційна структура підприємства, підрозділу** ..... 9

**1.3 Функції та завдання підрозділу** ..... 10

**1.4 Дерево цілей** ..... 12

**1.5 Аналіз існуючих рішень** ..... 13

**1.6 Апаратні засоби IoT-системи** ..... 16

**1.7 Побудова моделі системи у нотації BPMN** ..... 17

**Висновки до першого розділу** ..... 18

**РОЗДІЛ 2. ПРОЄКТУВАННЯ КОНЦЕПТУАЛЬНОЇ, ЛОГІЧНОЇ ТА ФІЗИЧНОЇ МОДЕЛЕЙ БАЗИ ДАНИХ** ..... 19

**2.1 Створення діаграм системи у нотаціях IDEF0, IDEF3 та DFD** ..... 19

**2.2 Побудова концепції автоматизованої системи в нотації UML** ..... 21

**2.2.1 Діаграма прецедентів Use Case** ..... 21

**2.2.2 Діаграма діяльності Activity Diagram** ..... 22

**2.2.3 Діаграма послідовності Sequence Diagram** ..... 23

**2.3 Концептуальна модель бази даних** ..... 25

**2.4 Логічна та фізична модель бази даних** ..... 27

**Висновки до другого розділу** ..... 30

**РОЗДІЛ 3. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ** ..... 31

**3.1 Мета програмного продукту** ..... 31

**3.2 Вхідні та вихідні дані** ..... 32

**3.3 Програмні засоби реалізації та обґрунтування їх вибору** ..... 34

**3.4 Блок-схема алгоритму** ..... 38

**3.5 Опис інтерфейсу програмного продукту** ..... 39

**3.6 Приклади застосування системи** ..... 44

**3.7 Інструкція користувача** ..... 48

**3.8 SWOT-аналіз** ..... 49

**Висновки до третього розділу** ..... 50

**РОЗДІЛ 4. ТЕСТУВАННЯ, ПЛАНУВАННЯ ТА ЕКОНОМІЧНЕ ОБҐРУНТУВАННЯ ПРОЄКТУ** ..... 52
**4.1 Тестування програмного продукту** ..... 52
**4.2 Формування структури робіт** ..... 53
**4.3 Календарний план та мережевий графік** ..... 54
**4.4 Ресурсне планування та розрахунок трудомісткості** ..... 55
**4.5 Бюджет проєкту** ..... 59
**4.6 Економічна ефективність впровадження** ..... 62
**Висновки до четвертого розділу** ..... 64
**ВИСНОВКИ** ..... 66
**СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ** ..... 67
**Додаток А** ..... 72
**Додаток Б** ..... 73
**Додаток В** ..... 74
**Додаток Г** ..... 76
**Додаток Д** ..... 80
**Додаток Е** ..... 81
**Додаток Ж** ..... 83
**Додаток З** ..... 84
**Додаток И** ..... 88

# ВСТУП

Морські вантажоперевезення забезпечують понад 80% обсягу світової торгівлі, проте галузь досі стикається з непрозорістю операцій, ризиками пошкодження вантажів і відсутністю зручних цифрових інструментів для клієнтів [3, 15, 23]. Більшість існуючих платформ виконують суто інформаційну функцію і не надають замовнику можливості самостійно керувати замовленням від початку до кінця. Аналіз ринку підтвердив, що жодна з розглянутих платформ не поєднує в єдиному середовищі оренду контейнерів, IoT-моніторинг, маршрутизацію і фінансовий облік — усі вони зводять взаємодію до заповнення заявки й очікування відповіді менеджера [7].

Метою роботи є проєктування та розробка IoT-системи забезпечення перевезень морським транспортом на прикладі платформи Logistics SWay, яка автоматизує процеси оренди контейнерів і забезпечує дистанційний моніторинг стану вантажів у режимі реального часу. Для досягнення цієї мети вирішено такі завдання: дослідження предметної області та аналіз існуючих рішень; проєктування архітектури системи і реляційної моделі бази даних; реалізація веб-платформи з підтримкою восьми типів IoT-датчиків, багатокомпонентним розрахунком вартості оренди та інтеграцією з Telegram-ботом; тестування і оцінка економічної ефективності впровадження [6, 7, 8, 24].

Об'єктом дослідження є процеси управління міжнародними морськими контейнерними перевезеннями, предметом — інформаційна система з підтримкою IoT-моніторингу для автоматизації оренди контейнерів та контролю умов транспортування. Практична цінність роботи полягає в тому, що розроблена платформа Logistics SWay є повністю функціональним продуктом на стеку PHP 8.2, Laravel, Vue.js, PostgreSQL, Redis і Docker, готовим до розгортання в реальному бізнес-середовищі [34]. Пояснювальна записка складається з чотирьох розділів — аналізу предметної області, проєктування системи, програмної реалізації та планування і економічного обґрунтування проєкту [4, 21, 22].

# РОЗДІЛ 1. АНАЛІЗ ТА ОПИС СФЕРИ ЗАСТОСУВАННЯ. ПОСТАНОВКА ЗАДАЧІ

## 1.1 Характеристика об'єкта дослідження

Морська логістика забезпечує левову частку міжнародного товарообігу, але більшість процесів у цій галузі досі залишаються ручними і непрозорими для клієнта [23, 25]. Компанії, що займаються зовнішньою торгівлею, часто стикаються з тривалими погодженнями, помилками в документах і відсутністю зручного цифрового інструменту для самостійної роботи із замовленням [1,3]. Саме ця ситуація стала відправною точкою для розробки платформи Logistics SWay.

Logistics SWay — це веб-платформа для оренди морських контейнерів із вбудованою підтримкою IoT-технологій. Вона орієнтована на підприємства і підприємців, які регулярно займаються міжнародними перевезеннями і хочуть керувати своїми замовленнями самостійно — без постійних дзвінків менеджеру і очікування відповідей. Клієнт може обрати контейнер, оформити оренду, відстежувати переміщення вантажу і отримувати актуальні дані з датчиків прямо через особистий кабінет.

Вибір саме цього підприємства як об'єкта дослідження пов'язаний із тим, що ринок оренди контейнерів активно зростає, а цифрових рішень із повноцінним функціоналом на ньому критично мало [24]. Більшість існуючих платформ зупиняються на рівні інформаційного сайту з формою зворотного зв'язку — і не дають клієнту жодних інструментів для реальної взаємодії з сервісом.

Впровадження такої системи дає підприємству змогу скоротити операційні витрати за рахунок автоматизації рутинних дій, отримувати актуальні дані про стан контейнерів через IoT-датчики, впорядкувати документообіг із митними і фінансовими структурами та підвищити довіру клієнтів через прозорість і зручність роботи з платформою [2, 6, 8].

Враховуючи зазначене, об'єктом дослідження є інформаційна система Logistics SWay, призначена для автоматизації оренди контейнерів і дистанційного моніторингу стану вантажів у сфері міжнародних морських перевезень [1].

## 1.2 Організаційна структура підприємства, підрозділу

Logistics SWay спеціалізується на організації контейнерних перевезень і

супроводі міжнародних логістичних операцій. Компанія має чотири основні блоки — адміністрацію, бухгалтерію, логістичний департамент і технічну службу див. рис. 1.1.

Адміністрація координує роботу всієї компанії і ухвалює ключові організаційні рішення. До неї входять кадрова служба, відділ забезпечення, управління ризиками, документообіг і юридичний відділ. Кадровики відповідають за підбір і введення в роботу нових співробітників, юристи супроводжують договори і стежать за відповідністю чинному законодавству.

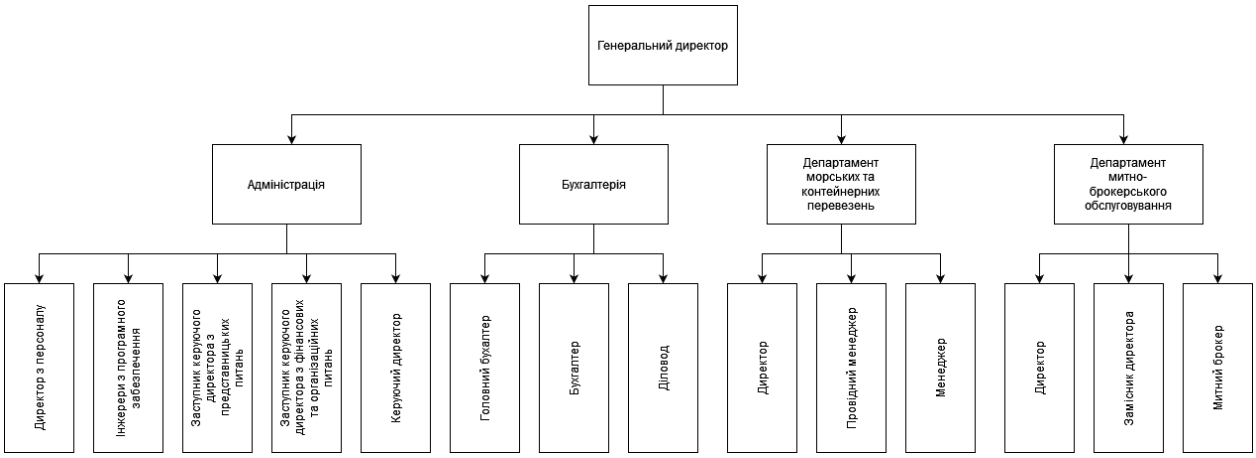



Рис. 1.1 — Рівень організаційної структури підприємства

Бухгалтерія веде фінансовий облік підприємства — контролює надходження і витрати, готує звітність і виконує податкові зобов'язання. Відділ аудиту регулярно проводить внутрішні перевірки, що допомагає вчасно виявляти розбіжності у фінансових операціях.

Департамент логістики та контейнерних перевезень — це серце операційної діяльності компанії. Він відповідає за планування маршрутів, узгодження відправлень і підтримку зв'язку з міжнародними партнерами. Диспетчери відстежують рух контейнерів, аналітики допомагають обирати оптимальні шляхи, менеджери взаємодіють із клієнтами.

Департамент технічного обслуговування стежить за станом контейнерного парку. Інженерна служба перевіряє обладнання перед відправленням, а ремонтний підрозділ усуває несправності і проводить регламентні роботи

### 1.3 Функції та завдання підрозділу

Департамент морських та контейнерних перевезень є основним операційним підрозділом Logistics SWay. Його робота пов’язана з організацією перевезень,

плануванням маршрутів, контролем руху контейнерів і взаємодією з клієнтами, портами, митними службами та партнерами.

До основних функцій департаменту належать планування та координація морських і контейнерних перевезень відповідно до встановлених термінів, умов договорів і вимог клієнтів. Підрозділ визначає оптимальні маршрути з урахуванням часу доставки, вартості перевезення, портових процедур і митного оформлення [23]. Крім того департамент бере участь у підготовці та перевірці транспортної документації, серед іншого вантажних накладних, договорів, дозволів і страхових документів.

Окрему роль у роботі підрозділу відіграє менеджерський відділ, який підтримує зв’язок із клієнтами, узгоджує умови перевезення, інформує про статус доставки та допомагає вирішувати проблемні ситуації. Диспетчерська служба контролює переміщення контейнерів і стежить за дотриманням графіків. Водночас департамент веде облік контейнерного парку, контролює наявність контейнерів, їхній технічний стан і впровадження в межах поточних замовлень.

Основні завдання підрозділу полягають у забезпеченні своєчасного перевезення вантажів, зменшенні витрат на логістичні операції та підвищенні прозорості роботи з контейнерами. Для цього департамент використовує автоматизовані інструменти відстеження, обробки документації та обміну інформацією між підрозділами. Важливим завданням до того ж є оцінка операційних ризиків, з-поміж іншого можливих затримок, погодних умов, митних обмежень або технічних проблем із контейнерами.

Для виконання своїх функцій департамент постійно взаємодіє з іншими структурними підрозділами підприємства. Бухгалтерія отримує інформацію про виконані перевезення для виставлення рахунків і контролю оплат. Митно-брокерський департамент супроводжує оформлення вантажів, а IT-відділ забезпечує технічну підтримку автоматизованих процесів. Загальний опис таких взаємозв’язків наведено в таблиці 1.1.

Таблиця 1.1 — Опис взаємозв’язків між підрозділами підприємства

| Підрозділ 1 | Підрозділ 2 | Взаємозв’язок |
|---|---|---|
| Департамент морських та контейнерних перевезень | Бухгалтерія | Передача інформації про виконані перевезення для оформлення оплат і виставлення рахунків |
| Департамент морських та контейнерних перевезень | Департамент митно-брокерського обслуговування | Координація митного оформлення контейнерів до та після перевезення |
| Адміністрація | Департамент морських та контейнерних перевезень | Погодження планів перевезень, кадрових рішень і бюджету підрозділу |
| Департамент морських та контейнерних перевезень | ІТ-відділ | Технічна підтримка автоматизації процесів управління перевезеннями |
| Департамент морських та контейнерних перевезень | Відділ персоналу | Оформлення документів щодо найму, переведення працівників і вирішення кадрових питань |
| Департамент митно-брокерського обслуговування | Бухгалтерія | Передача даних для оформлення митних платежів і бухгалтерських операцій |
| Адміністрація | Усі департаменти | Контроль виконання завдань, погодження бюджетів і загальне управління роботою підприємства |
| Бухгалтерія | Усі департаменти | Облік фінансових операцій, контроль витрат, митних платежів і заробітної плати |

## 1.4 Дерево цілей

На рис. 1.2 наведено дерево цілей проєкту — від загальної ідеї до конкретних функцій, які система має реалізувати.

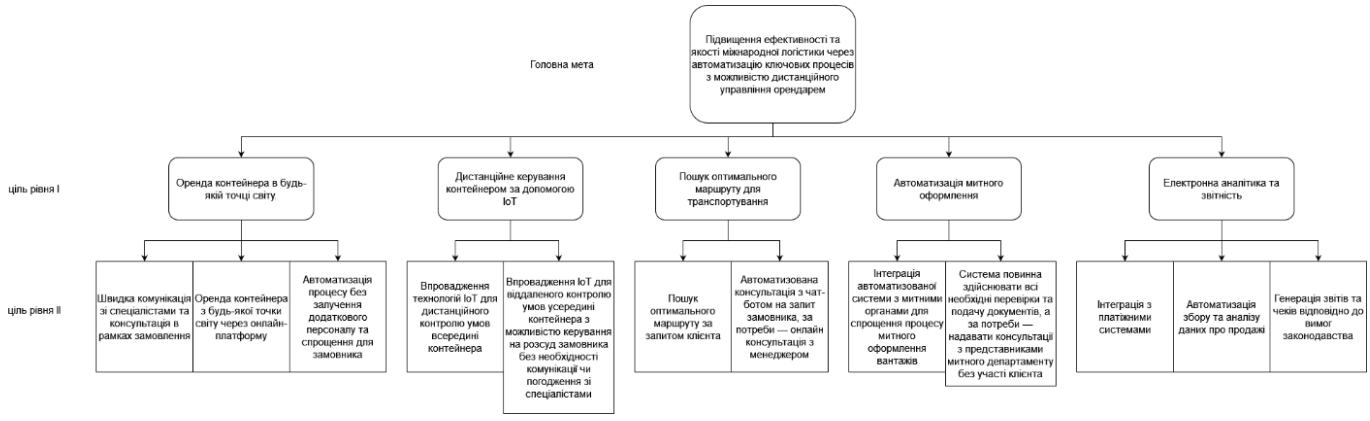



Рис. 1.2 — Дерево цілей проєкту

Загальна ідея проста: морська логістика досі переповнена ручними операціями, і більшість із них можна прибрати або суттєво скоротити за допомогою цифрових інструментів [3]. Саме навколо цього і будується дерево цілей платформи Logistics SWay.

Перше що потрібно клієнту — можливість самостійно оформити замовлення онлайн, без дзвінків і очікування відповіді менеджера. Вибрав контейнер, вказав строки і регіон — і процес запущено. Паралельно система має тримати під контролем умови всередині контейнера: температуру, вологість та інші параметри, які напряму впливають на збереження вантажу в дорозі. Для деяких категорій товарів це питання не зручності, а прямих збитків якщо щось піде не так.

Окремим завданням є маршрутизація — система сама аналізує варіанти і підбирає шлях з огляду на вартість, тривалість і портові особливості, замість того щоб менеджер робив це вручну. Поруч із цим платформа бере на себе частину роботи з митною документацією — готує дані і передає їх відповідальним фахівцям, що зменшує ризик помилок на цьому етапі [15]. І нарешті, все що відбувається в системі — перевезення, платежі, стан контейнерів — накопичується і перетворюється на аналітику, яка дає керівництву реальну картину роботи підприємства.

Кожен із цих напрямків вирішує конкретну задачу, і разом вони формують логічну основу для подальшого проєктування системи.

## 1.5 Аналіз існуючих рішень

У межах дослідження проаналізовано п'ять українських логістичних платформ, які працюють у сфері міжнародних і контейнерних перевезень. Мета — зрозуміти що вже є на ринку, де є прогалини і що саме має закрити Logistics SWay.

Translogistic.ua — типовий представник більшості розглянутих сайтів. Є опис послуг, є контакти, але далі цього справа не йде. Ні калькулятора, ні онлайн-бронювання, ні особистого кабінету — щоб щось замовити, потрібно писати або телефонувати менеджеру на рис. 1.3.

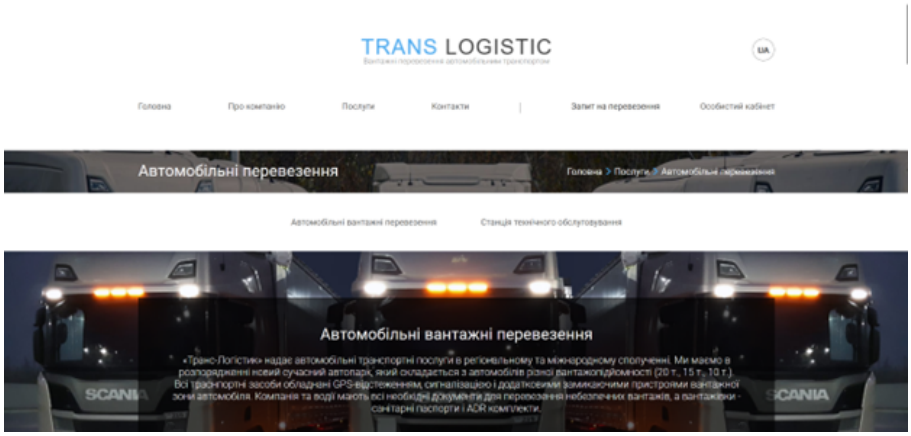



Рис. 1.3 — Головна сторінка сайту Translogistic.ua

LogisticPlus.com.ua має зрозумілу структуру і широкий перелік послуг, але детальної інформації по кожному напрямку бракує. Самостійно розрахувати вартість або відстежити замовлення теж не вийде — все одно через менеджера на рис. 1.4.

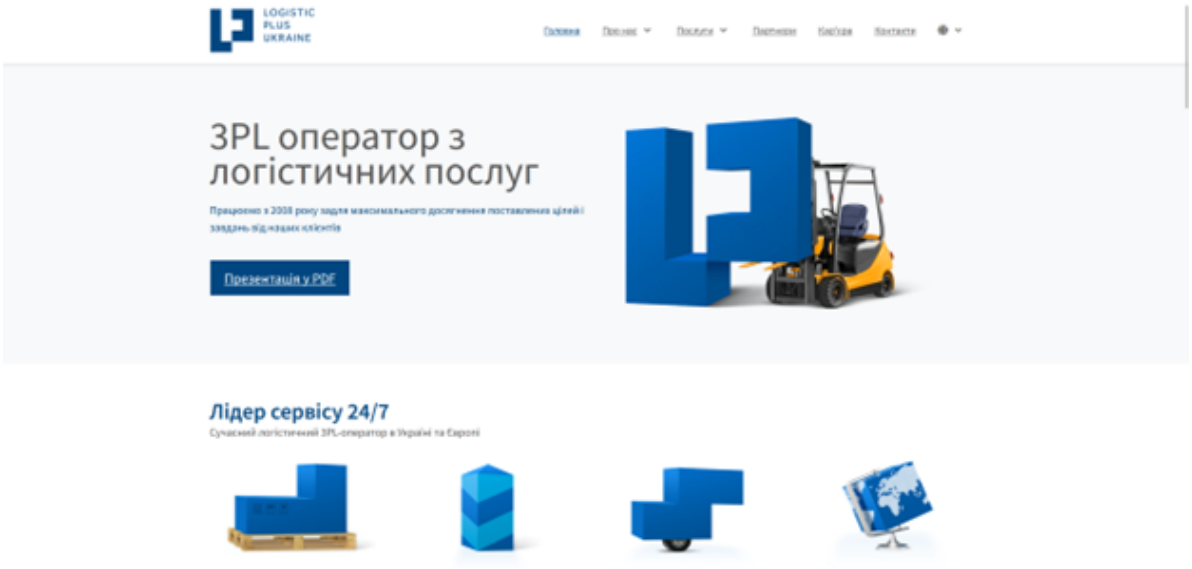



Рис. 1.4 — Головна сторінка сайту LogisticPlus.com.ua

Utec.ua виглядає найпродуманіше з погляду навігації — окремі розділи для різних видів перевезень полегшують пошук потрібної інформації. Онлайн-бронювання частково реалізовано, але особистого кабінету немає, тому повного контролю над замовленням клієнт так і не отримує на рис. 1.5.

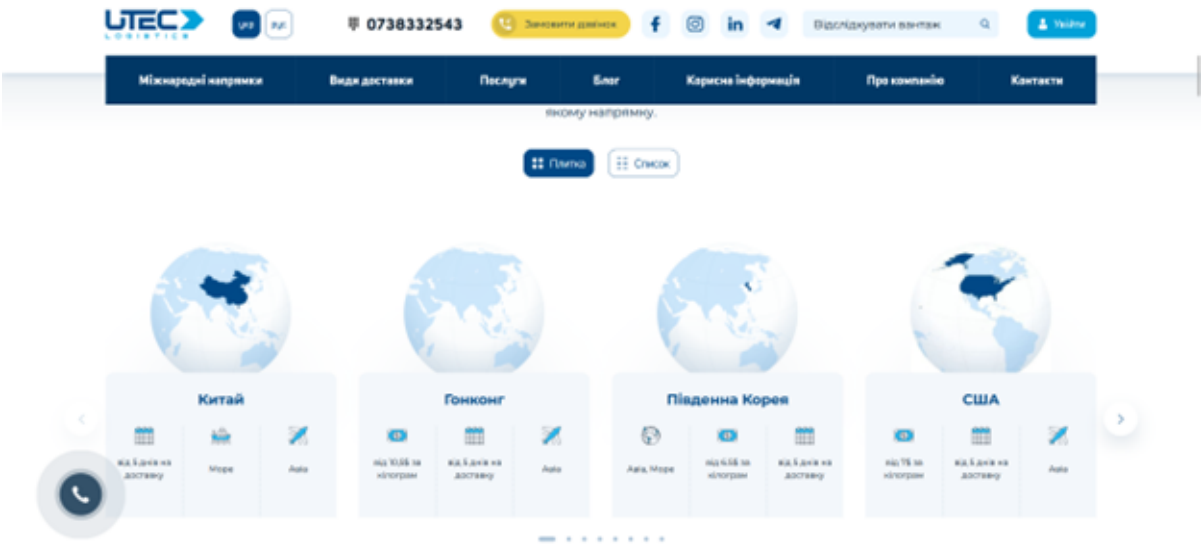



Рис. 1.5 — Головна сторінка сайту Utec.ua

Talman.com.ua — мінімалістичний і зручний для читання, але фактично це лише візитка компанії. Все зводиться до заявки і подальшого спілкування з менеджером, жодних інструментів для самостійної роботи немає на рис. 1.6.

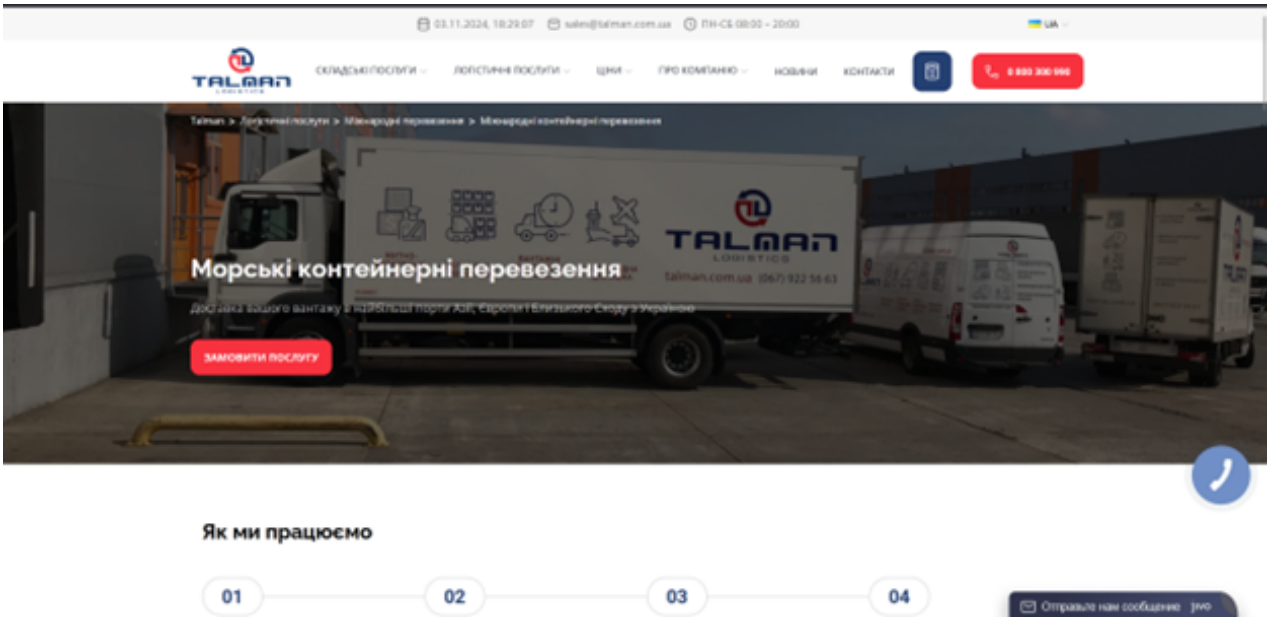



Рис. 1.6 — Головна сторінка сайту Talman.com.ua

Zammler.com.ua найбільш насичений контентом і має калькулятор — це виділяє його серед інших. Водночас інтерфейс перевантажений анімаціями і візуальними елементами, що заважає на рис. 1.7.

Рис. 1.7 — Сторінка контейнерних перевезень сайту Zammler.com.ua

За результатами аналізу жодна з платформ не дає клієнту можливості самостійно пройти весь шлях від вибору контейнера до контролю перевезення в одному місці [2]. Усі вони зупиняються на рівні інформаційного сайту з формою зворотного зв'язку. Саме цю прогалину і закриває Logistics SWay — особистий кабінет, онлайн-оренда, IoT-моніторинг і фінансова аналітика в єдиному середовищі таб.1.2. Додатково зібрана

статистика дає підприємству змогу аналізувати завантаженість маршрутів і ефективність виконаних перевезень.

Таблиця 1.2 — Порівняння функціональності логістичних платформ

| Сайт | Онлайн-калькулятор | Онлайн-бронювання | Розділ FAQ | Клієнтський кабінет | Контактна інформація | Опис послуг | Інтерактивність | Навігація |
|---|---|---|---|---|---|---|---|---|
| Translogistic.ua | Ні | Ні | Ні | Ні | Так | Так | Низька | Добра |
| LogisticPlus.com.ua | Ні | Ні | Ні | Ні | Так | Так | Низька | Добра |
| Utec.ua | Ні | Так | Так | Ні | Так | Так | Часткова | Відмінна |
| Talman.com.ua | Ні | Ні | Ні | Ні | Так | Так | Низька | Добра |
| Zammler.com.ua | Так | Ні | Ні | Ні | Так | Так | Часткова | Перевантажена |

## 1.6 Апаратні засоби IoT-системи

Щоб клієнт міг у реальному часі бачити що відбувається з його вантажем, кожен контейнер у межах системи Logistics SWay обладнується набором датчиків [2, 6, 7, 8]. Кожен із них відповідає за свій параметр і разом вони дають повну картину стану вантажу під час перевезення. Для передачі телеметрії в умовах відсутності мобільного зв'язку застосовується супутниковий канал Starlink [42].

Температуру всередині контейнера вимірює датчик DHT11 — якщо вона вийде за допустимі межі, система одразу надішле сповіщення клієнту [11]. Це критично для ліків, їжі або електроніки, які легко псуються від перегріву чи переохолодження. За вологістю стежить датчик HDS10 — надлишок вологи може призвести до конденсату і зіпсованого товару, особливо якщо всередині папір, тканина або металеві деталі. Удари та тряску фіксує віброметр «Автон» — якщо контейнер різко вдарили або він трясеться сильніше норми, це одразу потрапляє до журналу подій. Датчик нахилу I2C АТВ відстежує положення контейнера і помічає небезпечні нахили, які бувають під час шторму або при неправильному завантаженні на судно. Відкриття дверей реєструє датчик Zigbee SEVEN HOME Z-7061 — якщо хтось відчинив контейнер у невідповідний момент, клієнт про це дізнається. Де саме знаходиться контейнер у будь-

який момент показує GPS-трекер Jointech JT704 — він відстежує маршрут і одразу помічає відхилення від запланованого шляху [7]. Датчик освітлення PHS-06 реагує на різку зміну рівня світла всередині, що може вказувати на несанкціонований доступ або пошкодження стінки контейнера. І нарешті, датчик $CO_2$ Ballu Z19 контролює склад повітря всередині — це особливо важливо для харчових або фармацевтичних вантажів, де стан газового середовища напряму впливає на збереження товару таб. 1.3 [8, 11, 15].

Усі ці показники автоматично надходять на платформу, зберігаються в базі даних і відображаються в особистому кабінеті клієнта [14, 42]. Якщо щось виходить за норму — система надсилає сповіщення і клієнт може відреагувати ще під час перевезення, а не після того як вантаж вже прибув із проблемою.

Таблиця 1.3 — Апаратні компоненти IoT-моніторингу контейнера

| **Пристрій** | **Контрольований параметр** | **Призначення** |
|---|---|---|
| DHT11 | Температура | Контроль температурного режиму |
| HDS10 | Вологість | Виявлення ризику конденсату |
| Датчик вібрації | Удари та вібрація | Контроль механічних впливів |
| Датчик нахилу | Положення контейнера | Фіксація нахилу або перекидання |
| Zigbee-датчик дверей | Відкриття дверей | Контроль доступу |
| GPS-трекер | Геолокація | Відстеження маршруту |
| PHS-06 | Освітлення | Виявлення відкриття або пошкодження |
| $CO_2$-датчик | Концентрація $CO_2$ | Контроль газового середовища |

## 1.7 Побудова моделі системи у нотації BPMN

BPMN-діаграму процесу оренди контейнера наведено в додатку Б на рис. Б.1 [19]. Вона показує як замовлення проходить від першого запиту клієнта до завершення оренди — включно з діями системи, менеджерів, митного брокера, бухгалтера і власника контейнера.

Модель побудовано в одному пулі з окремими доріжками для кожного учасника. Процес стартує з подання клієнтом запиту — система обробляє дані, перевіряє доступні контейнери і формує пропозицію. Після вибору клієнтом відповідного

варіанта замовлення переходить до погодження між учасниками. Митний брокер опрацьовує документацію, менеджер організовує транспортування з урахуванням маршруту і строків, бухгалтер підтверджує оплату. Власник контейнера або відповідальна особа підтверджує що обладнання справне і готове до передачі. Заступник керівника контролює організаційну і фінансову сторону процесу.

У моделі передбачено кілька точок де процес може змінювати напрямок — погодження замовлення, підтвердження оплати і завершення оренди. Якщо все пройшло успішно — система оновлює статус, сповіщає учасників і зберігає дані для подальшої аналітики і звітності.

BPMN-модель наочно показує логіку процесу і допомагає визначити де ручні дії можна замінити автоматичними.

**Висновки до першого розділу**

У першому розділі досліджено предметну область морських контейнерних перевезень і визначено які завдання має вирішувати платформа Logistics SWay. Вивчено організаційну структуру підприємства і функції його підрозділів — стало зрозуміло що ручний обмін інформацією між ними сповільнює обробку замовлень і збільшує ймовірність помилок.

На основі дерева цілей визначено п'ять ключових напрямків системи: онлайн-оренда контейнерів, IoT-моніторинг умов перевезення, підбір маршруту, підтримка митного оформлення і формування електронної звітності.

Аналіз п'яти конкурентних платформ — Translogistic.ua, LogisticPlus.com.ua, Utec.ua, Talman.com.ua і Zammler.com.ua — показав що жодна з них не дає клієнту повноцінного набору інструментів для самостійної роботи із замовленням. Найчастіше відсутні особистий кабінет, онлайн-бронювання, IoT-моніторинг і зручна аналітика.

Описано апаратну складову системи — вісім типів датчиків для контролю температури, вологості, ударів, нахилу, стану дверей, освітлення, рівня $CO_2$ і геопозиції контейнера.

Побудована BPMN-модель процесу оренди показала послідовність дій між усіма учасниками і визначила етапи де автоматизація дасть найбільший ефект. Отримані результати стали основою для проєктування архітектури системи в наступному розділі.

# РОЗДІЛ 2. ПРОЄКТУВАННЯ КОНЦЕПТУАЛЬНОЇ, ЛОГІЧНОЇ ТА ФІЗИЧНОЇ МОДЕЛЕЙ БАЗИ ДАНИХ

## 2.1 Створення діаграм системи у нотаціях IDEF0, IDEF3 та DFD

Під час проєктування системи було побудовано діаграми у нотаціях IDEF0, IDEF3 та DFD [1, 32]. Вони потрібні для того, щоб показати роботу системи з різних сторін: загальну структуру процесів, послідовність виконання дій та рух інформації між учасниками оренди контейнерів.

IDEF0-діаграма функціонує для подання загальної функціональної моделі системи [32]. Вона показує, які вхідні дані використовуються в процесі оренди контейнера, які правила або обмеження впливають на виконання дій, які ресурси залучаються до роботи та який результат формується після завершення процесу. Такий тип діаграми дає змогу побачити систему не через окремі операції, а як цілісний процес, що поєднує клієнта, платформу, працівників підприємства та зовнішні організації.

На рис. 2.1 подано DFD-діаграму, яка показує обмін даними під час перевірки можливості прийняття контейнера в оренду [32]. У цьому процесі важливо зібрати підтвердження від кількох сторін, оскільки оренда контейнера залежить не лише від бажання клієнта, а й від технічної доступності контейнера, митних вимог і підтвердження з боку власника.

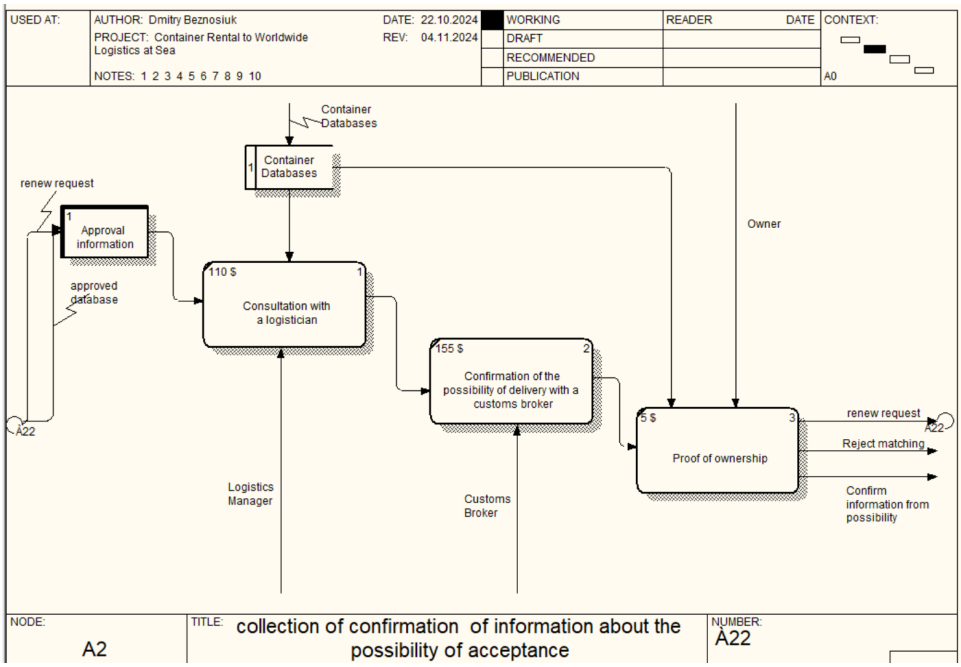



Рис. 2.1— DFD-діаграма процесу збору підтверджень щодо оренди контейнера

Перший етап пов’язаний із перевіркою контейнера логістом. Він порівнює запит клієнта з інформацією з бази контейнерів і даними, отриманими через автоматизовану платформу. Після цього формується попереднє підтвердження або запит на уточнення даних. Вартість цього етапу становить 110 USD.

На другому етапі до процесу залучається митний брокер. Він перевіряє, чи відповідають умови перевезення чинним митним вимогам. Якщо в документах або умовах транспортування є невідповідності, замовлення може бути повернене на доопрацювання або відхилене. Вартість цього етапу становить 155 USD.

Третій етап передбачає підтвердження права власності або права розпорядження контейнером. Власник контейнера перевіряє можливість його передачі в оренду. Якщо підтвердження не надходить, процес не може перейти до наступного етапу. Адміністративні витрати на цьому кроці становлять 5 USD.

Результатом процесу є або підтвердження можливості оренди контейнера, або відмова через невідповідність вимогам. Якщо на одному з етапів виникає потреба уточнити дані, запит повертається на повторну обробку.

На рис. 2.2 наведено IDEF3-діаграму, яка описує послідовність дій під час узгодження оренди контейнера з урахуванням вимог різних учасників процесу [32]. Ця модель більше зосереджена не на потоках даних, а на тому, у якій послідовності виконуються дії та які умови впливають на перехід між етапами.

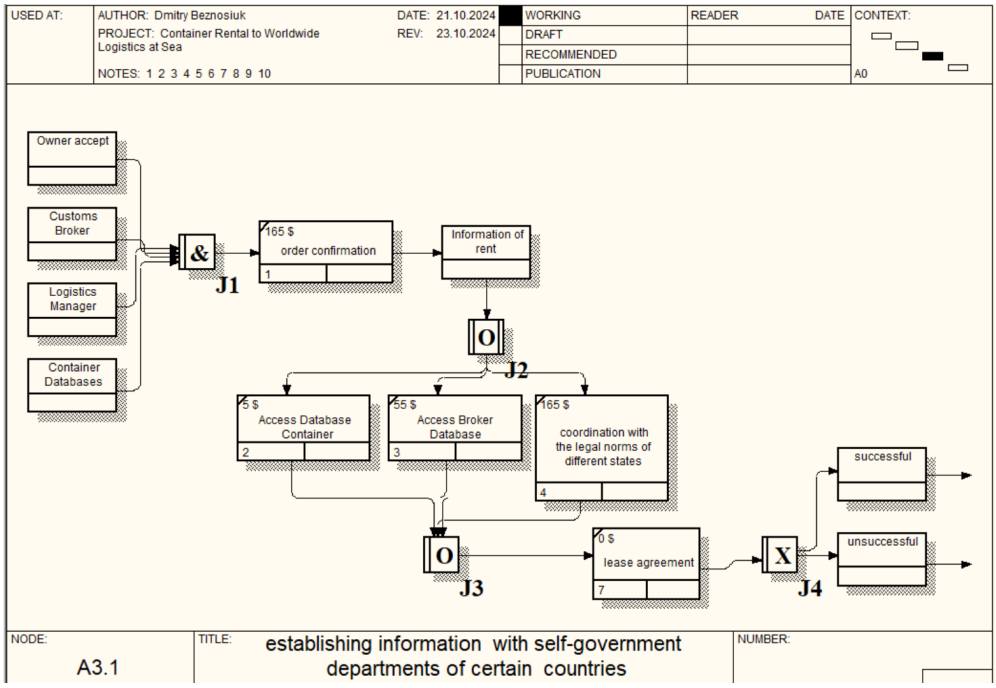



Рис. 2.2 — IDEF3-діаграма процесу узгодження оренди контейнера

Перший блок відповідає за підтвердження замовлення. На цьому етапі враховуються дані від власника контейнера, митного брокера, менеджера з логістики та бази контейнерів. Вузол J1 об’єднує ці потоки перед переходом до наступного кроку. Вартість цього етапу становить 165 USD.

Далі система звертається до бази даних контейнерів і бази митних брокерів. Це потрібно для перевірки доступності контейнера, його характеристик, а паралельно вибору відповідального брокера для супроводу митних процедур. Вартість цих дій становить 5 USD та 55 USD відповідно.

Наступний блок пов’язаний з узгодженням правових вимог різних країн. Це важливий етап для міжнародної оренди, оскільки перевезення може проходити через кілька юрисдикцій. Вузол J2 об’єднує необхідні вхідні дані перед початком правового погодження. Вартість цього етапу становить 165 USD.

Завершальним етапом є підписання договору оренди контейнера. У блоці «Lease Agreement» фіксується завершення попередніх погоджень, після чого договір підтверджує передачу контейнера клієнту на визначених умовах. Вузол J3 показує завершення процесу погодження.

У результаті процес може завершитися двома способами: успішним оформленням оренди або відмовою, якщо на одному з етапів виявлено невідповідність вимогам. Побудовані діаграми допомагають зрозуміти, як саме проходить обробка замовлення, які дані передаються між учасниками та на яких етапах система може скоротити кількість ручних дій [1, 21].

## 2.2 Побудова концепції автоматизованої системи в нотації UML

Для опису роботи системи Logistic SWay було побудовано три UML-діаграми: діаграму прецедентів, діаграму діяльності та діаграму послідовності [13, 20]. Вони показують систему з різних сторін: які функції доступні користувачам, як проходить процес оренди контейнера та як між собою взаємодіють учасники під час обробки замовлення.

### 2.2.1 Діаграма прецедентів Use Case

На рис. 2.3 показано основні можливості системи для двох груп користувачів: клієнта та персоналу. Клієнт працює з функціональним блоком, який охоплює перегляд інформації про послуги, редагування профілю, перегляд власних контейнерів,

оформлення оренди, доступ до статистики та пошук маршруту на карті. У тому числі передбачено можливість надіслати звернення або повідомлення персоналу через функцію зворотного зв’язку.

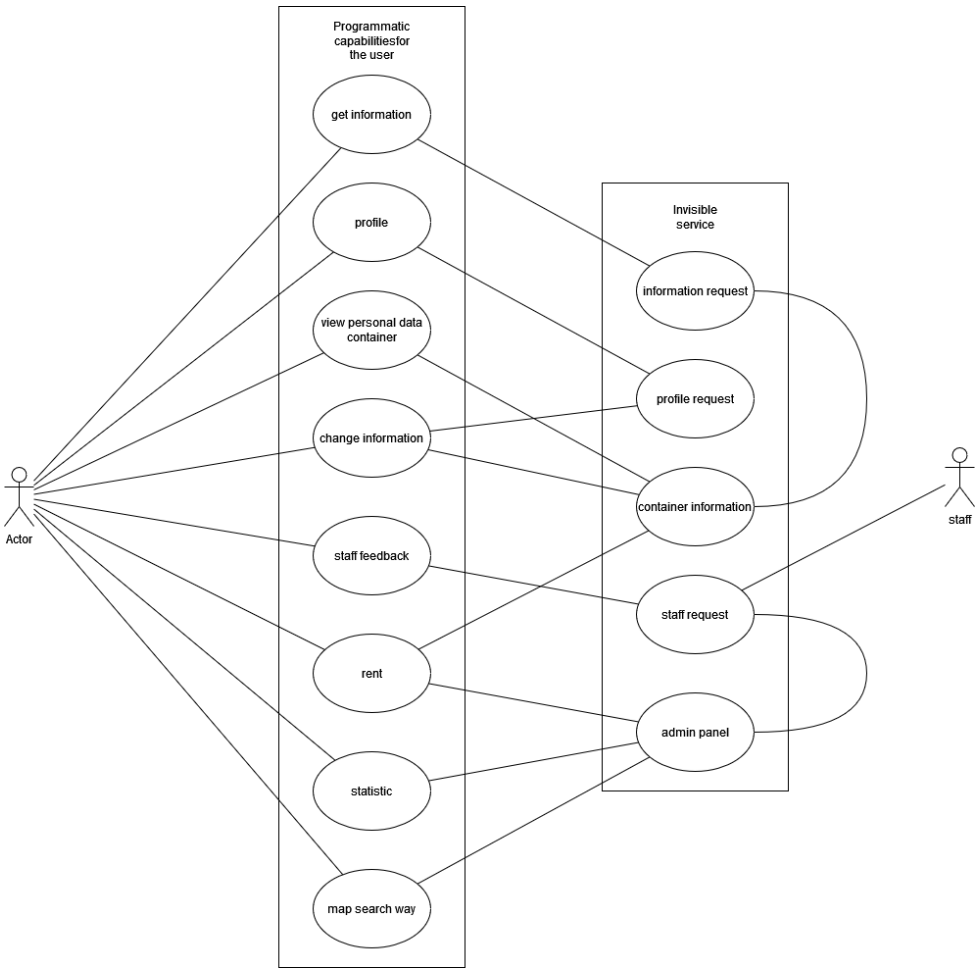



Рис. 2.3 — UML-діаграма прецедентів інформаційної системи оренди контейнерів

Окремо на діаграмі виділено внутрішній сервісний блок, через який проходять основні запити користувача. До нього належать запити на отримання інформації, дані профілю, відомості про контейнери та звернення до персоналу. Працівники системи взаємодіють із цим блоком через адміністративну панель, де можуть переглядати, опрацьовувати та уточнювати отримані запити. Така побудова сприяє реалізації частині стандартних дій виконувати автоматично, а персонал залучати переважно тоді, коли потрібна перевірка або додаткове рішення.

### 2.2.2 Діаграма діяльності Activity Diagram

На рис. 2.4 наведено послідовність дій під час обробки запиту на оренду контейнера. Процес починається з того, що клієнт подає запит. Після цього система перевіряє, чи потрібні додаткові погодження. Якщо замовлення не потребує уточнень, воно одразу переходить до формування.

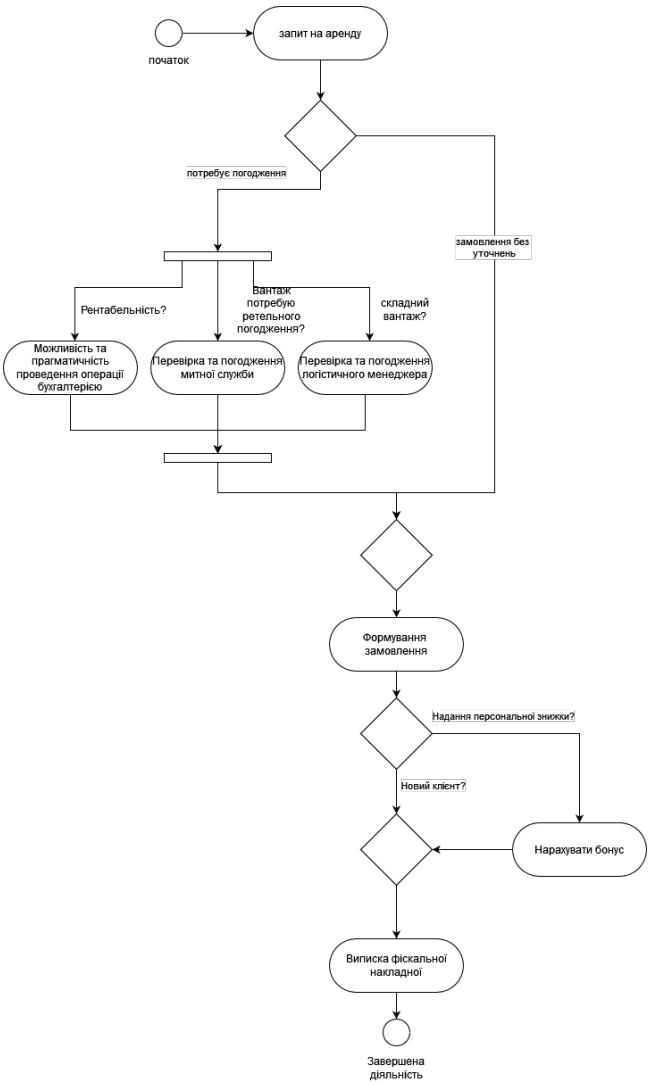



Рис. 2.4 — UML-діаграма діяльності інформаційної системи оренди контейнерів

Якщо погодження необхідне, система запускає кілька перевірок. Бухгалтерія оцінює фінансову доцільність операції, митна сторона перевіряє вимоги до вантажу, а логістичний менеджер аналізує складність перевезення. Після проходження потрібних перевірок система переходить до створення замовлення.

Далі перевіряється, чи має клієнт персональну знижку. Якщо клієнт новий, система може нарахувати бонус. Завершальним кроком є формування фінансового документа, який підтверджує проведення операції та закриває етап оформлення оренди.

### 2.2.3 Діаграма послідовності Sequence Diagram

На рис. 2.5 показано обмін повідомленнями між учасниками процесу оренди. Діаграми побудовано в середовищі draw.io [30]. У діаграмі беруть участь клієнт, база даних контейнерів, митний брокер, логіст, бухгалтер, власник контейнера та директор

з фінансів.

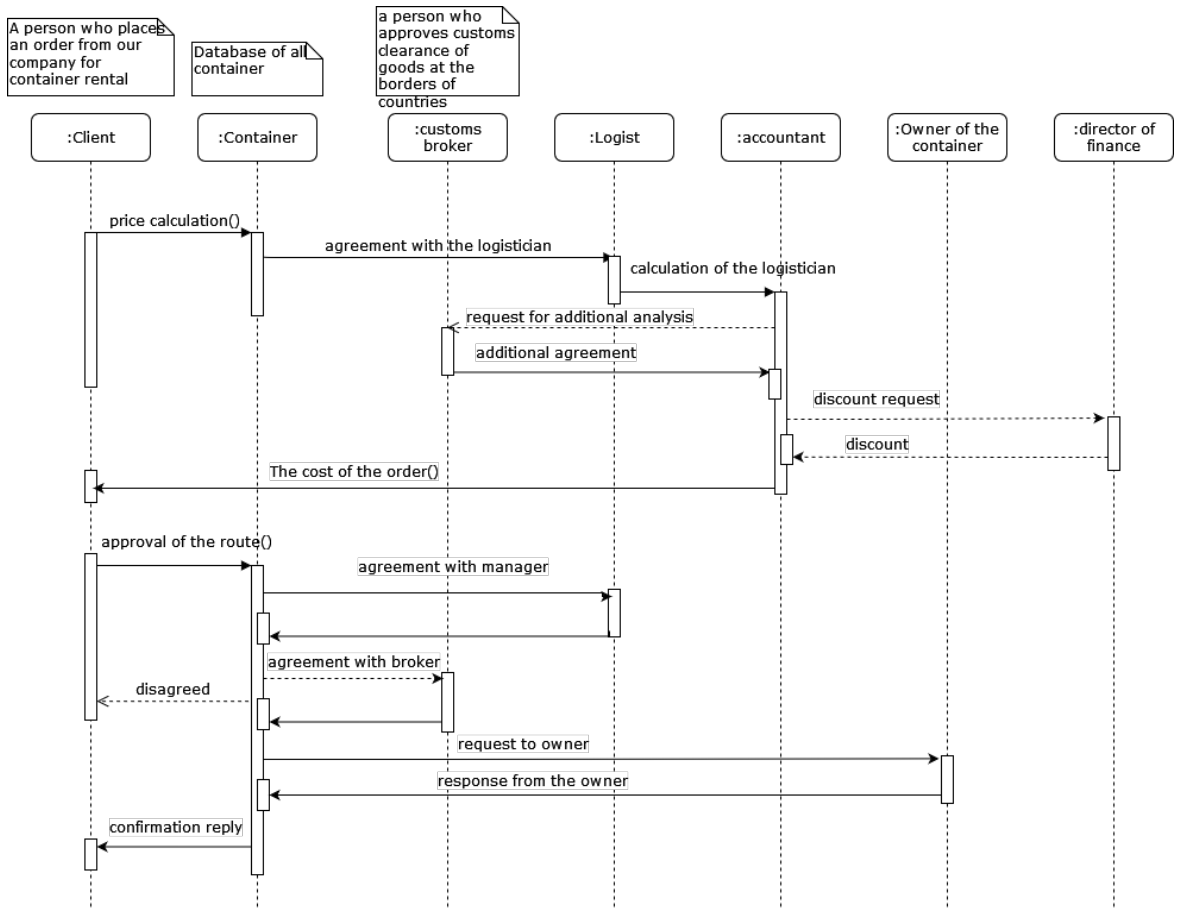



Рис. 2.5 — UML-діаграма послідовності інформаційної системи оренди контейнерів

Процес починається з того, що клієнт ініціює розрахунок вартості оренди. Запит передається логісту, який виконує розрахунок з урахуванням маршруту та інших умов перевезення. Якщо під час розрахунку виникає потреба у знижці, бухгалтер опрацьовує відповідний запит і, за необхідності, погоджує його з директором з фінансів. Після цього клієнту надсилається підсумкова вартість замовлення.

Наступним етапом є погодження маршруту. Система послідовно передає запит менеджеру та митному брокеру. Якщо маршрут або умови перевезення не відповідають вимогам, клієнт отримує повідомлення про відмову. Якщо погодження пройдено успішно, система звертається до власника контейнера для підтвердження його доступності. Після позитивної відповіді клієнт отримує остаточне підтвердження замовлення.

На рис. 2.6 наведено спрощений фрагмент діаграми класів платформи Logistics SWay. Програмну частину побудовано на Laravel за підходом MVC, що розділяє систему на чіткі рівні відповідальності [9, 12, 17, 34].

Рис. 2.6 — UML-діаграма класів системи

Моделі описують основні сутності — Rental, Container, User, Shipment, Route, Port та IoT-класи. Між ними визначено зв'язки «один до багатьох» і «багато до багатьох», що відображає реальну логіку: один клієнт може мати кілька замовлень, один маршрут — використовуватися в різних перевезеннях. Контролери приймають запити від користувача і передають їх до потрібних моделей або сервісів. Бізнес-логіку винесено в окремі сервісні класи — RentalPricingService розраховує вартість оренди, ShipmentProvisioningService готує відправлення, UserDashboardDataService формує дані для кабінету. Такий поділ робить код чистішим і не перевантажує контролери зайвою логікою [17, 18]. Класи запитів перевіряють коректність вхідних даних перед обробкою, а механізм подій автоматично запускає потрібні дії після зміни стану замовлення або надходження нових IoT-даних.

## 2.3 Концептуальна модель бази даних

На рис. 2.7 показано як влаштована база даних системи на рівні понять — без прив'язки до конкретної технології [10]. По суті це карта всіх об'єктів, з якими працює платформа, і того як вони між собою пов'язані.

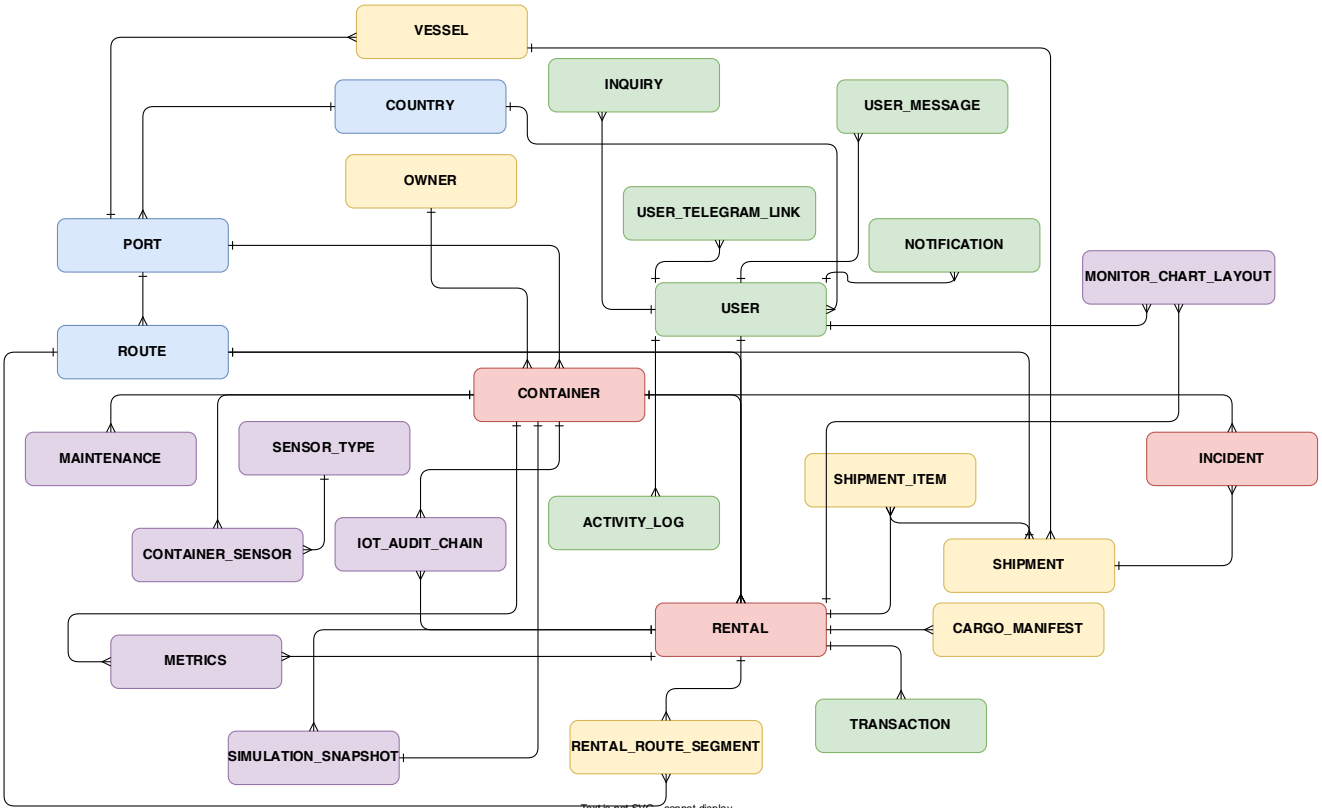



Рис. 2.7 — Концептуальна модель бази даних системи Logistics SWay

В основі лежить географія — країни і порти. Порти є відправними і кінцевими точками будь-якого маршруту, а країна впливає на податкову ставку при розрахунку вартості оренди.

Навколо сутності User побудовано весь користувацький блок. Це і клієнти, і персонал платформи. З ними пов'язані сповіщення, переписка, звернення від потенційних клієнтів, підключення Telegram-бота і журнал дій [44].

Контейнерний парк описують чотири сутності — власники, самі контейнери, їхня сервісна історія і підключені датчики з їх типами. Завдяки такому поділу кожен контейнер знає хто ним володіє, коли його обслуговували і які сенсори на ньому встановлені.

Логістичний блок побудований навколо оренди. Rental — це центральна сутність всього операційного процесу, яка тягне за собою маршрут, судно, порти, деталізацію по ділянках, фактичне відправлення і вантажну документацію [10,12].

Останній блок об'єднує IoT-моніторинг, фінанси і нештатні ситуації. Metrics збирає телеметрію з датчиків, IoTAuditChain захищає її від підробки, Transaction фіксує платежі, а Incident реєструє все що пішло не так під час перевезення.

## 2.4 Логічна та фізична модель бази даних

Логічна модель — це наступний крок після концептуальної [10]. Тут сутності вже перетворюються на конкретні таблиці з полями, ключами і зв'язками між ними. Фізична модель іде ще далі і враховує особливості обраної СУБД — у нашому випадку PostgreSQL [39]. На цьому рівні визначаються типи даних, індекси, правила каскадного видалення і механізми оптимізації. Оскільки обидва рівні тісно пов'язані, у цьому підрозділі їх розглянуто разом.

На рис. 2.8 наведено повну реляційну схему системи, яка налічує 35 таблиць, повніший опис полів основних таблиць наведено в додатку В у табл. В.1. Структуру нормалізовано, що усуває дублювання даних. Більшість таблиць мають сурогатний первинний ключ типу BIGSERIAL, зовнішні ключі задають правила взаємодії між пов'язаними записами.

Рис. 2.8 — Реляційна модель бази даних системи Logistics SWay

Географічна основа — таблиці countries і ports. Країни зберігають ISO-коди і податкові ставки для розрахунку вартості оренди, порти — координати і належність до країни.

Блок користувачів побудований навколо таблиці users, яка описує і клієнтів, і персонал через поле role. Підтримується м'яке видалення через deleted_at. Telegram-інтеграція реалізована через user_telegram_links з одноразовими кодами підтвердження. Сесії і відновлення паролів закривають sessions і password_reset_tokens.

Контейнерний парк охоплює таблиці owners, containers, maintenances, sensor_types і container_sensors. Кожен контейнер знає хто ним володіє, яка в нього сервісна історія і які датчики підключені. Унікальне обмеження на пару «контейнер — тип датчика» унеможливлює дублювання сенсорів.

Логістичний блок — найбільший. Таблиця routes описує маршрути між портами з геометрією шляху у форматі JSON, vessels зберігає дані суден з IMO-номером і технічними обмеженнями. Центральна таблиця rentals фіксує оренду і пов'язує клієнта, контейнер, маршрут, вантаж і умови доставки — для неї створено складені індекси для пришвидшення типових запитів. rental_route_segments деталізує маршрут по ділянках, shipments і shipment_items описують фактичне перевезення, cargo_manifests зберігає вантажну документацію з HS-кодами, incidents фіксує нештатні ситуації.

IoT-блок складається з кількох таблиць. metrics накопичує телеметрію і реалізована як партиційована — це критично при великих обсягах записів [39]. iot_audit_chain захищає дані криптографічним ланцюжком хешів: кожен запис містить хеш поточного і попереднього рядка, тому підробити телеметрію після збереження неможливо [6, 15, 39]. container_simulation_snapshots зберігає стан симуляції датчиків, monitor_chart_layouts — індивідуальні налаштування дашборду.

Фінансовий і комунікаційний блоки закривають таблиці transactions, notifications, user_messages і inquiries. Журналювання дій і запитів реалізовано через activity_logs і request_logs. Службові таблиці cache, jobs і failed_jobs забезпечують роботу черг і кешування — вони не описують предметну область, але необхідні для стабільної роботи платформи див. таб. 2.1.

Таблиця 2.1 — Таблиця полів в базі даних

| Таблиця | Основні поля | Типи даних | Призначення |
|---|---|---|---|
| **countries** | id, name, iso_code, phone_code, interest_tax | BIGSERIAL, VARCHAR(100), VARCHAR(3), VARCHAR(10), NUMERIC(5,2) | Довідник країн з ISO-кодом та податковою ставкою для розрахунку вартості оренди. |
| **users** | id, first_name, last_name, email, role, account_status, country_id, bonus_type, bonus_value, deleted_at | BIGSERIAL, VARCHAR(50), VARCHAR(50), VARCHAR(255), VARCHAR(50), VARCHAR(50), BIGINT (FK), VARCHAR(20), NUMERIC(12,2), TIMESTAMP | Користувачі системи — клієнти та персонал. Підтримується м'яке видалення та бонусна система. |
| **owners** | id, name, email, phone_number, deleted_at | BIGSERIAL, VARCHAR(100), VARCHAR(254), VARCHAR(20), TIMESTAMP | Власники контейнерів. Підтримується м'яке видалення. |
| **ports** | id, country_id, name, city, latitude, longitude | BIGSERIAL, BIGINT (FK), VARCHAR(100), VARCHAR(100), NUMERIC(10,7), NUMERIC(10,7) | Морські порти з географічними координатами для маршрутизації та відображення на карті. |
| **vessels** | id, name, imo_number, capacity_teu, status, current_port_id, berth_busy_until, out_of_service_until | BIGSERIAL, VARCHAR(100), VARCHAR(20), INTEGER, VARCHAR(50), BIGINT (FK), TIMESTAMPTZ, TIMESTAMPTZ | Судна з IMO-номером, місткістю та часовими мітками технічних обмежень. |
| **containers** | id, serial_number, type, width, length, height, max_weight, iot_active, current_status, owner_id, current_port_id | BIGSERIAL, VARCHAR(50), VARCHAR(50), NUMERIC(8,2), NUMERIC(8,2), NUMERIC(8,2), NUMERIC(10,2), BOOLEAN, VARCHAR(50), BIGINT (FK), BIGINT (FK) | Контейнери з габаритами, ознакою IoT та поточним статусом. Прив'язані до власника і порту. |

З точки зору оптимізації продуктивності для кожного атрибута підібрано тип даних відповідно до його семантики — NUMERIC з фіксованою точністю для фінансових розрахунків, TIMESTAMPTZ для часових міток з урахуванням часових зон, JSON для гнучких структур метаданих. На таблицях rentals та metrics визначено складені індекси за найбільш вживаними комбінаціями полів пошуку та фільтрації, що суттєво прискорює аналітичні запити. Таблиця metrics реалізована як партиційована (PARTITION BY LIST), що надає змогу ефективно зберігати та опрацьовувати великі обсяги телеметричних даних без деградації продуктивності.

Цілісність даних підтримується за допомогою зовнішніх ключів і правил CASCADE, SET NULL або RESTRICT, які застосовуються залежно від логіки зв’язку між таблицями. Для важливих полів використано обмеження NOT NULL та UNIQUE, що допомагає уникати порожніх або дубльованих значень там, де це може порушити роботу системи. У таблицях users, owners та containers передбачено м’яке видалення через поле deleted_at, завдяки чому запис можна вивести з активного стану без втрати пов’язаних даних.

Захист конфіденційних даних реалізовано через зберігання паролів у хешованому вигляді, а IoT-аудит — через криптографічний ланцюжок хешів у таблиці iot_audit_chain, що унеможливлює непомітну модифікацію телеметричних записів. Для верифікації підключення Telegram-бота застосовано одноразові токени з обмеженим терміном дії.

Структура бази даних поділена на кілька функціональних блоків: географічні довідники, користувачі, контейнерний парк, логістика, IoT-моніторинг, фінанси та комунікації [4]. Такий поділ робить схему зрозумілішою, спрощує подальшу підтримку системи та допомагає здійснити додавати нові функції без значної перебудови всієї бази даних.

**Висновки до другого розділу**

У другому розділі систему спроєктовано від верхнього рівня опису процесів до конкретної схеми бази даних. Спочатку через діаграми IDEF0, IDEF3 і DFD було розкладено процес оренди контейнера по кроках — хто що робить, яка інформація передається і де рішення приймаються вручну, а де їх можна передати системі. Це дало чітке розуміння того, як має працювати платформа зсередини.

UML-діаграми доповнили цю картину з різних боків: прецеденти показали функції для клієнта і персоналу, діаграма діяльності відобразила повний цикл замовлення, послідовність — як учасники взаємодіють між собою, а діаграма класів описала програмну структуру на базі MVC.

База даних спроєктована у три етапи — концептуальний, логічний і фізичний. На виході отримано 35 таблиць PostgreSQL, розбитих на функціональні блоки. Таблиця телеметрії партиційована щоб витримувати великі обсяги IoT-даних, цілісність забезпечується зовнішніми ключами і обмеженнями, а дані моніторингу захищено криптографічним ланцюжком — підробити їх після збереження неможливо.

# РОЗДІЛ 3. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

## 3.1 Мета програмного продукту

Галузь морської контейнерної логістики досі значною мірою залежить від ручної координації між замовником і менеджером. Уточнення наявності контейнера, очікування розрахунку вартості, окреме отримання документів і відсутність актуальної інформації про стан вантажу — це типовий досвід клієнта, який стикається з традиційним підходом до оформлення оренди. Logistics SWay створювалася саме для того, щоб змінити цю ситуацію.

За своєю суттю платформа являє собою веб-застосунок, що зводить воєдино оренду контейнерів, роботу з маршрутами, дистанційний контроль стану вантажу та фінансову складову. Клієнт не мусить звертатися до різних служб по черзі — всі необхідні дії виконуються в одному середовищі, від вибору контейнера до отримання платіжних документів після завершення перевезення.

Окреме місце в системі займає IoT-моніторинг. Кожен контейнер може бути оснащений датчиками восьми типів — температури, вологості, удару та вібрації, нахилу, відчинення дверей, освітлення, рівня $CO_2$ і GPS-трекером [6, 7, 8, 24]. Показники передаються в режимі, наближеному до реального часу і відображаються в особистому кабінеті користувача. Для вантажів із підвищеними вимогами до умов зберігання — фармацевтики, електроніки або харчових продуктів — це дає змогу помітити відхилення ще під час транспортування, а не після прибуття в порт призначення.

З технічного боку серверну частину реалізовано на PHP 8.2 та Laravel, клієнтську — на Vue.js із Inertia.js [37, 46]. Для збереження інформації задіяно PostgreSQL, Redis відповідає за кешування й обробку фонових задач [39, 41]. Розгортання системи організовано через Docker [29]. Програмна архітектура побудована за підходом MVC із виокремленням основної бізнес-логіки в сервісні класи — це спрощує подальший супровід і розширення платформи [9, 12, 17, 18].

Функціональне охоплення продукту включає багатокрокове оформлення оренди контейнерів чотирьох типів, управління маршрутами між портами з відображенням на карті, перегляд телеметрії датчиків, опрацювання платежів із підтримкою зовнішніх провайдерів, вантажну документацію, сповіщення через email і Telegram-бот та

адміністративну панель для управління замовленнями й контейнерним парком.

Logistics SWay — це не концептуальний макет, а повністю функціональний програмний продукт із робочою базою даних, реальним інтерфейсом і готовою до розгортання інфраструктурою. Ця система здатна скоротити кількість ручних операцій у логістичному процесі, підвищити інформованість клієнта щодо стану вантажу і спростити взаємодію між усіма сторонами перевезення.

### 3.2 Вхідні та вихідні дані

Програмний продукт Logistics SWay опрацьовує декілька категорій вхідних даних, що надходять як від користувачів через інтерфейс, так і автоматично — від IoT-датчиків та зовнішніх сервісів.

**Вхідні дані**

Першу категорію складають дані користувача. Під час реєстрації платформа отримує ім'я, прізвище, email, пароль, номер телефону, адресу та країну. Після входу клієнт може редагувати профіль, змінювати фото, налаштовувати канали сповіщень — email або Telegram. Для підключення Telegram-бота вводиться одноразовий верифікаційний код із обмеженим терміном дії.

Друга категорія охоплює дані замовлення на оренду контейнера. Воно формується через багатокроковий процес, під час якого клієнт зазначає тип контейнера (20ft standard, 40ft standard, 40ft high cube або 20ft reefer), порти відправлення та призначення, строки оренди, характеристики вантажу — масу, об'єм, кількість місць, задекларовану вартість. Окремо вказуються умови доставки (incoterm, тип завантаження, режим доставки) та параметри безпеки — ознака небезпечного вантажу, UN-номер, клас небезпечних вантажів, вимога митного оформлення, необхідність пломбування та супроводу. Для рефрижераторних контейнерів задається температурний діапазон зберігання вантажу.

Третю категорію становлять дані від IoT-датчиків. Вони надходять автоматично без участі клієнта і охоплюють показники восьми типів сенсорів: температура (°C), відносна вологість (%), прискорення удару (g), кут нахилу (°), стан дверей (відчинено/зачинено), рівень освітленості (лк), концентрація $CO_2$ (ppm) та GPS-координати [42]. Дані передаються кожні 15 хвилин і фіксуються з часовою міткою та метаданими в партиційованій таблиці [12, 14].

Четверта категорія — фінансові дані. Клієнт вибирає метод оплати та підтверджує транзакцію, після чого від зовнішнього платіжного провайдера надходить ідентифікатор операції та її статус.

**Вихідні дані**

Першою групою вихідних даних є інформація про оренду. Після оформлення замовлення клієнт отримує підтвердження з деталями, розрахованою вартістю та її розбивкою по складових у форматі JSON, поточним статусом і реквізитами для оплати. Після проведення платежу платформа формує контракт у форматі PDF.

Другу групу складають дані IoT-моніторингу. На дашборді відображаються актуальні показники датчиків та графіки їхньої динаміки за обраний період. Клієнт має змогу налаштувати власний макет відображення й порогові значення для автоматичних сповіщень. Поряд із цим платформа веде ланцюжок IoT-аудиту — послідовність хешованих записів подій, що підтверджує незмінність телеметричних даних після їх фіксації [15, 39].

Третю групу становлять логістичні дані. Платформа відображає маршрут на карті з візуалізацією морського шляху, деталізацію по сегментах із зазначенням судна, портів і планових дат, поточний статус відправлення та очікувану дату прибуття. За наявності нештатних ситуацій фіксується інцидент із визначенням типу, ступеня критичності та статусу розгляду.

Четверта група — фінансові вихідні дані. Клієнт отримує відомості про проведені транзакції із зазначенням суми, валюти, методу оплати та статусу. Адміністратор має доступ до зведеної фінансової звітності по всіх замовленнях платформи.

П'яту групу складають сповіщення. Платформа надсилає повідомлення через email або Telegram-бот у таких випадках: вихід показників датчиків за встановлені порогові значення, зміна статусу оренди чи відправлення, підтвердження або відхилення замовлення, успішне проведення або відхилення платежу. Кожне повідомлення містить пряме посилання на відповідну сторінку для швидкого переходу.

**Апаратні та програмні вимоги**

Для роботи з платформою на стороні клієнта достатньо сучасного браузера з підтримкою JavaScript — Chrome, Firefox, Edge або Safari актуальних версій.

Мінімальні вимоги до пристрою: двоядерний процесор із частотою від 2 ГГц, 4 ГБ оперативної пам'яті та стабільне інтернет-з'єднання від 10 Мбіт/с. Інтерфейс адаптований для різних розмірів екранів, проте основним середовищем є десктоп.

Серверна частина розгортається в Docker-контейнерах і потребує восьмиядерного процесора з частотою від 3 ГГц, щонайменше 16 ГБ оперативної пам'яті та SSD-накопичувача від 500 ГБ. PostgreSQL рекомендовано розмістити на виділеному вузлі або окремому контейнері, Redis може функціонувати на тому ж хості залежно від навантаження.

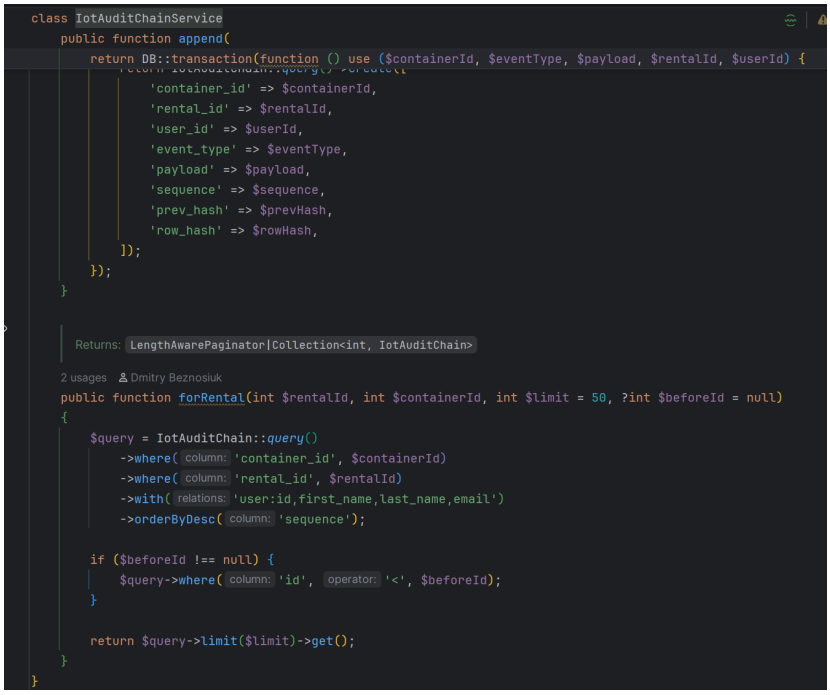
З огляду на нефункціональні вимоги — платформа розрахована на одночасне опрацювання щонайменше 500 запитів, час відгуку сторінок не перевищує 2 секунди, щоденне резервне копіювання бази даних та відновлення після збоїв протягом 5 хвилин. Захист інформації реалізовано через HTTPS, хешування паролів, валідацію вхідних даних та криптографічний захист ланцюжка IoT-аудиту [5].

### 3.3 Програмні засоби реалізації та обґрунтування їх вибору

Вибір технологій для Logistics SWay визначався не модними трендами, а конкретними потребами проекту — складною базою даних, великими обсягами IoT-телеметрії, фоновими задачами і сучасним реактивним інтерфейсом. Кожен інструмент у стеку виконує чітко визначену роль і замінити його чимось іншим без наслідків для архітектури не вийшло б.

Серверну частину написано на PHP, використано як основну мову серверної частини завдяки сучасному синтаксису, підтримці типізації, стабільній роботі з Laravel та широкому набору розширень для веброзробки [37]. На практиці це відчутно прискорює складні обчислення, як приклад розрахунок вартості оренди, де враховується кілька десятків параметрів одночасно. Крім того, PHP отримав суворіші типи і зручніший синтаксис, що робить код чистішим і надійнішим у довгостроковій підтримці. У структурі системи PHP відповідає за весь серверний цикл — приймає запити, виконує бізнес-логіку, спілкується з базою даних і формує відповіді.

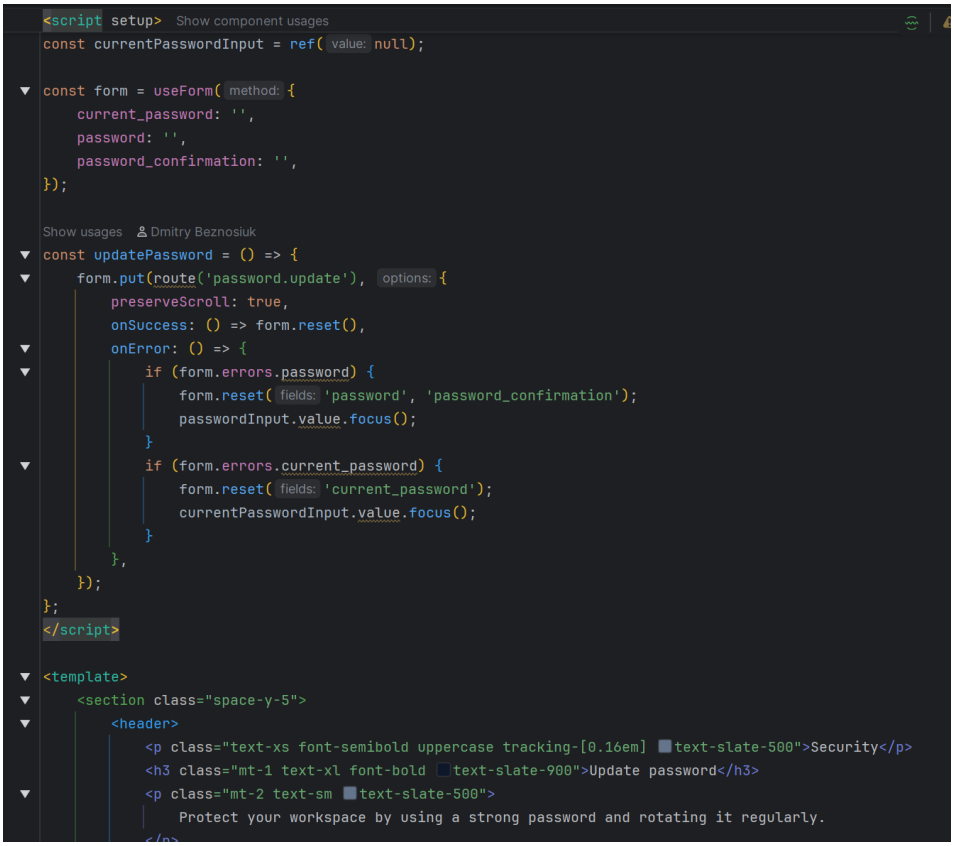
Фреймворк Laravel побудований поверх PHP і визначає загальну архітектуру платформи на рис 3.1 [34]. Замість того щоб з нуля писати маршрутизацію, автентифікацію або роботу з чергами, розробник отримує готові й добре перевірені механізми, які залишається лише підлаштувати під свої потреби.



```
class IotAuditChainService
    public function append(
        return DB::transaction(function () use ($containerId, $eventType, $payload, $rentalId, $userId) {
            'container_id' => $containerId,
            'rental_id' => $rentalId,
            'user_id' => $userId,
            'event_type' => $eventType,
            'payload' => $payload,
            'sequence' => $sequence,
            'prev_hash' => $prevHash,
            'row_hash' => $rowHash,
        ]);
    });
}

Returns: LengthAwarePaginator|Collection<int, IotAuditChain>

2 usages  Dmitry Beznosiuk
public function forRental(int $rentalId, int $containerId, int $limit = 50, ?int $beforeId = null)
{
    $query = IotAuditChain::query()
        ->where( column: 'container_id', $containerId)
        ->where( column: 'rental_id', $rentalId)
        ->with( relations: 'user:id,first_name,last_name,email')
        ->orderByDesc( column: 'sequence');

    if ($beforeId !== null) {
        $query->where( column: 'id', operator: '<', $beforeId);
    }

    return $query->limit($limit)->get();
}
}
```

Рис. 3.1 — Фрагмент PHP-коду системи

У проекті Laravel задіяний на повну — Eloquent ORM замінює сирі SQL-запити зручними моделями з описаними зв'язками, система черг через Redis обробляє все що не потребує миттєвої відповіді клієнту, механізм подій автоматично запускає потрібні дії при зміні стану замовлення, а планувальник виконує регулярні операції за розкладом на рис 3.2.



```
<script setup>  Show component usages
const currentPasswordInput = ref( value: null);

const form = useForm( method: {
    current_password: '',
    password: '',
    password_confirmation: '',
});

Show usages  Dmitry Beznosiuk
const updatePassword = () => {
    form.put(route('password.update'), options: {
        preserveScroll: true,
        onSuccess: () => form.reset(),
        onError: () => {
            if (form.errors.password) {
                form.reset( fields: 'password', 'password_confirmation');
                passwordInput.value.focus();
            }
            if (form.errors.current_password) {
                form.reset( fields: 'current_password');
                currentPasswordInput.value.focus();
            }
        },
    });
};
</script>

<template>
    <section class="space-y-5">
        <header>
            <p class="text-xs font-semibold uppercase tracking-[0.16em] text-slate-500">Security</p>
            <h3 class="mt-1 text-xl font-bold text-slate-900">Update password</h3>
            <p class="mt-2 text-sm text-slate-500">
                Protect your workspace by using a strong password and rotating it regularly.
            </p>
```

Рис. 3.2 — Vue.js компонент інтерфейсу

Клієнтський інтерфейс побудовано на Vue.js [46]. Цей фреймворк надає інструменти для описувати інтерфейс як набір незалежних компонентів — кожен має власну логіку, стан і розмітку. Це зручно і при розробці, і при подальшій підтримці, бо змінюючи один компонент не ризикуєш зачепити решту. Поруч із Vue працює Inertia.js — він з'єднує серверну і клієнтську частини так, що окремий API писати не потрібно [33]. Laravel просто передає дані у Vue-компоненти напряму, і сторінка оновлюється реактивно без перезавантаження. Збірку і компіляцію фронтенду виконує Vite — він працює набагато швидше від попередника Webpack, бо при зміні файлу оновлює лише той модуль який змінився, а не перезбирає все [45]. Стилізацію побудовано на Tailwind CSS — утилітарному фреймворку, де зовнішній вигляд описується прямо в розмітці без написання окремих CSS-файлів [43].

Для організація сховища обрано PostgreSQL, а не більш поширений MySQL, і на це є конкретні причини [39]. По-перше, PostgreSQL краще справляється зі складними реляційними схемами — а в проекті їх 35 таблиць із розгалуженими зв'язками і каскадними правилами, прикладом на рис. 3.3. По-друге, він підтримує партиціювання таблиць на рівні СУБД — таблиця telemetry, яка накопичує IoT-дані, поділена на партиції, що критично для продуктивності при великих обсягах записів. По-третє, PostgreSQL має розширення pgcrypto, яке використано для криптографічного захисту ланцюжка IoT-подій. Додатково база чудово працює з JSON-полями, які в проекті зустрічаються часто — розбивка вартості, параметри вантажу, налаштування датчиків.

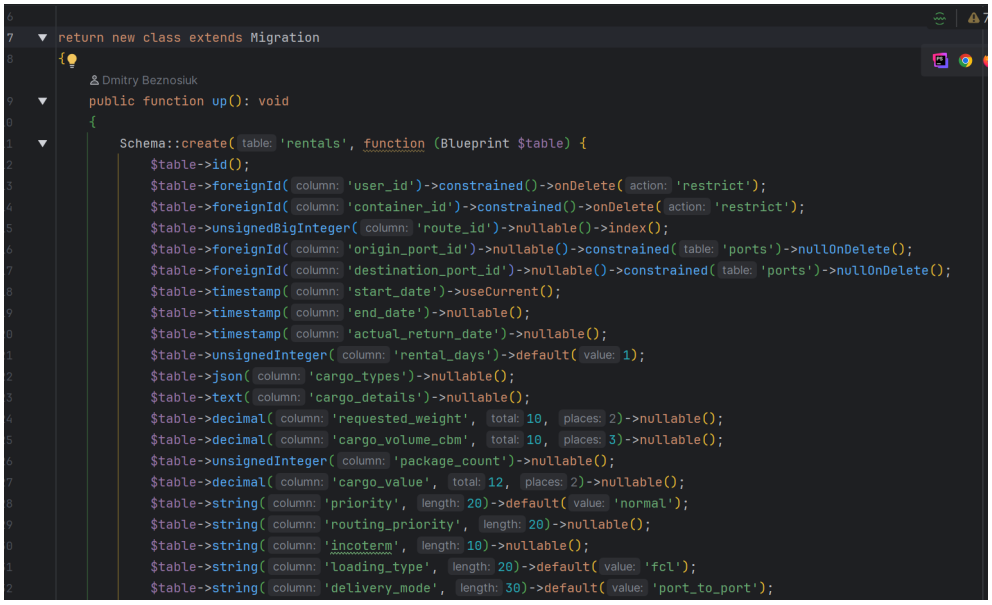



Рис. 3.3 — Схема бази даних PostgreSQL

Redis у програмному середовищі вирішує дві різні задачі [41]. Перша — кешування часто запитуваних даних, щоб не звертатися до бази даних кожен раз заново. Друга — черги фонових задач. Все що не потребує миттєвої відповіді клієнту — відправлення email, Telegram-повідомлень, генерація PDF-контрактів, обробка IoT-подій — відправляється в чергу і виконується у фоні. Це розвантажує основний цикл обробки запитів і робить інтерфейс більш чутливим.

Nginx приймає всі вхідні запити до системи [35]. Статичні файли — JS, CSS, зображення — він віддає сам, без участі PHP. Динамічні запити передає до PHP-FPM через FastCGI. Така схема є класичною і перевіреною — Nginx добре справляється з великою кількістю одночасних з'єднань, а PHP-FPM зосереджується виключно на виконанні коду.

Уся інфраструктура системи контейнеризована через Docker, рис 3.4 [29]. PHP-образ збирається у два етапи — спочатку встановлюються залежності через Composer, потім формується фінальний образ з усіма розширеннями. Це зменшує його розмір і пришвидшує повторні збірки. Docker Compose об'єднує чотири сервіси — php-fpm, nginx, postgres і redis — в єдину мережу [16]. Кожен ізольований у власному контейнері, і середовище розробника повністю збігається з тим, що буде на сервері. Для обробки потоків IoT-подій додатково застосовується Kafka [26].

Залежності PHP-частини описує файл composer.json і фіксує composer.lock — будь-яке середовище, що розгортає проект, отримує ідентичний набір пакетів [28]. Для налаштування домену і захисту трафіку задіяно Cloudflare [27, 31]. Фронтенд-залежності керуються через npm, а Vite при продакшн-збірці формує оптимізовані бандли з мінімізацією і хешуванням файлів для ефективного кешування в браузері [36].

```yaml
services:
  nginx:
    image: nginx:1.27-alpine
    container_name: ${APP_NAME:-laravel}_nginx
    restart: unless-stopped
    ports:
      - "${NGINX_HTTP_PORT:-80}:80"
      - "${NGINX_HTTPS_PORT:-443}:443"
    volumes:
      - ./:/var/www/html
      - ./docker/nginx/nginx.conf:/etc/nginx/conf.d/default.conf
      - ./docker/nginx/ssl:/etc/nginx/ssl
    depends_on:
      - php
    networks:
      - laravel

  php:
    build:
      context: .
      dockerfile: Dockerfile
    image: vuelarcontainer-php:latest
    container_name: ${APP_NAME:-laravel}_php
    restart: unless-stopped
    volumes:
      - ./:/var/www/html
      - ./docker/php/php.ini:/usr/local/etc/php/conf.d/custom.ini
    networks:
      - laravel
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      - PHP_OPCACHE_ENABLE=${PHP_OPCACHE_ENABLE:-1}
      - PHP_OPCACHE_MEMORY_CONSUMPTION=${PHP_OPCACHE_MEMORY_CONSUMPTION:-128}
      - PHP_OPCACHE_INTERNED_STRINGS_BUFFER=${PHP_OPCACHE_INTERNED_STRINGS_BUFFER:-8}
```

Рис. 3.4 — Dockerfile та docker-compose конфігурація

### 3.4 Блок-схема алгоритму

В додатку Ж на рис. Ж.1 наведено блок-схему алгоритму методу calculate() сервісу RentalPricingService — компонента платформи, що відповідає за формування вартості оренди контейнера.

Суть методу в тому, що вартість оренди не є фіксованою — вона складається з багатьох частин залежно від того, який контейнер обрано, куди і на скільки його везуть, що всередині і які додаткові умови клієнт поставив. Саме тому на вході метод приймає чимало параметрів: дані користувача і контейнера, дати оренди, відстань маршруту, характеристики вантажу, пріоритет і режим доставки, а водночас набір позначок — чи потрібне страхування, митний супровід, пломбування або конвой.

Розрахунок починається з базових величин — кількості днів оренди, відстані маршруту і стартової ціни у 5000 USD. До неї одразу застосовується коефіцієнт типу контейнера: стандартний — 1.00, high cube — 1.20, open top — 1.25, flat rack — 1.30, рефрижераторний — 1.40. Після цього додаються компонентні складові: 0.10 USD за

кілометр, 20 USD за день, 75 USD за кожен тип вантажу, а окрім цього складові за масу, об'єм і кількість місць.

Далі послідовно перевіряються умови надбавок. IoT-моніторинг додає 250 USD, express-пріоритет — 1200 USD, urgent — 550 USD. Небезпечний вантаж коштує додатково 900 USD, митне оформлення — 220 USD, доставка від дверей до дверей — 600 USD, часткове завантаження LCL — 180 USD, супровід — 450 USD, пломба — 75 USD. Екологічні опції додають від 140 до 260 USD. Страхування розраховується як максимум між 120 USD і 1.2% від задекларованої вартості вантажу.

Усі складові підсумовуються. Якщо оренда довша за 30 днів — застосовується знижка 10%. Після цього нараховується податок за ставкою країни клієнта і формується фінальна сума.

Результатом роботи методу є детальна розбивка по кожній складовій, яка зберігається разом із замовленням. Клієнт бачить не просто підсумкову цифру, а повну картину з чого вона склалася — і система в будь-який момент може відтворити цей розрахунок.

### 3.5 Опис інтерфейсу програмного продукту

Інтерфейс платформи Logistics SWay складається з публічної частини, доступної без авторизації, та особистого кабінету користувача. Публічна частина охоплює головну сторінку, сторінку сервісів і форму зворотного зв'язку. Особистий кабінет надає доступ до управління орендою, IoT-моніторингу та фінансових операцій. Нижче наведено опис кожного екрана платформи.

**Головна сторінка Home**

Головна сторінка — це перше що бачить відвідувач платформи. Вгорі розміщено навігаційну панель із логотипом, посиланнями на розділи Home, Services і Contact та кнопкою входу в кабінет. Центральний банер коротко пояснює суть сервісу — оренда морських контейнерів із IoT-відстеженням — і одразу показує масштаб компанії через ключові цифри. Дві кнопки на банері ведуть або до перегляду послуг, або одразу до оформлення запиту на рис 3.5.

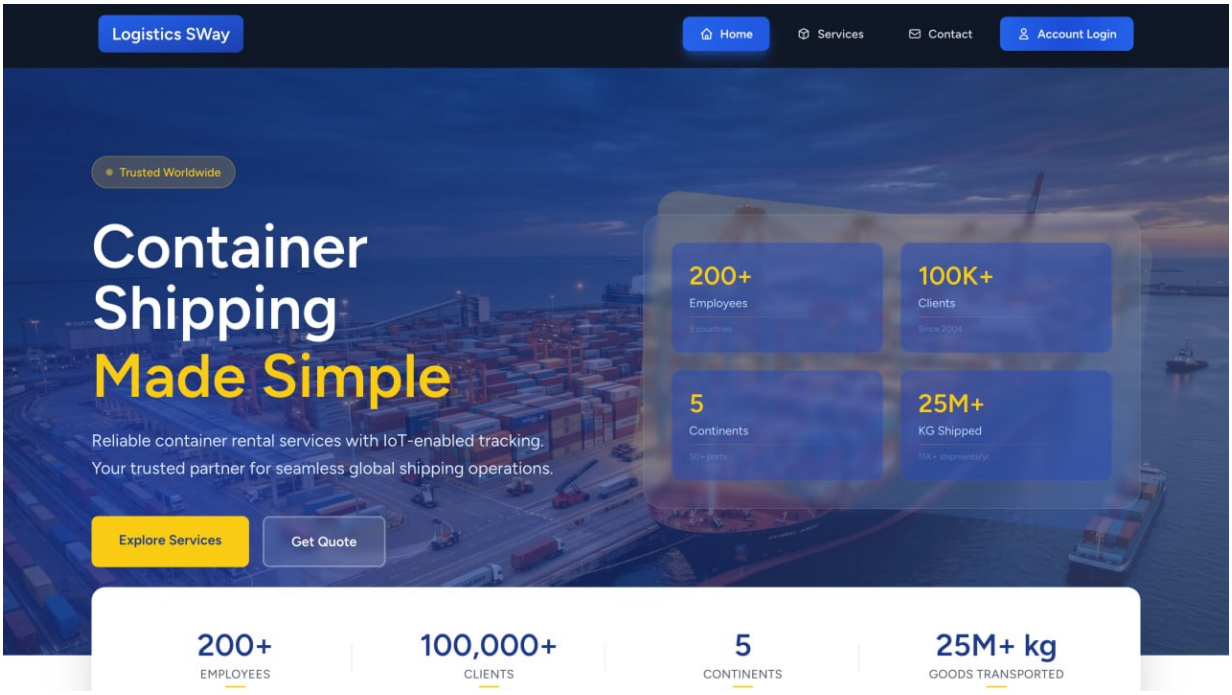



Рис. 3.5 — Головна сторінка платформи Logistics SWay. Верхня частина

Нижче розміщено три блоки з описом основних можливостей платформи — маршрутна мережа, IoT-моніторинг і онлайн-управління. Далі йде перелік послуг: FCL і LCL перевезення, відстеження вантажу, митна документація, складське зберігання і страхування. Завершує сторінку блок переваг із кнопкою Start Your Project, яка переводить користувача до подальшої взаємодії з платформою.

**Сторінка сервісів Services**

На рис. 3.6 показано головний банер сторінки сервісів. Ця сторінка призначена для ознайомлення користувача з основними можливостями платформи. У верхній частині розміщено заголовок Services & IoT Solutions, який вказує на два головні напрями роботи системи: логістичні послуги та IoT-моніторинг контейнерів.

У банері коротко описано, що платформа супроводжує процес перевезення від бронювання контейнера до завершення доставки. Поруч подано основні інформаційні показники: кількість клієнтів, кількість портів, географію роботи та наявність моніторингу. Такі дані допомагають користувачу швидко зрозуміти масштаб платформи та її призначення.

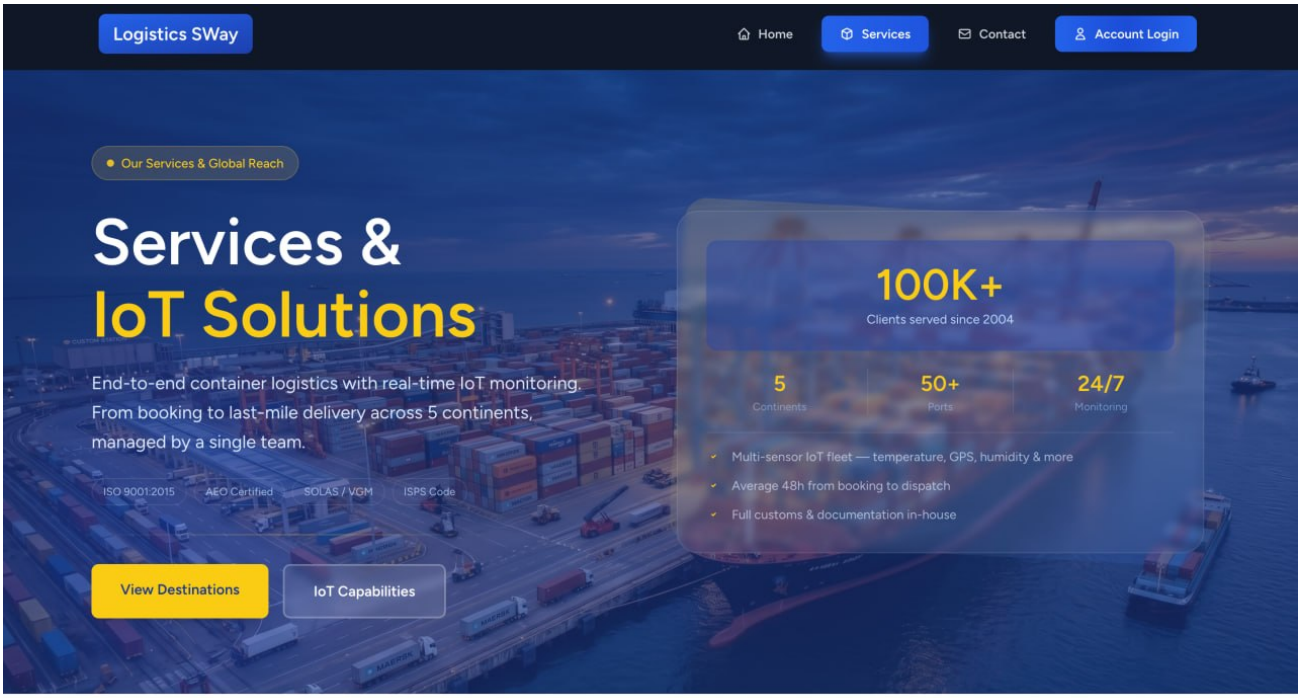



Рис. 3.6 — Сторінка сервісів. Головний банер

На рис. 3.7 наведено блок Countries We Work With, у якому показано країни та напрями, з якими працює платформа. Для кожної країни подано доступні режими перевезення та короткі особливості логістичного супроводу. Наприклад, для США вказано підтримку сертифікованого ланцюга постачання, а для Німеччини — митний супровід і регулярні відправлення.

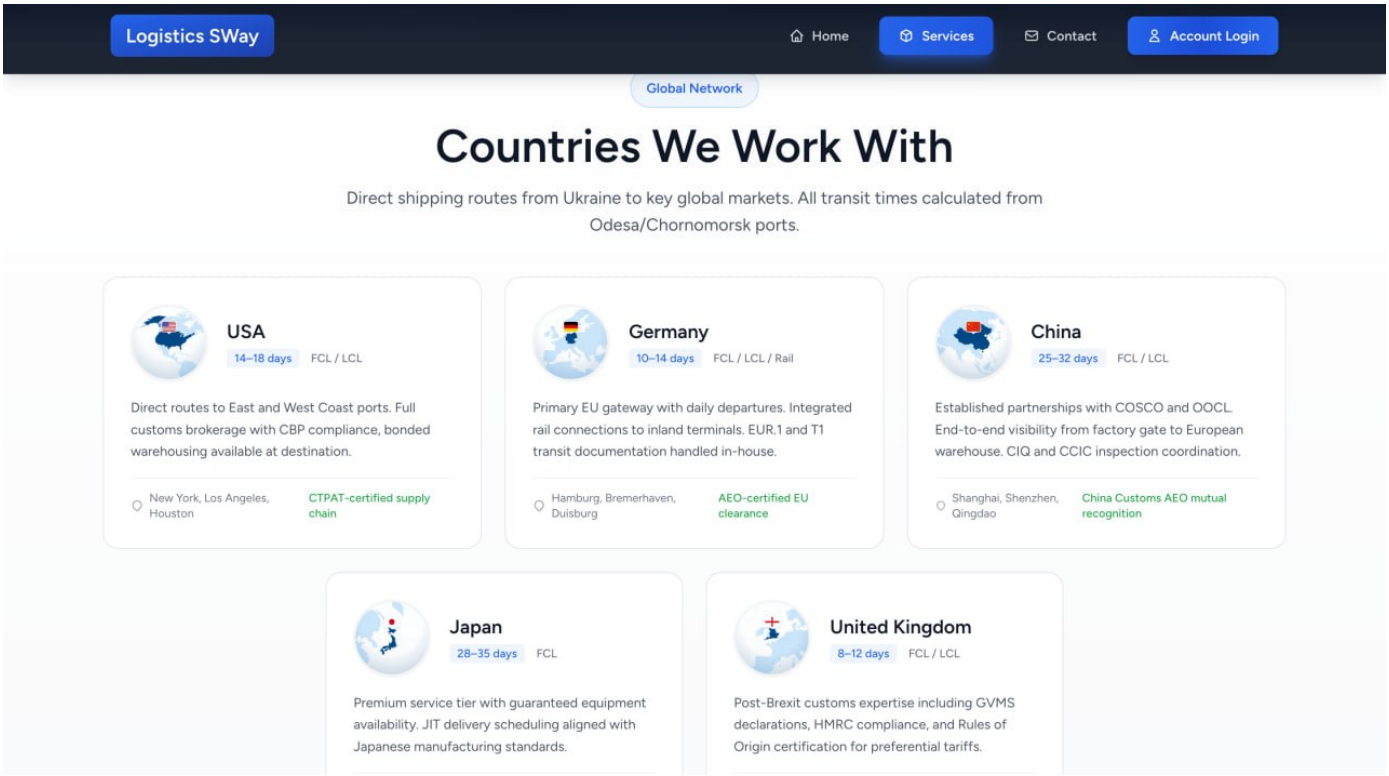



Рис. 3.7 — Сторінка сервісів. Блок «Countries We Work With»

На рис. 3.8 показано блок, присвячений IoT-можливостям платформи. У ньому подано картки з типами датчиків, які можуть використовуватися для контролю стану контейнера. Серед них є датчики температури, вологості, відкриття дверей, освітлення, удару, нахилу, рівня $CO_2$ та GPS-трекер. Для кожного датчика зазначено його призначення та основні технічні характеристики.

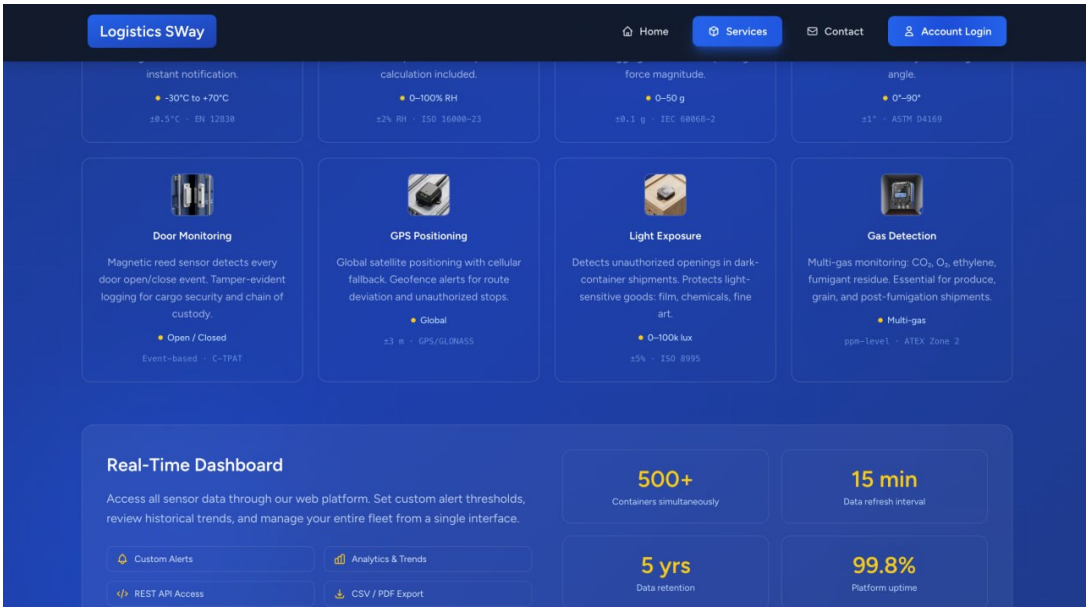



Рис. 3.8 — Сторінка сервісів. Блок IoT-датчиків та Real-Time Dashboard

Окремо на сторінці розміщено блок Real-Time Dashboard, який дає можливості відстеження контейнерів. У ньому зазначено кількість контейнерів, які можуть контролюватися одночасно, періодичність оновлення даних, строк зберігання інформації та показник доступності системи.

**Сторінка контактів Contact**

Сторінка контактів слугує точкою зв'язку між потенційним клієнтом і командою платформи. У верхній частині одразу розміщено всю необхідну інформацію про компанію. Дві кнопки — Send a Message і Call Us Now — пропонують два зручні способи зв'язатися з компанією.

Нижче розміщено форму Send a Message з полями для імені, email, теми звернення та текстового повідомлення, щоб з вами зв’язалися. Форма відправляє замовлення і так скоріше ви можете отримати консультацію чи допомогу на рис. 3.9.

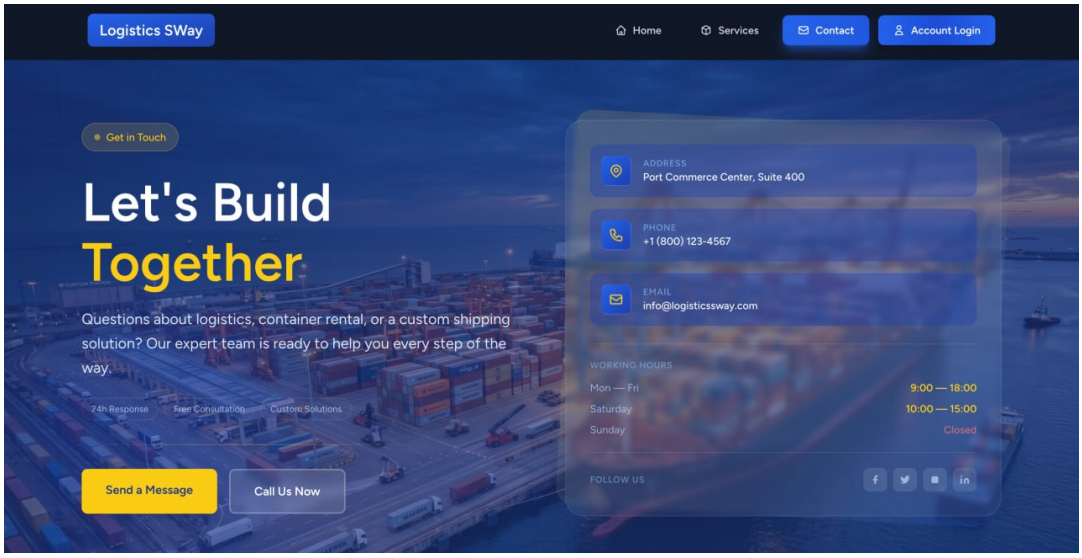



Рис. 3.9 — Сторінка Contact. Верхня частина з контактними даними

**Особистий кабінет користувача**

Після авторизації користувач потрапляє до особистого кабінету. Інтерфейс побудований як єдиний — ліва панель навігації надає доступ до розділів Dashboard на рис. 3.10, Finance, Rental, Request Rental і Support. Головний екран кабінету відображає зведений стан поточних операцій: кількість активних і завершених оренд, непрочитані сповіщення, стан профілю та ключові фінансові й логістичні показники з регулярним оновленням даних.

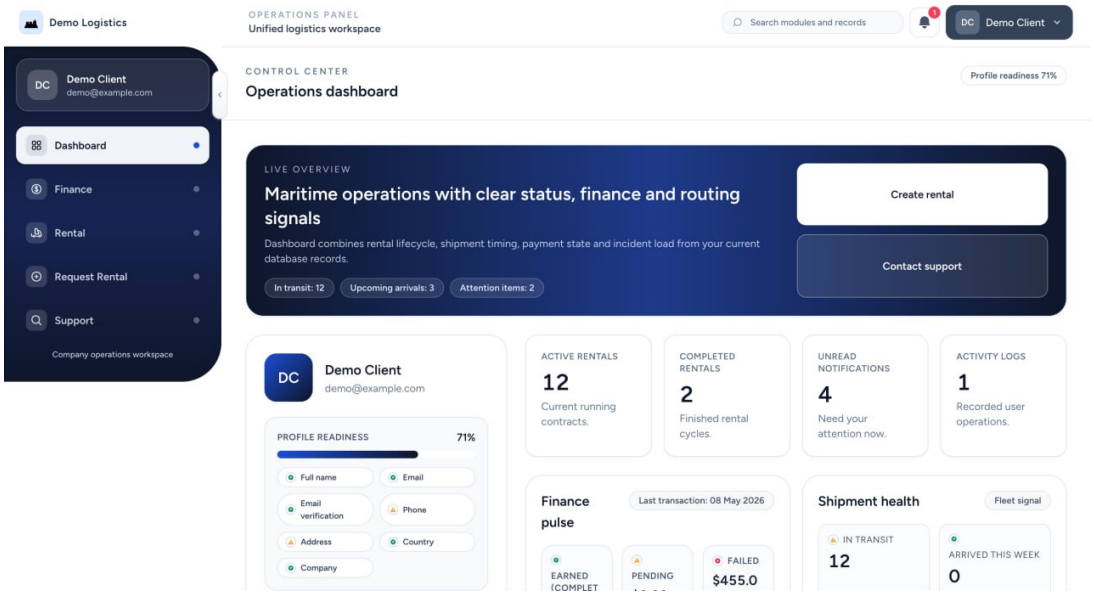



Рис. 3.10 — Головна сторінка особистого кабінету

Розділ Finance Monitoring, деталізує фінансову картину — загальні суми оплачених, очікуваних і невдалих транзакцій, розподіл орендних угод за статусами та платіжним станом. Уся інформація структурована у вигляді карток і таблиць, що дає

змогу швидко оцінити фінансовий стан без зайвих кроків.

### 3.6 Приклади застосування системи

У цьому підрозділі наведено покроковий сценарій роботи з платформою — від реєстрації клієнта до підтвердження оренди адміністратором. Це охоплює основний цикл і взаємодію між двома ролями системи: клієнтом і адміном.

Новий користувач переходить на сторінку реєстрації та заповнює форму, та по бажанню обирає країну. Після підтвердження реєстрації, система автоматично верифікує введені дані й створює обліковий запис. За наступного відвідування клієнт просто вводить логіниться до особистого кабінету.

Після авторизації клієнт опиняється на головному екрані кабінету на рис 3.11. Тут одразу видно загальний стан облікового запису — заповненість профілю, кількість активних оренд, непрочитані сповіщення. крім того підказує, які поля профілю варто доповнити, щоб пришвидшити обробку майбутніх замовлень.

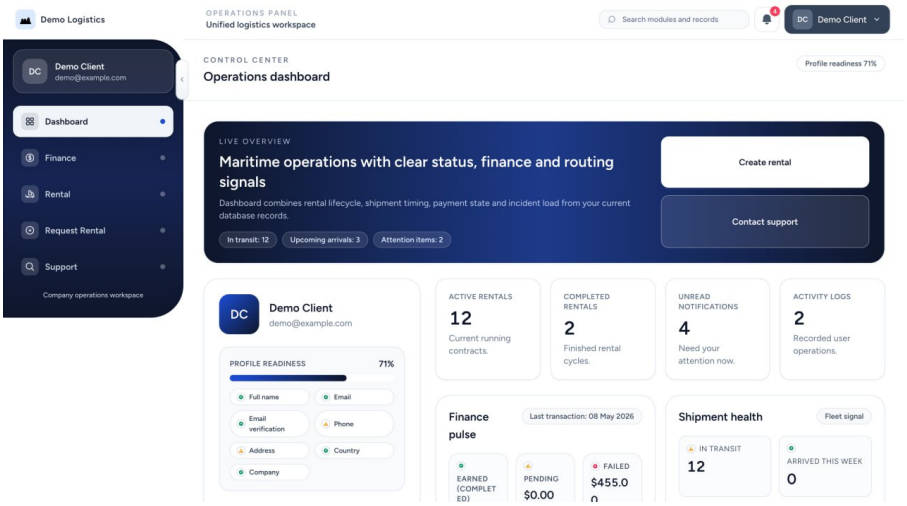



Рис. 3.11 — Operations Dashboard після першого входу

Клієнт переходить до розділу Request Rental і починає заповнювати форму замовлення. Спочатку обирає контейнер зі списку — система показує серійний номер, тип, габарити та поточний порт розташування. Далі вказує маршрут: порт відправлення, порт призначення, дати початку і завершення оренди. Одразу відображається розрахункову відстань і орієнтовний час у дорозі, див. рис. 3.12.

На наступному кроці клієнт описує вантаж — обирає типи (електроніка, одяг, продукти харчування тощо), вказує масу, об'єм, кількість місць і задекларовану вартість. Якщо вантаж належить до небезпечних категорій — система пропонує

заповнити UN-номер і клас небезпечності. Окремо можна обрати умови доставки (FCL або LCL, режим door-to-door або port-to-port), підключити страхування, вказати необхідність митного оформлення, пломбування або конвою.

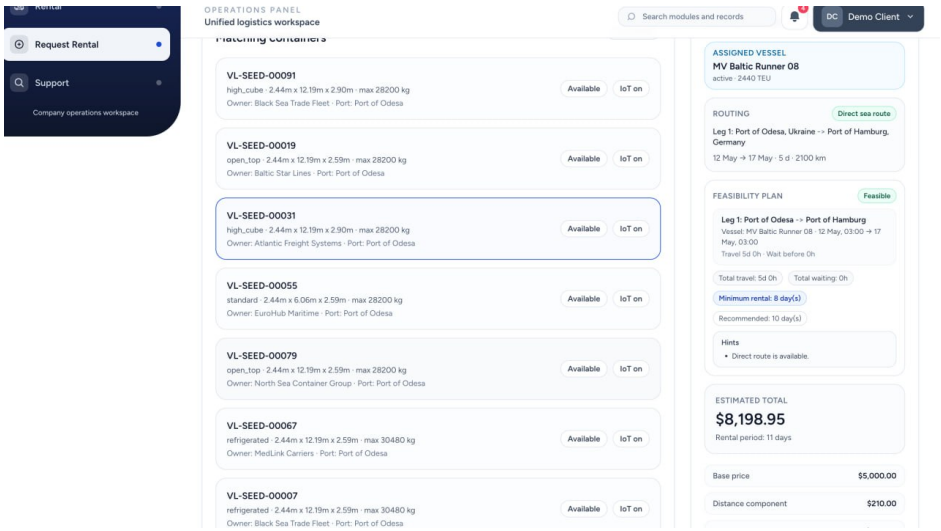



Рис. 3.12 — Сторінка Request Rental. Розрахунок вартості перед надсиланням заявки

Після заповнення всіх полів клієнт натискає Calculate Price — система викликає RentalPricingService і повертає детальну розбивку вартості по кожній складовій. Клієнт бачить не просто підсумкову суму, а з чого саме вона складається. Якщо все влаштовує — натискає кнопку підтвердження, і заявка надходить до системи зі статусом pending_approval.

Після надсилання заявки клієнт бачить її в розділі Rental із поточним статусом. На цьому етапі замовлення перебуває на розгляді в операційного відділу платформи рис 3.13. Система потім надсилає сповіщення клієнту про те, що заявку отримано й передано на перевірку.

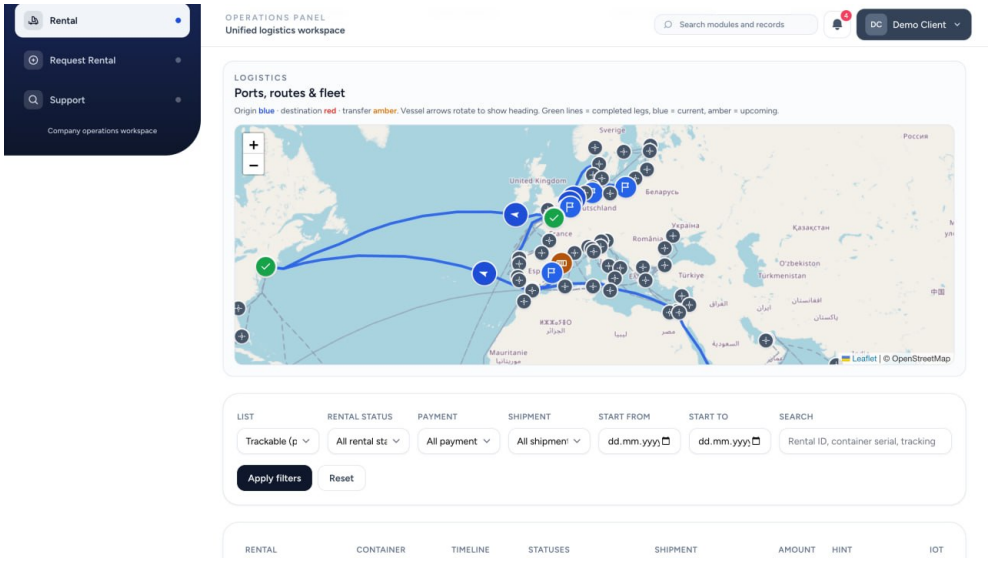



Рис. 3.13 — Сторінка Rental

З боку платформи співробітник відкриває адміністративну панель і бачить нове замовлення в черзі на опрацювання рис 3.14. Він переглядає всі деталі — дані клієнта, обраний контейнер, маршрут, параметри вантажу, митні вимоги і розбивку вартості. Якщо все відповідає вимогам і контейнер доступний на зазначені дати — співробітник підтверджує замовлення. Після підтвердження замовлення система зберігає інформацію про відповідального працівника та час розгляду заявки.

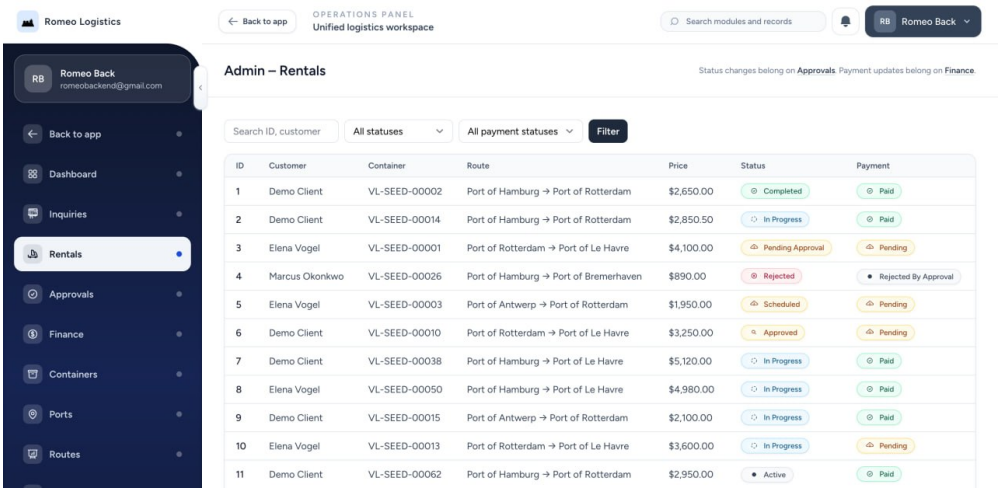



Рис. 3.14 — Адміністративна панель. Сторінка Rental

Одразу після зміни статусу платформа надсилає клієнту сповіщення — через особистий кабінет і, якщо підключено, через телеграм-бот, прикладом на рис 3.15. Клієнт повертається до розділу Rental і бачить, що замовлення підтверджено. Тепер стає доступна кнопка оплати.

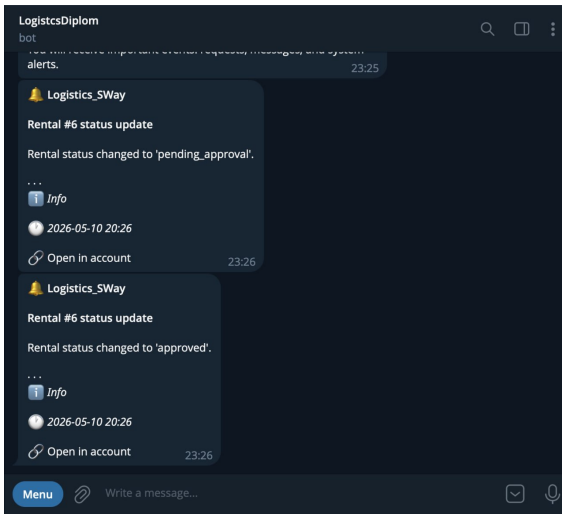



Рис. 3.15 — Сповіщення про підтвердження замовлення

Клієнт переходить до оплати, обирає зручний метод і підтверджує транзакцію. Потім передає запит до зовнішнього платіжного провайдера й отримує у відповідь статус операції та ідентифікатор транзакції. Після успішного проведення платежу

запис у таблиці transactions набуває статусу paid, а поле payment_status в таблиці rentals оновлюється відповідно. Статус замовлення переходить до scheduled або одразу до in_progress — залежно від дати початку оренди рис. 3.16.

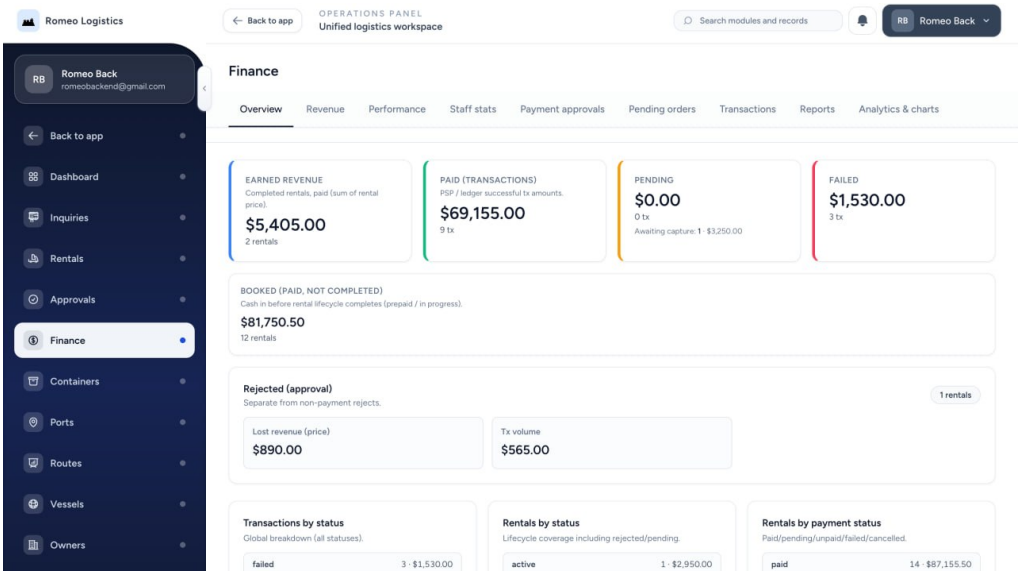



Рис. 3.16 — Сторінка фінансів

Коли оренда стає активною, клієнт отримує доступ до IoT-дашборду конкретного контейнера. На екрані відображаються актуальні показники всіх підключених датчиків — температура, вологість, стан дверей, GPS тощо. Дані оновлюються кожні 15 хвилин. Якщо будь-який із показників виходить за встановлені порогові значення — система автоматично формує сповіщення і надсилає його клієнту, на рис. 3.17.

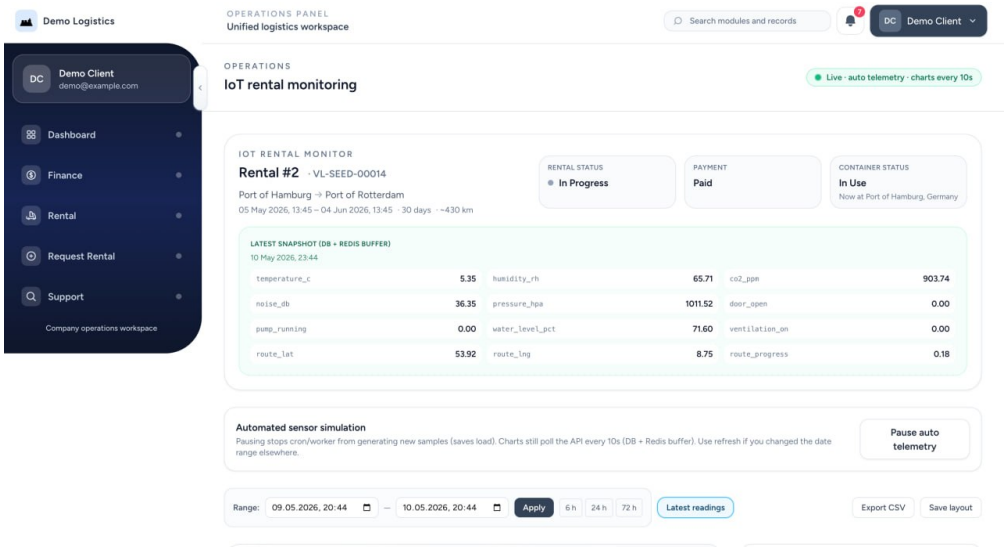



Рис. 3.17 — IoT-дашборд із показниками датчиків під час активної оренди

## 3.7 Інструкція користувача

Для роботи з платформою достатньо будь-якого сучасного браузера та інтернет-з'єднання — встановлювати нічого не потрібно. Найзручніше користуватися з комп'ютера або ноутбука, особливо під час роботи з документами та графіками.

**Реєстрація та налаштування профілю**

Щоб створити обліковий запис, потрібно натиснути Account Login і заповнити форму реєстрації. Після входу рекомендується одразу заповнити профіль повністю — це спрощує подальше оформлення оренди. Якщо пароль забуто, на сторінці входу є посилання Forgot your password? — після переходу на пошту надійде інструкція для відновлення. Для підключення Telegram-сповіщень потрібно зайти в налаштування профілю, скопіювати одноразовий код і надіслати його боту платформи.

**Навігація по кабінету**

Після авторизації користувач потрапляє до Operations Dashboard. Ліва панель містить основні розділи: Dashboard — головний екран із загальним станом оренд і платежів; Finance — всі транзакції з датами, сумами і статусами; Rental — список замовлень із деталями кожного; Request Rental — форма для нової оренди; Support — форма звернення до підтримки.

**IoT-дашборд**

Стає доступним після того як оренда переходить в активний стан. Відкрити його можна через розділ Rental у деталях замовлення. На екрані відображаються показники датчиків — температура, вологість, стан дверей, GPS та інші. Можна обрати діапазон дат і переглянути графіки за потрібний період. Порогові значення налаштовуються окремо для кожного датчика — при перевищенні система надсилає сповіщення в кабінеті та через Telegram.

**Типові ситуації**

Якщо замовлення довго очікує підтвердження — варто перевірити розділ сповіщень, можливо платформа запросила додаткові документи. Якщо дані датчиків не оновлюються — телеметрія передається лише для активних оренд, для запланованих дашборд може бути недоступним. Якщо є питання по оплаті — всі транзакції з деталями зберігаються в розділі Finance.

### 3.8 SWOT-аналіз

SWOT-аналіз застосовано для оцінки платформи Logistics SWay не лише з технічного боку, а й з точки зору її реального залучення на ринку морської логістики таблиця 3.1 [1, 3].

Таблиця 3.1 — SWOT-аналіз платформи Logistics SWay

| Категорія | Опис |
|---|---|
| **Сильні сторони (S)** | Об'єднання в одному середовищі повного циклу логістичних операцій — від оформлення заявки до моніторингу вантажу. Восьмитипна IoT-інфраструктура з передачею телеметрії кожні 15 хвилин і криптографічним ланцюжком аудиту, що гарантує незмінність даних. Сучасний технологічний стек: Laravel, Vue.js, Inertia.js, PostgreSQL, Redis, Docker. Детальна розбивка вартості оренди по двадцяти складових через RentalPricingService. Інтеграція з Telegram-ботом для оперативних сповіщень. Партиціонування таблиці метрик для опрацювання великих обсягів IoT-даних. |
| **Слабкі сторони (W)** | Залежність від стабільного інтернет-з'єднання та справності IoT-датчиків. Відсутність нативного мобільного застосунку — користувачі обмежені адаптивним веб-інтерфейсом. Фіксовані коефіцієнти розрахунку вартості закладені в коді сервісу і потребують виведення в адміністративну конфігурацію. Частина інтеграцій (платіжні провайдери, митні служби) у межах навчального проєкту реалізована як прототипи й вимагатиме додаткового тестування перед комерційним впровадженням. |
| **Можливості (O)** | Зростання попиту на цифрові логістичні рішення з прозорим відстеженням вантажів. Розширення платформи мобільним застосунком, інтеграція з реальними API портів, митних служб і судноплавних компаній. Підключення додаткових платіжних провайдерів. Розвиток аналітичного модуля з прогнозуванням завантаженості маршрутів і строків доставки. Розширення можливостей Telegram-бота до повноцінного інтерфейсу управління замовленнями. Корпоративна інтеграція через REST API з ERP-системами підприємств. |
| **Загрози (T)** | Конкуренція з боку великих логістичних операторів із власними цифровими сервісами. Кіберзагрози, пов'язані з обробкою персональних, фінансових і вантажних даних. Зміни в міжнародному митному законодавстві та вимогах до сертифікації IoT-обладнання. Залежність від зовнішніх платіжних провайдерів — припинення їх роботи безпосередньо впливає на фінансовий модуль. Технічні збої датчиків та проблеми зі зв'язком під час морських перевезень. |

### Сильні сторони

Головна перевага платформи в тому, що клієнт отримує все в одному місці — вибір контейнера, розрахунок вартості, погодження, оплату, маршрут і моніторинг вантажу. Не потрібно писати менеджеру щоб дізнатися де контейнер або скільки

коштує доставка. IoT-система контролює вісім параметрів — температуру, вологість, удари, нахил, стан дверей, освітлення, рівень $CO_2$ і геопозицію. Якщо щось виходить за норму, клієнт дізнається одразу, а не після прибуття вантажу з проблемою. Криптографічний захист телеметрії гарантує, що зафіксовані дані неможливо змінити заднім числом.

**Слабкі сторони**

Платформа залежить від стабільності зовнішніх складових — датчиків, мережі і платіжних сервісів. Якщо щось із цього дає збій, частина функцій стає недоступною. Деякі інтеграції в поточній версії реалізовані як прототипи і потребуватимуть доопрацювання перед повноцінним комерційним запуском. Відсутність мобільного застосунку теж є обмеженням — адаптивний вебінтерфейс зручний, але не замінює нативного додатку.

**Можливості**

Найближчі напрямки розвитку — мобільний застосунок і підключення реальних API портів та митних служб. Перспективним є разом із тим розширення Telegram-бота до повноцінного інструменту управління, щоб клієнт міг виконувати дії прямо з месенджера без відкриття браузера. Інтеграція з ERP-системами через наявний REST API відкриває доступ до корпоративних клієнтів із власною інфраструктурою.

**Загрози**

Платформа обробляє персональні дані, фінансові операції і вантажну документацію — тому захист інформації залишається пріоритетом [13]. Митне законодавство, тарифи і вимоги до сертифікації IoT-обладнання можуть змінюватися, і система має бути готова до таких оновлень. Конкуренція з боку великих логістичних операторів реальна, але Logistics SWay виграє за рахунок вузької спеціалізації і глибокого підлаштування під специфіку морських контейнерних перевезень.

**Висновки до третього розділу**

У третьому розділі описано програмну реалізацію, від визначення мети продукту до аналізу його сильних і слабких сторін.

На початку розділу сформульовано призначення системи і коло завдань, які вона вирішує для клієнтів у сфері міжнародних морських перевезень. Описано вхідні та вихідні дані платформи — від параметрів замовлення і характеристик вантажу до

телеметрії IoT-датчиків і фінансових сповіщень. Визначено апаратні та програмні вимоги для коректної роботи як клієнтської, так і серверної частини.

Детально розглянуто технологічний стек системи. Серверну частину та клієнтську. Для утримання даних задіяно PostgreSQL із партиціонуванням таблиці метрик, Redis відповідає за кешування і черги фонових задач, уся інфраструктура розгортається через Docker. Кожен інструмент обрано з конкретним обґрунтуванням, а не лише за популярністю.

Алгоритм розрахунку вартості оренди, реалізований у RentalPricingService, наочно відображено у блок-схемі. Метод враховує понад двадцять складових — базову ціну, тип контейнера, відстань маршруту, характеристики вантажу, умови доставки, митне оформлення, страхування, екологічні преференції та податкову ставку країни клієнта. Результат повертається як детальна розбивка у форматі JSON, що забезпечує повну прозорість ціноутворення для замовника.

Описано інтерфейс платформи — публічну частину зі сторінками Home, Services і Contact, а окрім цього особистий кабінет із розділами Dashboard, Finance, Rental, Request Rental і Support. Наведено приклад повного операційного циклу від реєстрації клієнта до активного IoT-моніторингу під час перевезення. Підготовлено інструкцію користувача з описом основних розділів кабінету і порядком дій у типових ситуаціях.

За результатами SWOT-аналізу визначено, що головними перевагами системи є комплексність рішення, сучасна технічна база і надійна IoT-інфраструктура з криптографічним захистом даних. Серед напрямків розвитку — мобільний застосунок, розширення аналітичного модуля і підключення реальних API портів та митних служб. Основні ризики пов'язані з кібербезпекою і залежністю від зовнішніх провайдерів.

# РОЗДІЛ 4. ТЕСТУВАННЯ, ПЛАНУВАННЯ ТА ЕКОНОМІЧНЕ ОБҐРУНТУВАННЯ ПРОЄКТУ

## 4.1 Тестування програмного продукту

Тестування проводилося для перевірки роботи основних модулів системи та відповідності реалізованих функцій поставленим вимогам [4, 5, 21, 22]. У межах роботи було використано два підходи: ручне тестування основних сценаріїв користувача та автоматизоване модульне тестування за допомогою PHPUnit.

**Мануальне тестування**

Мануальне тестування охоплювало основні дії користувача в системі: реєстрацію, авторизацію, створення заявки на оренду контейнера, розрахунок вартості, перегляд IoT-показників, роботу з фінансовим розділом і надсилання звернення через контактну форму. Результати перевірки наведено в таблиці 4.1.

Таблиця 4.1 — Результати мануального тестування

| № | Тест-кейс | Очікуваний результат | Фактичний результат | Статус |
|---|---|---|---|---|
| 1 | Реєстрація нового користувача з коректними даними | Обліковий запис створено, користувач переходить до кабінету | Відповідає очікуваному | Пройдено |
| 2 | Вхід із неправильним паролем | Система показує повідомлення про помилку авторизації | Відповідає очікуваному | Пройдено |
| 3 | Оформлення заявки на оренду контейнера | Замовлення створюється зі статусом pending_approval | Відповідає очікуваному | Пройдено |
| 4 | Розрахунок вартості оренди через RentalPricingService | Система формує детальну розбивку ціни у форматі JSON | Відповідає очікуваному | Пройдено |
| 5 | Перегляд IoT-показників активної оренди | Відображається дашборд із даними датчиків | Відповідає очікуваному | Пройдено |
| 6 | Надсилання повідомлення через форму Contact | Звернення зберігається в таблиці inquiries | Відповідає очікуваному | Пройдено |
| 7 | Підключення Telegram-бота через код підтвердження | Статус підключення змінюється на active | Відповідає очікуваному | Пройдено |
| 8 | Перегляд фінансового розділу | Транзакції відображаються за статусами | Відповідає очікуваному | Пройдено |

**Модульне тестування**

Автоматизоване тестування виконувалося засобами PHPUnit у тестовому середовищі Laravel [38]. Основну увагу було приділено сервісним класам і логіці роботи системи. Включаючи перевірялися розрахунок вартості оренди, валідація вхідних даних, створення записів у базі даних і коректність роботи окремих моделей [5, 38]. Фрагмент результатів виконання тестів наведено на рис. 4.1

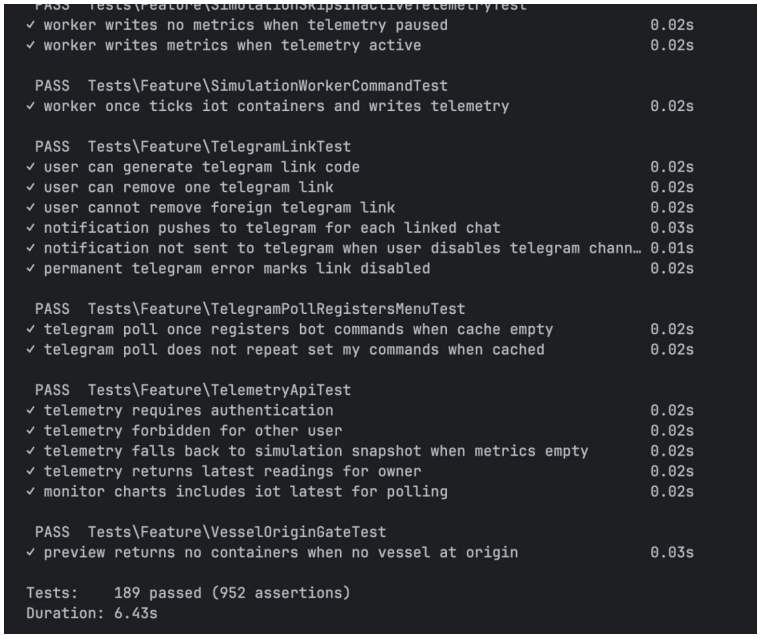



Рис. 4.1 — Вивід результатів PHPUnit-тестів

За результатами запуску автоматизованих тестів усі 189 тестових сценаріїв виконано успішно [38]. У межах тестування було проведено 952 перевірок, помилок не виявлено. Час виконання тестового набору становив приблизно 6,43 секунди, що підтверджує коректну роботу перевірених модулів у межах тесту на сервері.

## 4.2 Формування структури робіт

Для організації роботи над проєктом побудовано ієрархічну структуру робіт (WBS), яка показує як увесь проєкт розбивається на менші частини, хто за що відповідає і в якій послідовності все виконується [40]. Структура наведена в Додатку А на рис. А1 і включає чотири рівні деталізації.

На верхньому рівні — весь проєкт як єдине ціле: платформа, IoT-модуль, логістичний блок і митна підсистема. Другий рівень розбиває це на окремі напрямки — модуль оренди контейнерів, система дистанційного моніторингу, оптимізація маршрутів, автоматизація митних процедур і аналітика. Такий поділ відповідає реальній архітектурі продукту і забезпечує реалізацію різним командам працювати паралельно не заважаючи одне одному.

Третій і четвертий рівні деталізують кожен напрямок до конкретних задач. Модуль оренди охоплює розробку інтерфейсу, інтеграцію з базою даних і підключення платіжних сервісів. IoT-напрямок включає налаштування датчиків, тестування передачі телеметрії і алгоритми контролю умов усередині контейнера. Логістичний модуль деталізується до алгоритмів маршрутизації і перевірки коректності маршрутів.

Митна підсистема — до генератора документів і тестування сценаріїв оформлення вантажів. Кожну задачу можна оцінити окремо за часом і ресурсами та закріпити за конкретним фахівцем.

WBS стала відправною точкою для побудови діаграми Ганта і визначення критичного шляху. Завдяки чіткому розподілу вдалося синхронізувати роботу Backend, Frontend, IoT і DevOps команд і тримати якість під контролем на кожному етапі.

### 4.3 Календарний план та мережевий графік

Календарний план проєкту сформовано в середовищі ProjectLibre на основі WBS, розробленої на попередньому етапі [40]. Усі завдання організовано за дворівневим принципом — верхній рівень охоплює фази проєкту загалом, нижній деталізує кожну до конкретних операцій. Тривалість підзавдань зазначено в робочих днях, а загальна тривалість фаз розраховується автоматично — при зміні будь-якого завдання план перераховується одразу.

На діаграмі Ганта (Додаток Г) відображено часові рамки і залежності між завданнями. Синім виділено роботи з часовим буфером, червоним — критичний шлях, до якого належать завдання системного архітектора, Backend Lead і Frontend Lead. Якщо будь-яке з них затягнеться — кінцева дата проєкту перенесеться на пізніший термін.

Для наочної демонстрації логіки виконання робіт побудовано мережевий графік (PERT-діаграму) у тому ж ProjectLibre. Графік охоплює три послідовні фази.

На першій фазі (Додаток Г рис. Г2) процес стартує з аналізу предметної області — його результати передаються до наступного етапу розробки структурної схеми. Після цього паралельно опрацьовуються вибір IoT-обладнання і моделювання бізнес-процесів. Послідовна структура цього відрізку навмисна — вона знижує ймовірність хибних рішень на архітектурному рівні, які потім дорого виправляти.

Друга фаза (Додаток Г рис. Г3) — найбільш насичена з погляду навантаження. Після того як DevOps-інженер завершує налаштування інфраструктури, роботи розходяться на три незалежні потоки. Backend-команда займається проєктуванням бази даних, реалізацією серверної логіки та підключенням платіжних провайдерів. Frontend-розробники паралельно будують компоненти інтерфейсу. Третій потік — логістика та IoT — охоплює алгоритми маршрутизації, налаштування IoT-модуля і підключення

датчиків. Завдання критичного шляху позначено червоним, решта — синім.

Третя фаза (Додаток Г рис. Г4) збирає всі паралельні потоки в єдине ціле — комплексне тестування і запуск. Після проходження тестування підготовлено документацію і зроблено фінальну здачу проєкту.

Для зручності контролю підготовлено зведену таблицю (Додаток Е рис. Е1), де для кожного завдання вказано тривалість, календарні межі, логічні залежності, відповідальних виконавців і вільний резерв часу. Наприклад, завдання «Дизайн UI/UX» має резерв 10 днів, що дає певну гнучкість без ризику для загального графіка.

Завдання поза критичним шляхом мають такі резерви: налаштування вебсервера — 39 днів, розробка IoT-алгоритмів — 23 дні, інтеграція митного модуля — 16 днів, реалізація IoT-дашборду — 18 днів. Гістограма завантаження ресурсів у нижній частині рис. 4.2 демонструє рівномірний розподіл навантаження по більшості учасників команди. Виняток становлять Backend Lead і Frontend Lead — на критичних відрізках у них немає жодного буфера, що вимагає окремої уваги керівника проєкту

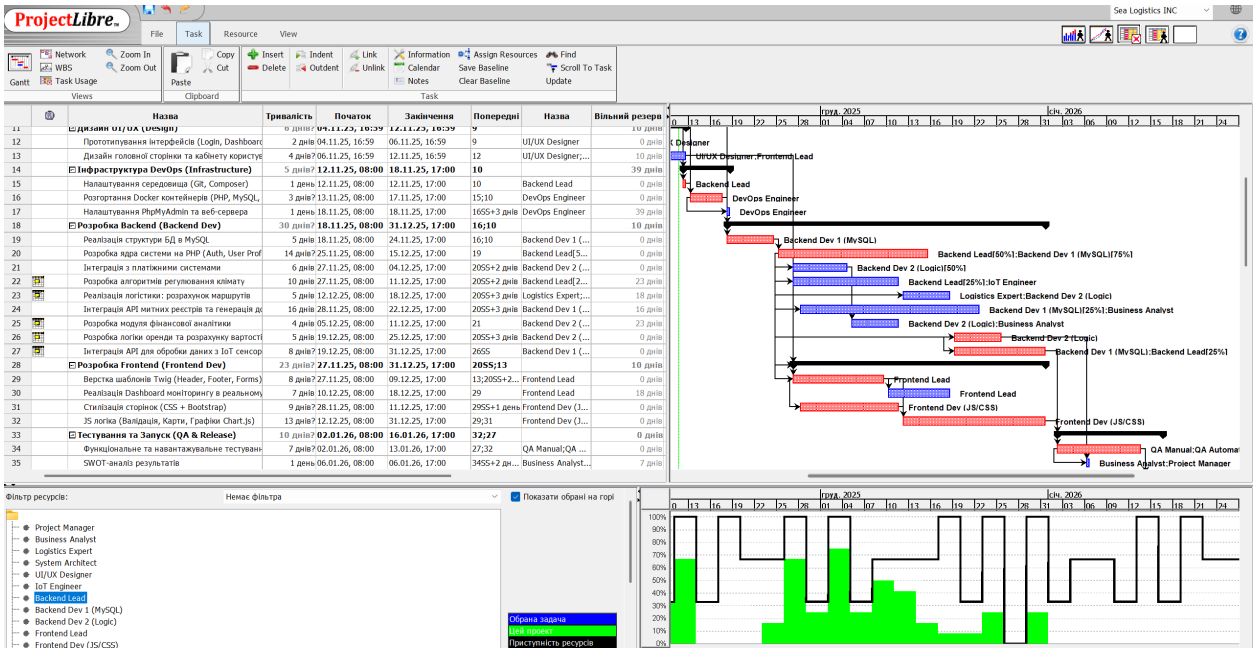



Рис. 4.2 — Діаграма Ганта критичного шляху та завантаження ресурсів

## 4.4 Ресурсне планування та розрахунок трудомісткості

До виконання проєкту залучено команду фахівців, які відповідають за аналітику, проєктування, розробку, тестування та налаштування інфраструктури. Умовно команду можна поділити на кілька груп: управління та аналітика, інженерія й дизайн, backend- і frontend-розробка, тестування та DevOps-супровід.

Графіки завантаження показують, що основне навантаження розподілене

відповідно до етапів розробки. Backend Lead найбільше залучений під час створення серверної логіки та роботи з базою даних. Business Analyst активніше працює на початкових етапах, коли уточнюються вимоги, і на завершальних етапах, коли перевіряється відповідність результату поставленим задачам. Frontend Lead підключається після затвердження структури інтерфейсу та API, коли можна повноцінно реалізовувати клієнтську частину системи рис. 4.3 та рис 4.4.

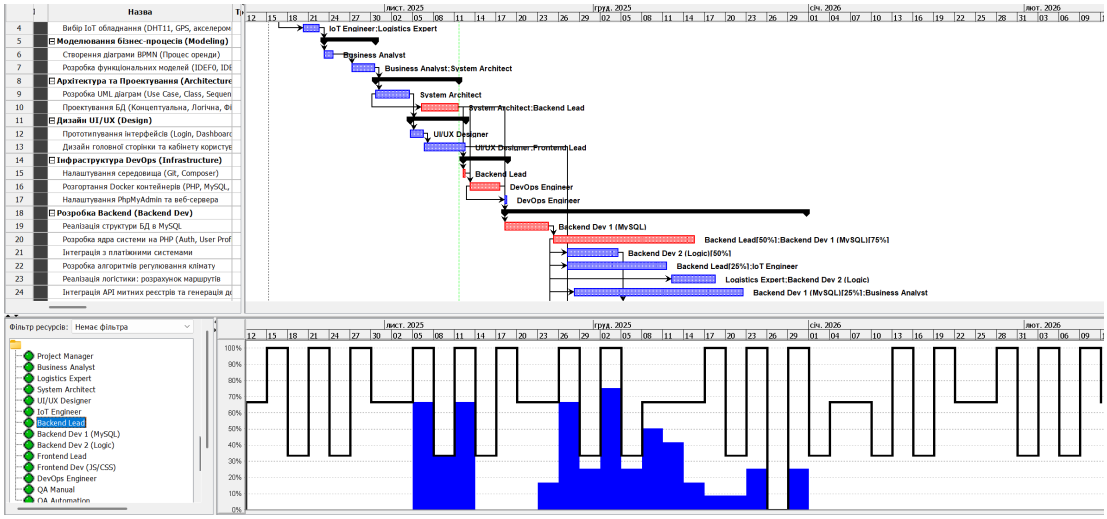



Рис. 4.3 — Графік завантаження Backend Lead на проєкті

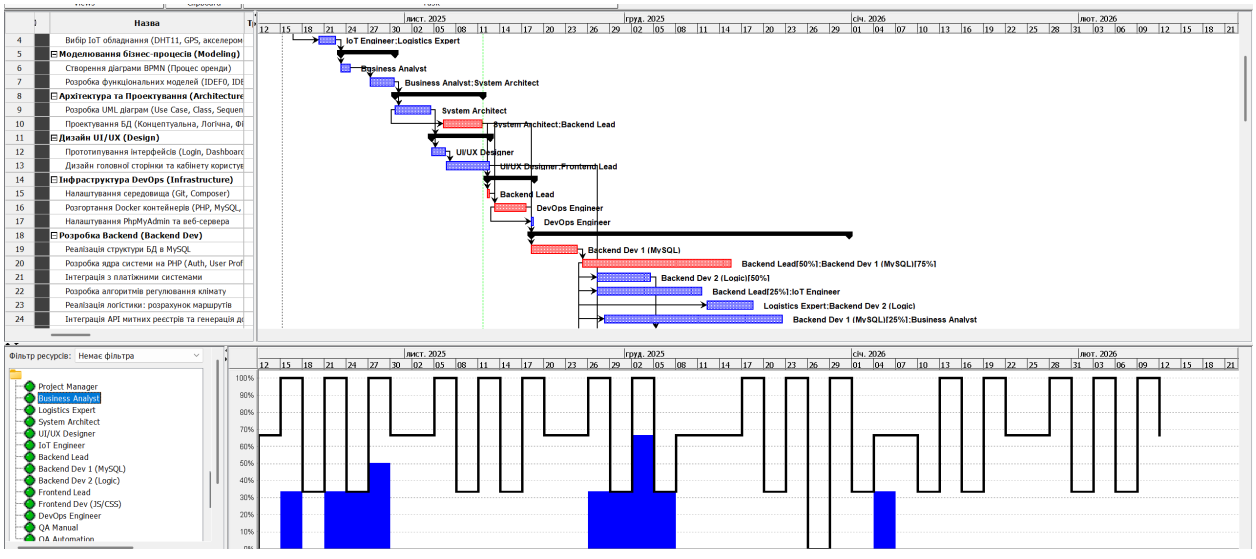



Рис. 4.4 — Розподіл активності бізнес-аналітика протягом проєкту

Технічна база проєкту включає робочі станції для розробників, серверне середовище для запуску застосунку, бази даних і сервісів, а окрім цього IoT-комплекти для перевірки роботи телеметрії. Серверна частина може бути розгорнута у трьох середовищах: для розробки, тестування та експлуатації рис 4.5.

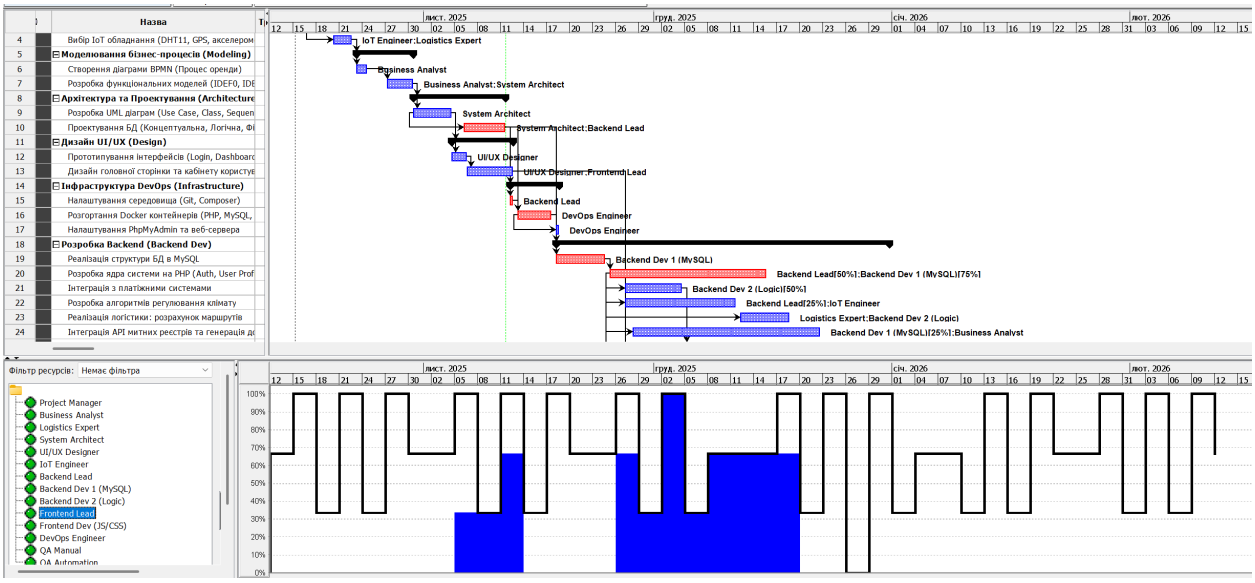



Рис. 4.5 — Графік залучення Frontend Lead до проєкту

Трудомісткість розробки розраховано методом умовних коефіцієнтів [21]. Під час розрахунку враховано функціональні можливості системи, кількість інформаційних потоків, складність бази даних, наявність IoT-моніторингу та потребу в обробці даних у режимі, близькому до реального часу.

Проєкт віднесено до задач типу “Управління технологічними процесами та якістю продукції”, оскільки система виконує моніторинг стану контейнерів і підтримує управління логістичними операціями. За ступенем новизни проєкт можна віднести до групи В, тобто до типових проєктних рішень з окремими елементами оригінальності. Група складності алгоритмів — 3, оскільки система містить маршрутизацію, обробку телеметрії та роботу з різними ролями користувачів. Кількісні параметри інформаційних потоків наведено в таблиці 4.2.

Таблиця 4.2 — Вхідні параметри розрахунку трудомісткості

| Вид інформації | Позначення | Кількість наборів даних / стан |
|---|---|---|
| Змінна інформація | ЗІ | m = 7 |
| Нормативно-довідкова інформація | НДІ | n = 5 |
| База даних | БД | p = 1 |
| Обробка в реальному часі | РЧ | так |
| Телекомунікаційне управління | ТОУ | так |

Таблиця 4.3 — Поправочні коефіцієнти трудомісткості

| Показник | Позначення | Значення | Обґрунтування |
|---|---|---|---|
| Коефіцієнт змінної інформації | $k_1$ | 1 | Стандартна обробка вхідних потоків |
| Коефіцієнт довідкової інформації | $k_2$ | 0.72 | Використання стандартних довідників портів/країн |
| Коефіцієнт бази даних | $k_3$ | 2.08 | Складна структура зв'язків у БД |
| Коефіцієнт трудомісткості робіт | $k_p$ | 0.975 | Розраховано |
| Коеф. складності контролю | $k_k$ | 1.1 | Необхідність валідації даних від IoT-сенсорів |

Коефіцієнт трудомісткості $k_p$ розраховується за формулою 4.1:

$$k_p = \frac{k_1 \cdot m + k_2 \cdot n + k_3 \cdot p}{m+n+p} \quad (4.1)$$

Загальна трудомісткість ($T_s$) проєкту складається з суми витрат часу на кожну стадію життєвого циклу. Базові значення ($T_\beta$) взяті з нормативних таблиць для відповідної кількості форм (m = 7, n = 5) [21].

*Стадія 1: Передпроєктне дослідження ($T_1$)* - Включає аналіз предметної області та ініціацію.

$$T_1 = 67 \text{ люд.-днів}$$

*Стадія 2: Технічне завдання ($T_2$)* - Розробка вимог та специфікацій.

$$T_2 = 31 \text{ люд.-день}$$

*Стадія 3: Технічний проєкт ($T_3$)* - Проєктування архітектури, БД та інтерфейсів.

$$T_3 = T_{\beta 3} \cdot k_p \cdot k_{o3} = 177 \cdot 0{,}975 \cdot 1.26 = 217.4 \text{ люд.-день}$$

*Стадія 4: Робочий проєкт ($T_4$)* - Написання коду (Backend/Frontend), інтеграція IoT, тестування. Це найбільш трудомістка стадія.

$$T_4 = T_{\beta 4} \cdot k_p \cdot k_{o4} \cdot k_k = 307 \cdot 0{,}975 \cdot 1.32 \cdot 1 = 395.1 \text{ люд.-днів}$$

*Стадія 5: Впровадження ($T_5$)* - Розгортання на серверах, навчання персоналу, здача в експлуатацію.

$$T_5 = T_{\beta 5} \cdot k_p \cdot k_{o5} \cdot k_k = 74 \cdot 0.975 \cdot 1.21 \cdot 1 = 87.3 \text{ люд.-днів}$$

Сумарна трудомісткість проєкту ($T_s$) становить:

$$T_s = 67 + 31 + 218 + 396 + 87 = 799 \text{ люд.-днів}$$

Згідно з розрахунками в лабораторній роботі, ефективний фонд робочого часу одного виконавця на весь проєкт становить $\Phi = 530$годин. Оскільки фактичне навантаження менше за доступний фонд, ресурсів команди достатньо для виконання завдань без переробок.

Розрахуємо календарну тривалість проєкту $M$, виходячи з місячної норми робочого часу 165 годин (22 робочих дні ×7.5 годин):

$$\Phi = \frac{530}{7.5} = 70.6$$

$$M = \frac{\Phi}{22} = \frac{70.6}{22} \approx 3.2 \text{ місяця}$$

Отже, для виконання такого проєкту потрібно таку чисельність виконавців $\chi$, виконавців, обраховується за:

$$\chi = \frac{T_\Sigma}{\Phi} = \frac{799}{70.6} \approx 12$$

Сумарна трудомісткість проєкту $T_\Sigma$, розрахована за нормативами, становить 799 люд.-днів. Переведемо цей показник у людино-години, враховуючи тривалість робочого дня $t_{\text{day}} = 7.5$годин:

$$T_{\text{hours}} = T_\Sigma \times t_{\text{day}} = 799 \times 7.5 = 5992.5 \text{ людино-годин}$$

$$\text{Навантаження} = \frac{5992.5}{14} \approx 428 \text{ годин/люд}$$

Розрахунковий термін реалізації проєкту становить 3.2 місяця. Це узгоджується з директивним терміном (3 місяці) та підтверджує, що команда з 14 фахівців здатна виконати весь обсяг робіт у межах встановленого фонду робочого часу з різних галузей.

### 4.5 Бюджет проєкту

Бюджет проєкту Logistics SWay сформовано з урахуванням основних витрат, необхідних для розробки та запуску системи. До них належать оплата праці команди, закупівля техніки, експлуатаційні витрати та навчання представників замовника. Розрахунок виконано для команди з 14 фахівців на тримісячний період роботи над проєктом [1, 21].

**Фонд оплати праці**

Основною статтею бюджету є оплата праці проєктної команди [1]. До неї входять

фахівці з управління, аналітики, архітектури, розробки, тестування, DevOps-супроводу та IoT-напряму.

Загальний фонд оплати праці за весь період реалізації становить 2 850 000 грн таблиця 4.4.

Таблиця 4.4 — Витрати на оплату праці проєктної команди

| Посада | Місячна ставка, грн | Витрати за 3 міс., грн |
|---|---|---|
| Project Manager | 60 000 | 180 000 |
| Business Analyst | 55 000 | 165 000 |
| Logistics Expert | 50 000 | 150 000 |
| System Architect | 90 000 | 270 000 |
| UI/UX Designer | 55 000 | 165 000 |
| IoT Engineer | 70 000 | 210 000 |
| Backend Lead | 85 000 | 255 000 |
| Backend Dev 1 (DB) | 75 000 | 225 000 |
| Backend Dev 2 | 75 000 | 225 000 |
| Frontend Lead | 80 000 | 240 000 |
| Frontend Dev | 65 000 | 195 000 |
| DevOps Engineer | 85 000 | 255 000 |
| QA Manual | 45 000 | 135 000 |
| QA Automation | 60 000 | 180 000 |
| **Разом** | **950 000** | **2 850 000** |

**Витрати на технічне забезпечення ($V_2$)**

Для роботи команди закуплено 14 комплектів комп'ютерної техніки. Первісна вартість одного ПК — 40 000 грн, коефіцієнт оновлення $k_{ун} = 0.12$.

Балансова вартість одного ПК:

$$Ц_{ПК} = Ц_Р \times (1 + k_{ун}) = 40\,000 \times 1.12 = 44\,800 \text{ грн}$$

Загальні витрати на техніку для 14 осіб:

$$V_2 = 44\,800 \times 14 = 627\,200 \text{ грн}$$

**Експлуатаційні витрати ($V_1$)**

Охоплюють амортизацію, електроенергію та поточний ремонт за 3 місяці розробки. Норма амортизації $Н_а = 5\%$.

$$З_{АМ} = 627\,200 \times 0.05 = 31\,360 \text{ грн}$$

Витрати на електроенергію (потужність парку 3.5 кВт, фонд часу 495 год, тариф 9.56 грн/кВт·год, коефіцієнт реалізації 0.7):

$$З_{ЕЛ} = 3.5 \times 495 \times 9.56 \times 0.7 \approx 11\,594 \text{ грн}$$

Витрати на ремонт та матеріали (6% і 5% від балансової вартості):

$$З_{Р+МАТ} = 627\,200 \times 0.11 = 68\,992 \text{ грн}$$

Сумарні експлуатаційні витрати:

$$V_1 = 31\,360 + 11\,594 + 68\,992 = 111\,946 \text{ грн}$$

**Витрати на навчання ($V_4$)**

Підготовка приміщення не передбачена — задіяно наявний офіс. Навчання проводиться для 5 ключових представників замовника:

$$V_4 = 5 \times 10\,000 = 50\,000 \text{ грн}$$

**Зведений бюджет проєкту**

$$V_\Sigma = \text{ФОП} + V_2 + V_1 + V_4 = 2\,850\,000 + 627\,200 + 111\,946 + 50\,000$$
$$= 3\,639\,146 \text{ грн}$$

Основну частку витрат — близько 78% — складає фонд оплати праці, що є характерним для IT-проєктів з високою інтелектуальною складовою. Витрати на технічне забезпечення є разовими інвестиціями, що забезпечать команду необхідними інструментами на весь тримісячний термін розробки. Середнє завантаження членів команди становить 85–90%, що залишає достатній буфер для комунікації і вирішення непередбачуваних технічних питань без ризику зриву дедлайнів таблиця 4.5.

Таблиця 4.5 — Зведений бюджет

| **Стаття витрат** | **Сума, грн** | **Частка, %** |
|---|---|---|
| Фонд оплати праці | 2 850 000 | 78,3 |
| Технічне забезпечення | 627 200 | 17,2 |
| Експлуатаційні витрати | 111 946 | 3,1 |
| Навчання персоналу | 50 000 | 1,4 |
| **Разом** | **3 639 146** | **100** |

### 4.6 Економічна ефективність впровадження

Оцінку економічної ефективності впровадження платформи Logistics SWay виконано на основі кількох показників: чистого приведеного доходу NPV, внутрішньої норми доходності IRR, рентабельності інвестицій ROI та терміну окупності PP [22]. Ці показники дозволяють оцінити, наскільки доцільним є впровадження системи з фінансової точки зору.

Загальний обсяг інвестицій у проєкт, з урахуванням оплати праці команди, закупівлі техніки, експлуатаційних витрат і навчання персоналу, становить:

$$V_{\Sigma} = 3\,639\,146 \text{ грн}$$

Тривалість інвестиційної фази становить приблизно 3,2 місяця. У цей період система ще не приносить доходу, оскільки виконується її розробка, тестування та підготовка до впровадження. Для приведення майбутніх доходів до поточної вартості використано ставку дисконту:

$$E = 20\%$$

Після запуску системи очікується отримання економічного ефекту за рахунок автоматизації оренди контейнерів, зменшення кількості ручних операцій, скорочення часу обробки заявок і кращого контролю за станом вантажів. Додаткову користь може дати IoT-моніторинг, оскільки він дає можливість швидше виявляти проблеми з температурою, вологістю, ударами, нахилом або відкриттям контейнера.

*Розрахунок чистого приведеного доходу NPV*

Для розрахунку прийнято, що протягом трьох років експлуатації система щороку приноситиме 5 500 000 грн доходу. Дисконтований дохід за роками становить:

$$\frac{5\,500\,000}{1{,}2} = 4\,583\,333 \text{ грн}$$

$$\frac{5\,500\,000}{1{,}2^2} = 3\,819\,444 \text{ грн}$$

$$\frac{5\,500\,000}{1{,}2^3} = 3\,182\,870 \text{ грн}$$

Сумарний дисконтований дохід за три роки:

$$\Sigma = 4\,583\,333 + 3\,819\,444 + 3\,182\,870 = 11\,585\,647 \text{ грн}$$

Чистий приведений дохід:

$$NPV = 11\,585\,647 - 3\,639\,146 = 7\,946\,501 \text{ грн}$$

Отримане значення NPV > 0, тому за заданих умов проєкт можна вважати економічно доцільним.

*Річний економічний ефект*

Річний економічний ефект визначається з урахуванням нормативної ставки ефективності за формулою 4.2:

$$E_{\text{річ}} = P_{\text{річ}} - V_{\Sigma} \cdot E_{H} \tag{4.2}$$

$$E_{\text{річ}} = 5\,500\,000 - 3\,639\,146 \cdot 0{,}2 = 4\,772\,170{,}8 \text{ грн}$$

Отже, прогнозований річний економічний ефект становить приблизно 4 772 171 грн.

Коефіцієнт економічної ефективності за формулою 4.3

$$K_{\text{еф}} = \frac{P_{\text{річ}}}{V_{\Sigma}} \tag{4.3}$$

$$K_{\text{еф}} = \frac{5\,500\,000}{3\,639\,146} \approx 1{,}51$$

Це означає, що на кожну вкладену гривню припадає приблизно 1,51 грн прогнозованого річного доходу.

*Внутрішня норма доходності IRR*

Внутрішня норма доходності показує таку ставку дисконту, за якої чистий приведений дохід дорівнює нулю:

$$0 = \sum_{t=1}^{3} \frac{5\,500\,000}{(1+IRR)^{t}} - 3\,639\,146$$

За наведених вхідних даних значення IRR перевищує 100%, оскільки прогнозовані щорічні надходження більші за початкові інвестиції, а термін окупності становить менше одного року. Це свідчить про високий запас фінансової стійкості проєкту за умови досягнення запланованих доходів.

*Рентабельність інвестицій ROI*

$$ROI = \frac{16\,500\,000 - 3\,639\,146}{3\,639\,146} \cdot 100\%$$

$$ROI \approx 353\%$$

Рентабельність інвестицій за три роки становить приблизно 353%. Це означає, що за прогнозованими показниками система може не лише повернути вкладені кошти,

а й забезпечити значний додатковий дохід.

*Термін окупності PP*

$$PP = \frac{3\,639\,146}{5\,500\,000} \approx 0{,}66 \text{ року}$$

$$0{,}66 \text{ року} \approx 8 \text{ місяців}$$

Отже, орієнтовний термін окупності проєкту становить близько 8 місяців.

За результатами розрахунків можна зробити висновок, що впровадження платформи Logistics SWay є економічно доцільним за умови досягнення прогнозованого рівня доходів. Проєкт має позитивне значення NPV, високий показник ROI та короткий термін окупності таблиця 4.6. Найбільший економічний ефект очікується за рахунок автоматизації процесу оренди, зменшення ручної роботи, швидшої обробки замовлень і вдосконалення прозорості контролю за контейнерами під час перевезення.

Таблиця 4.6 — Зведені показники економічної ефективності

| **Показник** | **Значення** |
|---|---|
| Загальні інвестиції (V∑) | 3 639 146 грн |
| Чистий приведений дохід (NPV) | 7 946 501 грн |
| Річний економічний ефект (Еріч) | 4 772 170,8 грн |
| Коефіцієнт ефективності (Кеф) | 1,51 |
| Рентабельність (ROI, 3 роки) | 353% |
| Термін окупності (PP) | 8 місяців |

**Висновки до четвертого розділу**

У цьому розділі наведено огляд результатів тестування, виконано планування робіт, розраховано трудомісткість, бюджет проєкту та показники економічної ефективності.

У межах тестування перевірив основні модулі системи: авторизацію користувачів, оформлення оренди контейнерів, IoT-моніторинг, інтеграцію з тг-ботом, телеметричний API та фінансовий облік. Ручне тестування показало, що основні користувацькі сценарії працюють відповідно до очікуваних результатів. Автоматизоване тестування у тому числі пройшло успішно: виконано 189 тестів і 952 перевірки, помилок не виявлено.

Для організації робіт було сформовано WBS-структуру, побудовано діаграму

Ганта та мережевий графік у середовищі ProjectLibre. Це дало змогу визначити послідовність виконання етапів, розподілити завдання між учасниками команди та встановити критичний шлях проєкту. Критичний шлях проходить через налаштування інфраструктури, розробку серверної частини та створення клієнтського інтерфейсу. Окремі задачі, які не входять до критичного шляху, мають резерв часу від 10 до 39 днів.

Для реалізації проєкту передбачено команду з 14 фахівців. Розрахована трудомісткість становить 799 люд.-днів, що відповідає орієнтовному строку виконання 3,2 місяця. Загальна трудомісткість у людино-годинах становить 5992,5 людино-годин, а середнє навантаження на одного учасника команди — приблизно 428 годин.

Загальний бюджет проєкту становить 3 639 146 грн. Найбільшу частку витрат займає фонд оплати праці — 78,3% від загальної суми, що є типовим для IT-проєктів, де основні витрати пов’язані з роботою команди розробки, аналітики, тестування та налаштування інфраструктури.

Економічні розрахунки показали, що за прогнозованого рівня доходів впровадження платформи є доцільним. Чистий приведений дохід NPV за три роки становить 7 946 501 грн, рентабельність інвестицій ROI — 353%, а орієнтовний термін окупності — близько 8 місяців. Такі показники свідчать, що проєкт має потенціал для подальшого впровадження та використання як основи у сфері міжнародної контейнерної логістики.

# ВИСНОВКИ

У кваліфікаційній роботі розроблено IoT-систему забезпечення перевезень морським транспортом — повнофункціональну веб-платформу Logistics SWay, що автоматизує процеси оренди контейнерів і надає змогу дистанційно відстежувати стан вантажів з регулярним оновленням даних.

У першому розділі досліджено предметну область морської контейнерної логістики, проаналізовано організаційну структуру підприємства і сформовано дерево цілей. Огляд існуючих ринкових рішень підтвердив, що жодна з розглянутих платформ не об'єднує оренду контейнерів, IoT-моніторинг і фінансовий облік в єдиному середовищі. Визначено перелік апаратних засобів IoT-системи і побудовано BPMN-модель процесу оренди.

У другому розділі спроєктовано архітектуру системи, побудовано функціональні та UML-діаграми, розроблено реляційну модель бази даних із 35 таблицями. Окремо реалізовано партиціонування таблиці телеметрії і криптографічний ланцюжок аудиту IoT-подій, що унеможливлює підробку зафіксованих показників після їх запису.

У третьому розділі описано програмну реалізацію платформи на сучасному технологічному стеку з використанням контейнеризації. Реалізовано алгоритм розрахунку вартості оренди за двадцятьма складовими, модуль IoT-моніторингу, інтеграцію з Telegram-ботом та адміністративну панель. Наведено приклади задіяння системи й інструкцію користувача. За результатами SWOT-аналізу визначено конкурентні переваги платформи і напрямки подальшого розвитку.

У четвертому розділі проведено тестування — 189 тестів і 952 перевірки пройдено без жодної помилки. Виконано планування проєкту із загальним бюджетом 3 639 146 грн і терміном реалізації 3.2 місяця. Розрахунок економічної ефективності підтвердив інвестиційну привабливість розробки: чистий приведений дохід за три роки становить 7 946 501 грн, рентабельність — 353%, а термін повернення вкладених коштів — 8 місяців.

Поставлена мета досягнута — платформа Logistics SWay є готовим до впровадження рішенням, що закриває реальні потреби цифровізації морської логістики і має підтверджену практичну та економічну цінність.

# СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ


1. Пономаренко В. С. Проектування інформаційних систем : посіб. / В. С. Пономаренко (ред.). – Київ : Академія, 2012. – 488 с.

2. Костенко С. В., Гнатюк С. О. IoT-рішення в системах моніторингу: технічні та безпекові аспекти [Електронний ресурс]. – Системний аналіз та інформаційні технології. – 2021. – № 3. – С. 125–139. – Режим доступу: https://sait.kpi.ua (дата звернення: 05.02.2026).

3. Черненко О. І. Цифровізація логістичних процесів: світові тенденції та перспективи для України [Електронний ресурс]. – Економіка та суспільство. – 2022. – № 38. – Режим доступу: https://economyandsociety.in.ua/index.php/journal/article/view/1456 (дата звернення: 05.02.2026).

4. ДСТУ ISO/IEC 12207:2014. Системна та програмна інженерія. Процеси життєвого циклу програмного забезпечення. – Київ : ДП «УкрНДНЦ», 2014. – 97 с.

5. ДСТУ ISO/IEC 25010:2013. Системна та програмна інженерія. Вимоги та оцінювання якості систем і програмного забезпечення. – Київ : ДП «УкрНДНЦ», 2013. – 32 с.

6. Al-Fuqaha A. et al. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications [Електронний ресурс]. – IEEE Communications Surveys & Tutorials. – 2015. – Vol. 17, № 4. – P. 2347–2376. – Режим доступу: https://ieeexplore.ieee.org/document/7123563 (дата звернення: 25.01.2026).

7. Atzori L., Iera A., Morabito G. The Internet of Things: A Survey [Електронний ресурс]. – Computer Networks. – 2010. – Vol. 54, № 15. – P. 2787–2805. – Режим доступу: https://www.sciencedirect.com/science/article/abs/pii/S1389128610001568 (дата звернення: 25.01.2026).

8. Bandyopadhyay D., Sen J. Internet of Things: Applications and Challenges in Technology and Standardization [Електронний ресурс]. – Wireless Personal Communications. – 2011. – Vol. 58, № 1. – P. 49–69. – Режим доступу: https://link.springer.com/article/10.1007/s11277-011-0288-5 (дата звернення: 01.02.2026).

9. Bass L., Clements P., Kazman R. Software Architecture in Practice [Електронний ресурс]. – 3rd ed. – Boston : Addison-Wesley, 2012. – 624 p. – Режим

доступу: https://www.informit.com/store/software-architecture-in-practice-9780136886099 (дата звернення: 05.03.2026).

10. Elmasri R., Navathe S. Fundamentals of Database Systems [Електронний ресурс]. – 7th ed. – London : Pearson, 2015. – 1280 p. – Режим доступу: https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003471 (дата звернення: 20.01.2026).

11. EN 12830:1999. Temperature recorders for the transport, storage and distribution of chilled, frozen, deep-frozen food and ice cream [Електронний ресурс]. – Brussels : CEN, 1999. – Режим доступу: https://standards.cen.eu/dyn/www/f?p=204:110:0::::FSP_PROJECT:1661 (дата звернення: 20.02.2026).

12. Fowler M. Patterns of Enterprise Application Architecture [Електронний ресурс]. – Boston : Addison-Wesley, 2002. – 560 p. – Режим доступу: https://martinfowler.com/books/eaa.html (дата звернення: 20.01.2026).

13. Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language [Електронний ресурс]. – 3rd ed. – Boston : Addison-Wesley, 2003. – 208 p. – Режим доступу: https://martinfowler.com/books/uml.html (дата звернення: 20.01.2026).

14. GSMA. LTE-M Deployment Guide to Basic Feature Set Requirements [Електронний ресурс]. – London : GSMA, 2019. – Режим доступу: https://www.gsma.com/iot/resources/lte-m-deployment-guide (дата звернення: 20.02.2026).

15. ISO 28000:2022 — Security and resilience — Security management systems — Requirements [Електронний ресурс]. – Geneva : ISO, 2022. – Режим доступу: https://www.iso.org/standard/79612.html (дата звернення: 10.03.2026).

16. Kim G., Debois P., Willis J., Humble J. The DevOps Handbook [Електронний ресурс]. – Portland : IT Revolution Press, 2016. – 480 p. – Режим доступу: https://itrevolution.com/product/the-devops-handbook (дата звернення: 25.01.2026).

17. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. – Boston : Prentice Hall, 2017. – 432 p.

18. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. – Boston : Prentice Hall, 2008. – 431 p.

19. Object Management Group. Business Process Model and Notation (BPMN). Version 2.0 [Електронний ресурс]. – 2011. – Режим доступу: https://www.omg.org/spec/BPMN/2.0 (дата звернення: 15.02.2026).

20. Object Management Group. Unified Modeling Language Specification. Version 2.5.1 [Електронний ресурс]. – 2017. – Режим доступу: https://www.omg.org/spec/UML/2.5.1 (дата звернення: 15.02.2026).

21. Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach [Електронний ресурс]. – 8th ed. – New York : McGraw-Hill Education, 2019. – 976 p. – Режим доступу: https://www.amazon.com/Software-Engineering-Practitioners-Approach-Pressman-ebook/dp/B00HZ3BD5G (дата звернення: 15.01.2026).

22. Sommerville I. Software Engineering [Електронний ресурс]. – 10th ed. – London : Pearson, 2016. – 816 p. – Режим доступу: https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258 (дата звернення: 15.01.2026).

23. UNCTAD. Review of Maritime Transport 2023 [Електронний ресурс]. – Geneva : United Nations, 2023. – Режим доступу: https://unctad.org/publication/review-maritime-transport-2023 (дата звернення: 25.02.2026).

24. Wortmann F., Flüchter K. Internet of Things: Technology and Value Added [Електронний ресурс]. – Business & Information Systems Engineering. – 2015. – Vol. 57, № 3. – P. 221–224. – Режим доступу: https://link.springer.com/article/10.1007/s12599-015-0383-3 (дата звернення: 01.02.2026).

25. World Shipping Council. Top 50 World Container Ports [Електронний ресурс]. – 2023. – Режим доступу: https://www.worldshipping.org/top-50-ports (дата звернення: 25.02.2026).

26. Apache Kafka Documentation [Електронний ресурс]. – Режим доступу: https://kafka.apache.org/documentation (дата звернення: 14.02.2026).

27. Cloudflare DNS Documentation [Електронний ресурс]. – Режим доступу: https://developers.cloudflare.com/dns (дата звернення: 14.02.2026).

28. Composer — Dependency Manager for PHP. Офіційна документація [Електронний ресурс]. – Режим доступу: https://getcomposer.org/doc (дата звернення: 13.02.2026).

29. Docker Documentation [Електронний ресурс]. – Режим доступу: https://docs.docker.com (дата звернення: 12.02.2026).

30. draw.io — Diagrams.net Documentation [Електронний ресурс]. – Режим доступу: https://www.drawio.com/doc (дата звернення: 01.03.2026).

31. Dynadot — Domain Registration and Management Help Center [Електронний ресурс]. – Режим доступу: https://www.dynadot.com/community/help (дата звернення: 01.03.2026).

32. IDEF0 — Integration Definition for Function Modeling. Federal Information Processing Standards Publication 183 [Електронний ресурс]. – National Institute of Standards and Technology, 1993. – Режим доступу: https://www.idef.com/idef0-function_modeling_method (дата звернення: 15.02.2026).

33. Inertia.js — Build single-page apps without building an API. Офіційна документація [Електронний ресурс]. – Режим доступу: https://inertiajs.com (дата звернення: 11.02.2026).

34. Laravel — The PHP Framework For Web Artisans. Офіційна документація [Електронний ресурс]. – Режим доступу: https://laravel.com/docs (дата звернення: 10.02.2026).

35. Nginx Documentation [Електронний ресурс]. – Режим доступу: https://nginx.org/en/docs (дата звернення: 13.02.2026).

36. npm Documentation — Node Package Manager [Електронний ресурс]. – Режим доступу: https://docs.npmjs.com (дата звернення: 05.03.2026).

37. PHP: Hypertext Preprocessor. Офіційна документація [Електронний ресурс]. – Режим доступу: https://www.php.net/docs.php (дата звернення: 10.02.2026).

38. PHPUnit — The PHP Testing Framework. Офіційна документація [Електронний ресурс]. – Режим доступу: https://phpunit.de/documentation.html (дата звернення: 13.02.2026).

39. PostgreSQL: The World's Most Advanced Open Source Relational Database. Офіційна документація [Електронний ресурс]. – Режим доступу: https://www.postgresql.org/docs (дата звернення: 12.02.2026).

40. ProjectLibre — Open Source Project Management [Електронний ресурс]. – Режим доступу: https://www.projectlibre.com/product/projectlibre-open-source (дата звернення: 01.03.2026).

41. Redis — The Real-time Data Platform. Офіційна документація [Електронний ресурс]. – Режим доступу: https://redis.io/docs (дата звернення: 12.02.2026).

42. Starlink — Technical Specifications and Coverage [Електронний ресурс]. – SpaceX, 2024. – Режим доступу: https://www.starlink.com/specifications (дата звернення: 10.03.2026).

43. Tailwind CSS — A utility-first CSS framework. Офіційна документація [Електронний ресурс]. – Режим доступу: https://tailwindcss.com/docs (дата звернення: 11.02.2026).

44. Telegram Bot API. Офіційна документація [Електронний ресурс]. – Режим доступу: https://core.telegram.org/bots/api (дата звернення: 14.02.2026).

45. Vite — Next Generation Frontend Tooling. Офіційна документація [Електронний ресурс]. – Режим доступу: https://vitejs.dev/guide (дата звернення: 11.02.2026).

46. Vue.js — The Progressive JavaScript Framework. Офіційна документація [Електронний ресурс]. – Режим доступу: https://vuejs.org/guide/introduction (дата звернення: 10.02.2026).

# ДОДАТОК А

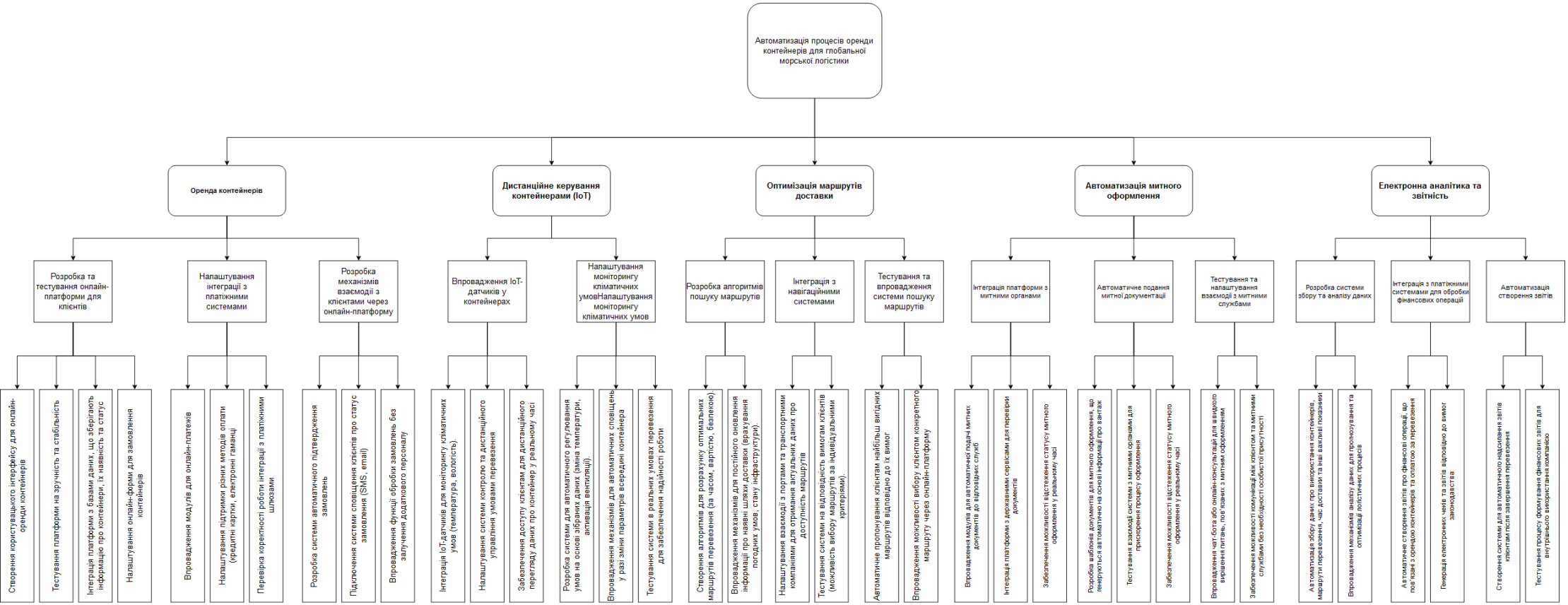



Рисунок А.1 – WBS дерево проєкту

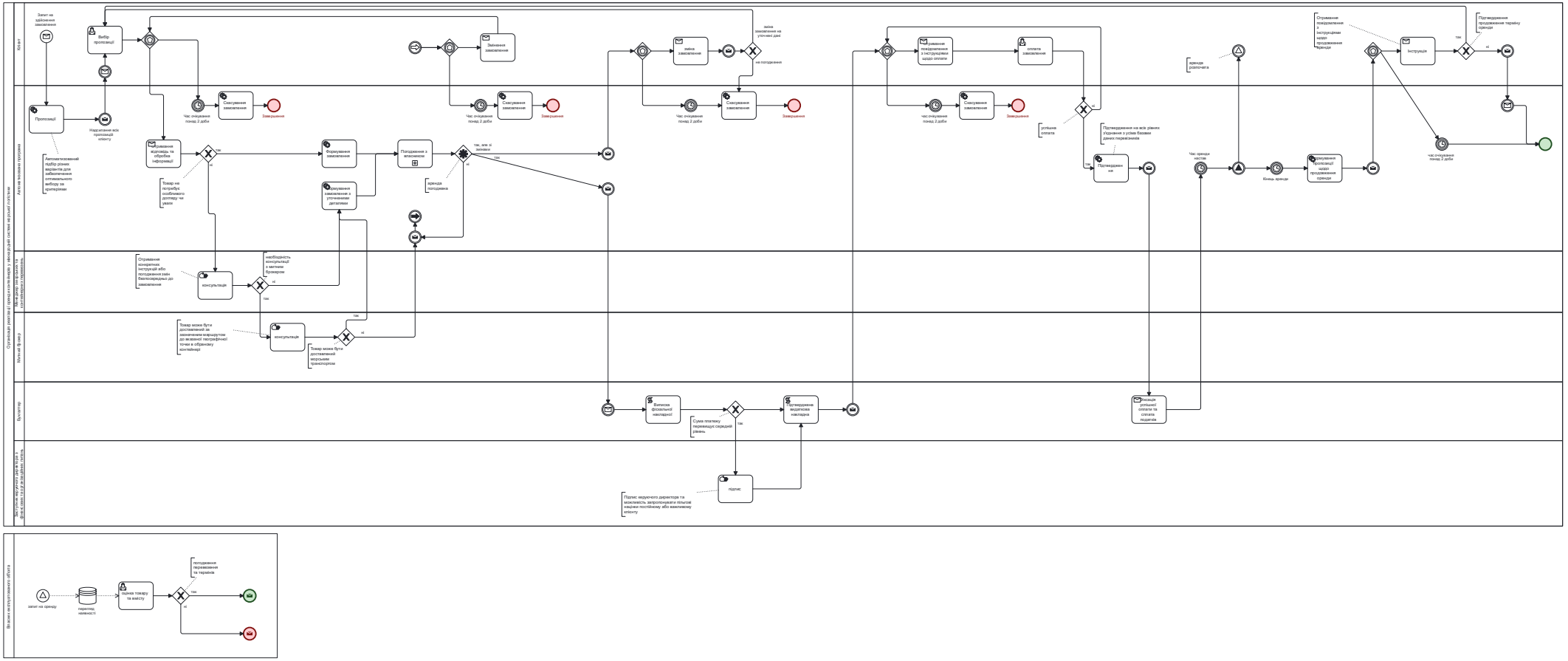


Рис. Б.1 — BPMN-діаграма процесу оренди контейнера в системі Logistics SWay

# ДОДАТОК В

Таблиця В.1 — полів в базі даних

| Таблиця | Основні поля | Типи даних | Призначення |
|---|---|---|---|
| **countries** | id, name, iso_code, phone_code, interest_tax | BIGSERIAL, VARCHAR(100), VARCHAR(3), VARCHAR(10), NUMERIC(5,2) | Довідник країн з ISO-кодом та податковою ставкою для розрахунку вартості оренди. |
| **users** | id, first_name, last_name, email, role, account_status, country_id, bonus_type, bonus_value, deleted_at | BIGSERIAL, VARCHAR(50), VARCHAR(50), VARCHAR(255), VARCHAR(50), VARCHAR(50), BIGINT (FK), VARCHAR(20), NUMERIC(12,2), TIMESTAMP | Користувачі системи — клієнти та персонал. Підтримується м'яке видалення та бонусна система. |
| **owners** | id, name, email, phone_number, deleted_at | BIGSERIAL, VARCHAR(100), VARCHAR(254), VARCHAR(20), TIMESTAMP | Власники контейнерів. Підтримується м'яке видалення. |
| **ports** | id, country_id, name, city, latitude, longitude | BIGSERIAL, BIGINT (FK), VARCHAR(100), VARCHAR(100), NUMERIC(10,7), NUMERIC(10,7) | Морські порти з географічними координатами для маршрутизації та відображення на карті. |
| **vessels** | id, name, imo_number, capacity_teu, status, current_port_id, berth_busy_until, out_of_service_until | BIGSERIAL, VARCHAR(100), VARCHAR(20), INTEGER, VARCHAR(50), BIGINT (FK), TIMESTAMPTZ, TIMESTAMPTZ | Судна з IMO-номером, місткістю та часовими мітками технічних обмежень. |
| **containers** | id, serial_number, type, width, length, height, max_weight, iot_active, current_status, owner_id, current_port_id | BIGSERIAL, VARCHAR(50), VARCHAR(50), NUMERIC(8,2), NUMERIC(8,2), NUMERIC(8,2), NUMERIC(10,2), BOOLEAN, VARCHAR(50), BIGINT (FK), BIGINT (FK) | Контейнери з габаритами, ознакою IoT та поточним статусом. Прив'язані до власника і порту. |
| **sensor_types** | id, slug, name, category, is_optional, telemetry_keys | BIGSERIAL, VARCHAR(64), VARCHAR(120), VARCHAR(50), BOOLEAN, JSON | Каталог типів IoT-датчиків з параметрами вимірювання у форматі JSON. |
| **container_sensors** | id, container_id, sensor_type_id, enabled, config | BIGSERIAL, BIGINT (FK), BIGINT (FK), BOOLEAN, JSON | Прив'язка датчиків до контейнерів з індивідуальними налаштуваннями. Унікальна пара контейнер–тип. |
| **maintenances** | id, container_id, maintenance_date, maintenance_type, description, cost, technician_name | BIGSERIAL, BIGINT (FK), TIMESTAMP, VARCHAR(50), TEXT, NUMERIC(15,2), VARCHAR(100) | Сервісна історія контейнерів. Видаляється каскадно при видаленні контейнера. |
| **routes** | id, origin_port_id, destination_port_id, estimated_days, distance, sea_path, route_status | BIGSERIAL, BIGINT (FK), BIGINT (FK), INTEGER, FLOAT, JSON, VARCHAR(50) | Маршрути між портами з розрахунковою тривалістю та геометрією шляху у форматі JSON. |
| **rentals** | id, user_id, container_id, route_id, origin_port_id, destination_port_id, start_date, end_date, rental_days, cargo_types, price, price_breakdown, status, payment_status, is_telemetry_active | BIGSERIAL, BIGINT (FK), BIGINT (FK), BIGINT (FK), BIGINT (FK), BIGINT (FK), TIMESTAMP, TIMESTAMP, INTEGER, JSON, NUMERIC(15,2), JSON, VARCHAR(50), VARCHAR(50), BOOLEAN | Центральний операційний об'єкт системи. Фіксує учасників, маршрут, вантаж, митні дані, умови доставки та фінансові розрахунки. |
| **rental_route_segments** | id, rental_id, segment_order, from_port_id, to_port_id, route_id, vessel_id, planned_departure_at, travel_duration_hours, status | BIGSERIAL, BIGINT (FK), SMALLINT, BIGINT (FK), BIGINT (FK), BIGINT (FK), BIGINT (FK), TIMESTAMPTZ, SMALLINT, VARCHAR(30) | Деталізація маршруту оренди по окремих ділянках із часовими буферами очікування. |
| **shipments** | id, vessel_id, route_id, leg_sequence, departure_date, arrival_date, tracking_number, status | BIGSERIAL, BIGINT (FK), BIGINT (FK), SMALLINT, TIMESTAMP, TIMESTAMP, VARCHAR(50), VARCHAR(50) | Відправлення з унікальним номером відстеження та фактичними датами руху. |

| | | | |
|---|---|---|---|
| **shipment_items** | id, shipment_id, container_id, rental_id, loaded_at, condition_on_arrival | BIGSERIAL, BIGINT (FK), BIGINT (FK), BIGINT (FK), TIMESTAMP, VARCHAR(50) | Вміст відправлень — контейнери з оцінкою стану вантажу при прибутті. |
| **cargo_manifests** | id, rental_id, name, hs_code, weight_kg, volume_m3, is_dangerous, declared_value | BIGSERIAL, BIGINT (FK), VARCHAR(255), VARCHAR(20), NUMERIC(12,2), NUMERIC(10,2), BOOLEAN, NUMERIC(15,2) | Вантажна документація з HS-кодом для митного оформлення та ознакою небезпечності. |
| **incidents** | id, type, severity, container_id, shipment_id, insurance_policy_number, reported_at, resolution_status | BIGSERIAL, VARCHAR(50), VARCHAR(20), BIGINT (FK), BIGINT (FK), VARCHAR(100), TIMESTAMP, VARCHAR(50) | Нештатні ситуації під час перевезення з ступенем критичності та статусом розслідування. |
| **metrics** | id, container_id, rental_id, type, value, unit, meta, recorded_at, metrics_partition_key | BIGINT, BIGINT (FK), BIGINT (FK), VARCHAR(64), NUMERIC(12,4), VARCHAR(100), JSON, TIMESTAMPTZ, BIGINT | Партиційована таблиця телеметричних даних IoT-датчиків з метаданими у форматі JSON. |
| **iot_audit_chain** | id, container_id, rental_id, user_id, event_type, sequence, prev_hash, row_hash | BIGSERIAL, BIGINT (FK), BIGINT (FK), BIGINT (FK), VARCHAR(64), BIGINT, VARCHAR(64), VARCHAR(64) | Криптографічно захищений ланцюжок подій IoT. Хешування унеможливлює підробку даних. |
| **container_simulation_snapshots** | id, container_id, rental_id, sensor_state, actuators, last_tick_at | BIGSERIAL, BIGINT (FK), BIGINT (FK), JSON, JSON, TIMESTAMPTZ | Поточний стан симуляції датчиків. Один запис на контейнер (UNIQUE container_id). |
| **monitor_chart_layouts** | id, user_id, rental_id, name, is_default, config | BIGSERIAL, BIGINT (FK), BIGINT (FK), VARCHAR(120), BOOLEAN, JSON | Індивідуальні налаштування дашборду IoT-моніторингу користувача у форматі JSON. |
| **transactions** | id, rental_id, amount, currency, status, payment_method, external_provider_id, refund_reason | BIGSERIAL, BIGINT (FK), NUMERIC(15,2), VARCHAR(10), VARCHAR(50), VARCHAR(50), VARCHAR(100), TEXT | Платіжні операції за орендою з підтримкою повернень та зовнішніх провайдерів. |
| **notifications** | id, user_id, title, message, type, action_url, is_read | BIGSERIAL, BIGINT (FK), VARCHAR(255), TEXT, VARCHAR(50), VARCHAR(500), BOOLEAN | Системні сповіщення для користувачів з посиланням на дію та ознакою прочитання. |
| **user_messages** | id, recipient_user_id, sender_user_id, subject, body, read_at | BIGSERIAL, BIGINT (FK), BIGINT (FK), VARCHAR(255), TEXT, TIMESTAMP | Особиста переписка між користувачами платформи. |
| **inquiries** | id, name, email, phone_number, subject, message, source, handling_status, submitted_by_user_id | BIGSERIAL, VARCHAR(255), VARCHAR(255), VARCHAR(20), VARCHAR(255), TEXT, VARCHAR(20), VARCHAR(40), BIGINT (FK) | Звернення потенційних клієнтів з джерелом надходження та статусом обробки. |
| **user_telegram_links** | id, user_id, telegram_chat_id, telegram_user_id, status, linked_at | BIGSERIAL, BIGINT (FK), BIGINT, BIGINT, VARCHAR(32), TIMESTAMP | Прив'язка облікового запису до Telegram-бота. Унікальний telegram_chat_id. |
| **activity_logs** | id, user_id, action, model_name, model_id, old_values, new_values, ip_address, device_type | BIGSERIAL, BIGINT (FK), VARCHAR(255), VARCHAR(50), BIGINT, JSON, JSON, VARCHAR(45), VARCHAR(40) | Журнал змін сутностей системи з геолокаційними даними та інформацією про пристрій. |
| **request_logs** | id, user_id, path, method, ip_address, country_code, browser, device_type, created_at | BIGSERIAL, BIGINT (FK), VARCHAR(500), VARCHAR(10), VARCHAR(45), VARCHAR(10), VARCHAR(80), VARCHAR(40), TIMESTAMP | Журнал HTTP-запитів з геолокацією та даними про браузер і платформу. |

# ДОДАТОК Г

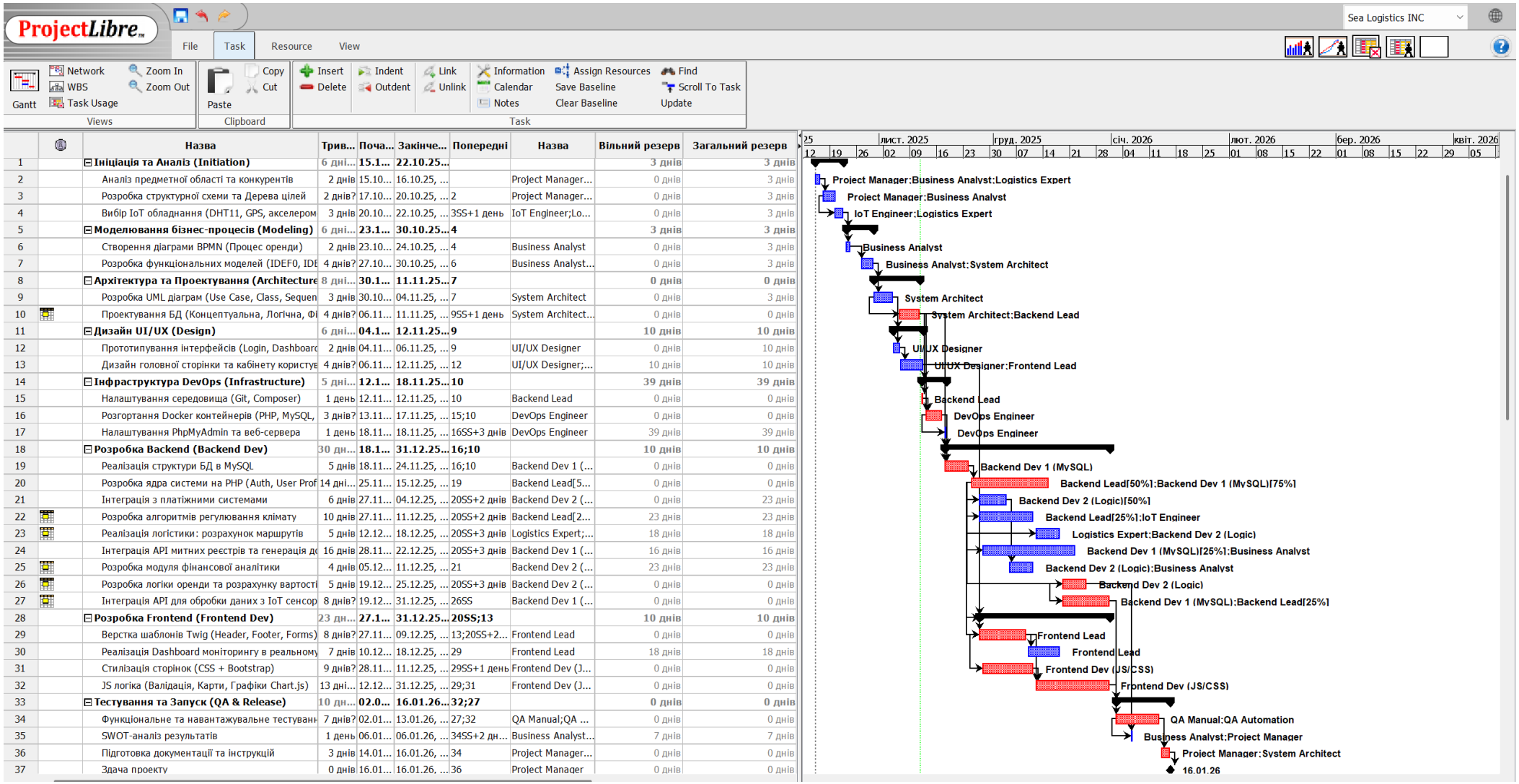



Рисунок Г.1 – Рекомендації щодо впровадження

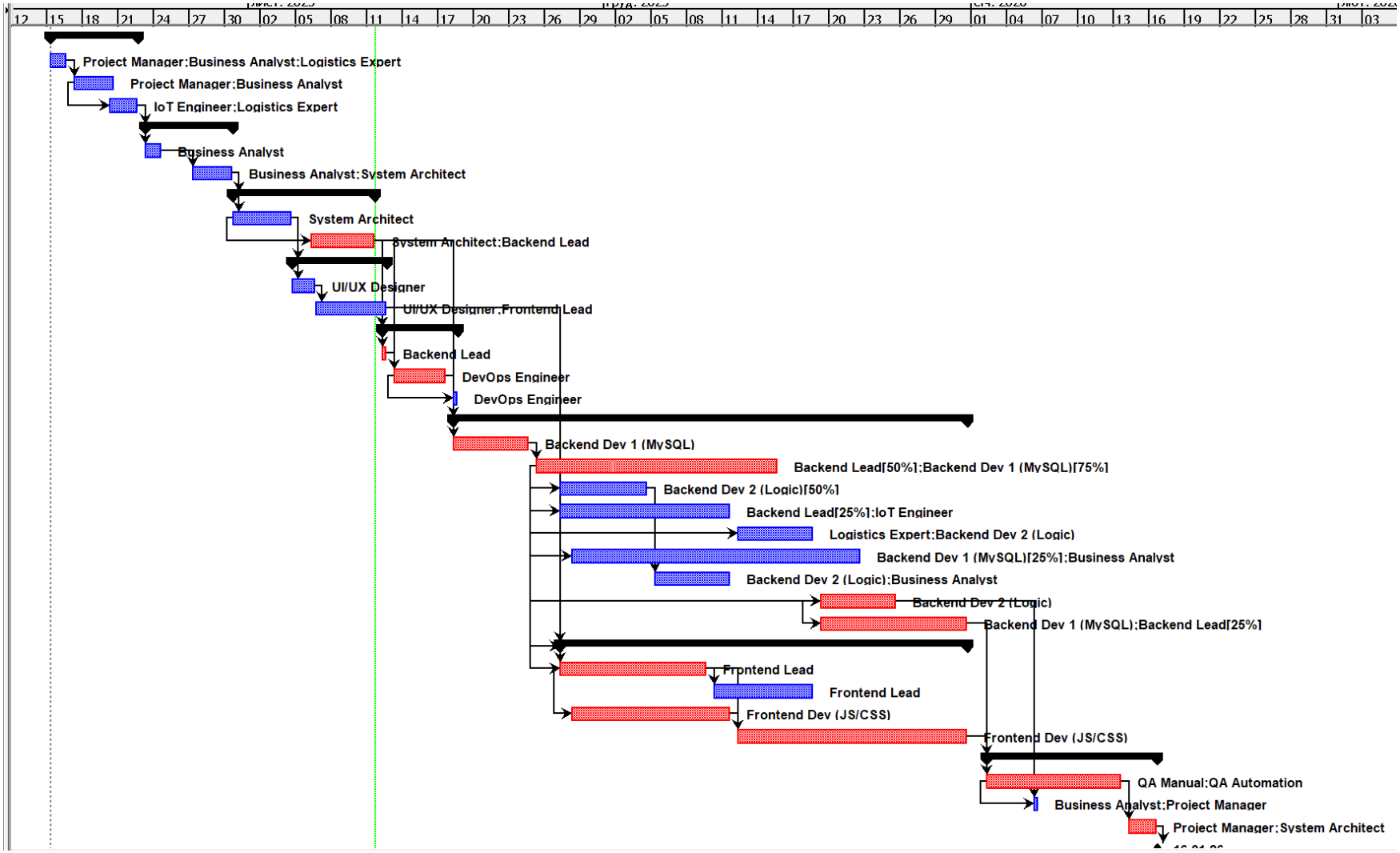
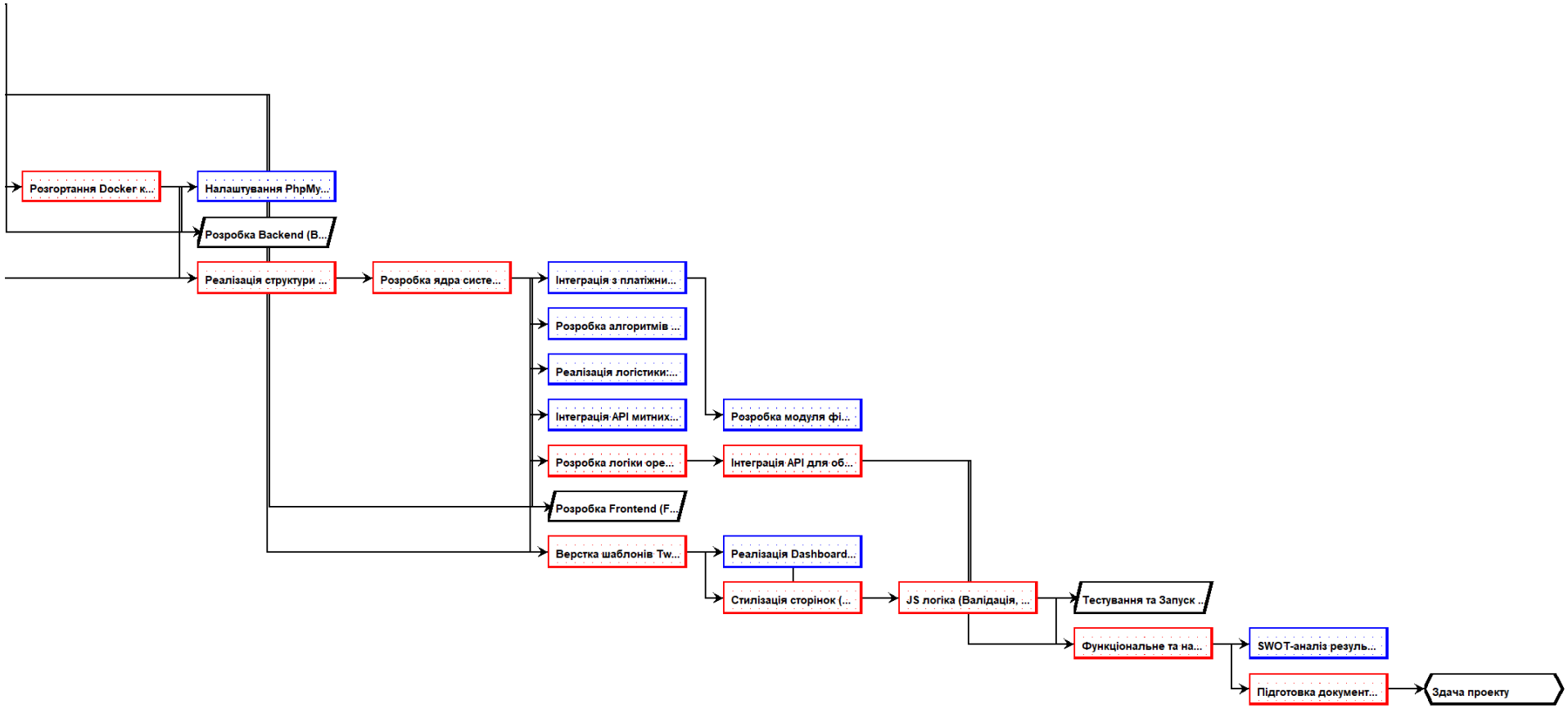


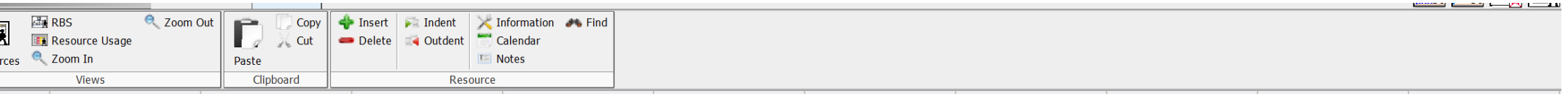

Рисунок Г.2 – Календарний план

Рисунок Г.3 – Мережева діаграма початку




Т

Рисунок Г.4 – Мережева діаграма технічної реалізації та реалізації всього проєкту



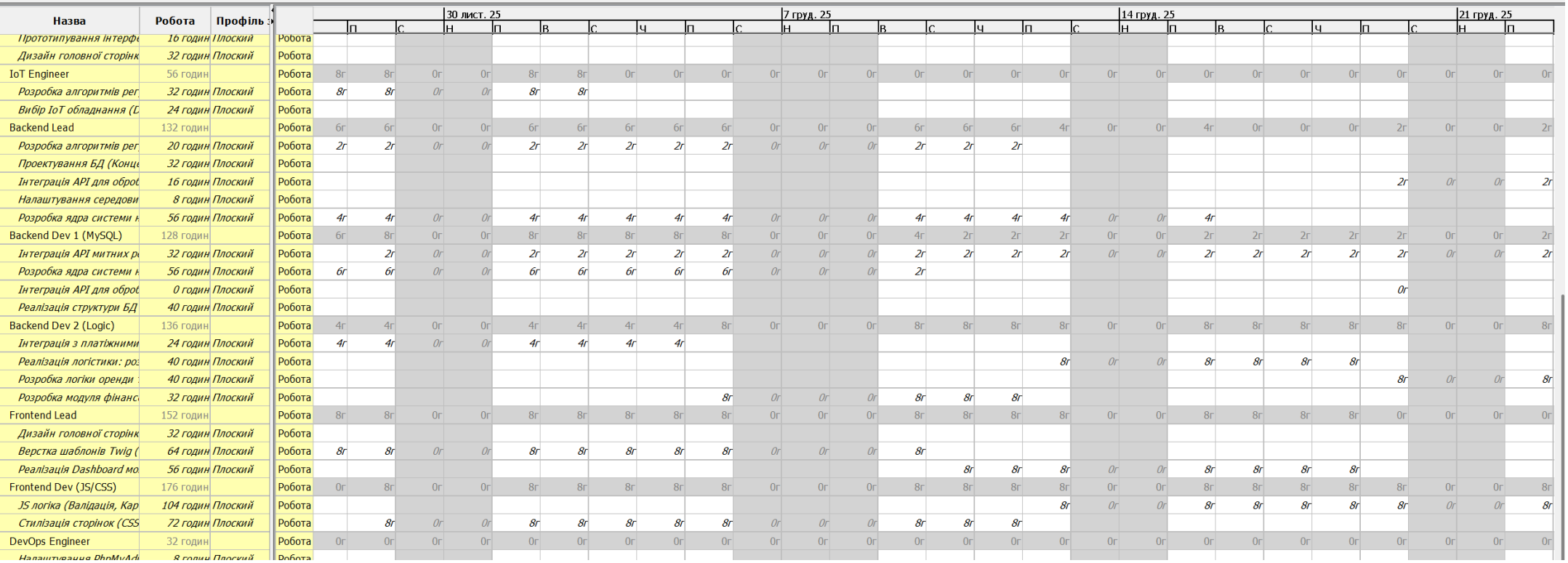

| | Назва | RBS | Тип | E-mail адреса | Од.вим. матеріалів | Ініціалы | Групувати | Максимальне використ... | Стандартна ставка | Ставка понаднормових |
|---|---|---|---|---|---|---|---|---|---|---|
| | Project Manager | MGT.01 | Робота | pm@logistics.co | | PM | Management | 100% | 10,00 USD/година | 15,00 USD/година |
| | Business Analyst | ANL.01 | Робота | ba@logistics.co | | BA | Analysis | 100% | 17,00 USD/година | 25,00 USD/година |
| | Logistics Expert | EXP.01 | Робота | expert@logistics.co | | LOG | Consulting | 100% | 8,00 USD/година | 20,00 USD/година |
| | System Architect | ENG.01 | Робота | architect@logistics.co | | ARC | Engineering | 100% | 10,00 USD/година | 15,00 USD/година |
| | UI/UX Designer | DES.01 | Робота | designer@logistics.co | | DES | Design | 100% | 5,00 USD/година | 7,00 USD/година |
| | IoT Engineer | HRD.01 | Робота | iot@logistics.co | | IOT | Hardware | 100% | 9,00 USD/година | 10,00 USD/година |
| | Backend Lead | DEV.01 | Робота | backend.lead@logistics.co | | BL | Dev | 100% | 15,00 USD/година | 20,00 USD/година |
| | Backend Dev 1 (MySQL) | DEV.02 | Робота | dev1@logistics.co | | BD1 | Dev | 100% | 12,00 USD/година | 15,00 USD/година |
| | Backend Dev 2 (Logic) | DEV.03 | Робота | dev2@logistics.co | | BD2 | Dev | 100% | 12,00 USD/година | 15,00 USD/година |
| | Frontend Lead | DEV.04 | Робота | frontend.lead@logistics.co | | FL | Dev | 100% | 15,00 USD/година | 21,00 USD/година |
| | Frontend Dev (JS/CSS) | DEV.05 | Робота | frontend.dev@logistics.co | | FD | Dev | 100% | 10,00 USD/година | 14,00 USD/година |
| | DevOps Engineer | OPS.01 | Робота | devops@logistics.co | | OPS | Infrastructure | 100% | 14,00 USD/година | 25,00 USD/година |
| | QA Manual | QA.01 | Робота | qa.manual@logistics.co | | QAM | Quality | 100% | 8,00 USD/година | 10,00 USD/година |
| | QA Automation | QA.02 | Робота | qa.auto@logistics.co | | QAA | Quality | 100% | 9,00 USD/година | 10,00 USD/година |

Рисунок Д.1 – Таблиця ресурсів та їх реалізація в проєкті

## Додаток Е

| | Назва | Тривалість | Початок | Закінчення | Попередні | Назва | Вільний резерв | Загальний резерв | % завершення | Вартість |
|---|---|---|---|---|---|---|---|---|---|---|
| | **Ініціація та Аналіз (Initiation)** | **6 днів?** | **15.10.25, 08:00** | **22.10.25, 17:00** | | | **3 днів** | **3 днів** | **0%** | **800,00 USD** |
| | Аналіз предметної області та конкурентів | 2 днів | 15.10.25, 08:00 | 16.10.25, 17:00 | | Project Manager;Business Analyst;Logistics Ex... | 0 днів | 3 днів | 0% | 288,00 USD |
| | Розробка структурної схеми та Дерева цілей | 2 днів? | 17.10.25, 08:00 | 20.10.25, 17:00 | 2 | Project Manager;Business Analyst | 0 днів | 3 днів | 0% | 296,00 USD |
| | Вибір IoT обладнання (DHT11, GPS, акселером | 3 днів | 20.10.25, 08:00 | 22.10.25, 17:00 | 3SS+1 день | IoT Engineer;Logistics Expert | 0 днів | 3 днів | 0% | 216,00 USD |
| | **Моделювання бізнес-процесів (Modeling)** | **6 днів?** | **23.10.25, 08:00** | **30.10.25, 16:59** | **4** | | **3 днів** | **3 днів** | **0%** | **795,99 USD** |
| | Створення діаграми BPMN (Процес оренди) | 2 днів | 23.10.25, 08:00 | 24.10.25, 17:00 | 4 | Business Analyst | 0 днів | 3 днів | 0% | 272,00 USD |
| | Розробка функціональних моделей (IDEF0, IDE | 4 днів? | 27.10.25, 08:00 | 30.10.25, 16:59 | 6 | Business Analyst;System Architect | 0 днів | 3 днів | 0% | 523,99 USD |
| | **Архітектура та Проектування (Architecture** | **8 днів?** | **30.10.25, 16:59** | **11.11.25, 17:00** | **7** | | **0 днів** | **0 днів** | **0%** | **1040,00 USD** |
| | Розробка UML діаграм (Use Case, Class, Sequen | 3 днів | 30.10.25, 16:59 | 04.11.25, 16:59 | 7 | System Architect | 0 днів | 3 днів | 0% | 240,00 USD |
| | Проектування БД (Концептуальна, Логічна, Фі | 4 днів? | 06.11.25, 08:00 | 11.11.25, 17:00 | 9SS+1 день | System Architect;Backend Lead | 0 днів | 0 днів | 0% | 800,00 USD |
| | **Дизайн UI/UX (Design)** | **6 днів?** | **04.11.25, 16:59** | **12.11.25, 16:59** | **9** | | **10 днів** | **10 днів** | **0%** | **720,00 USD** |
| | Прототипування інтерфейсів (Login, Dashboard | 2 днів | 04.11.25, 16:59 | 06.11.25, 16:59 | 9 | UI/UX Designer | 0 днів | 10 днів | 0% | 80,00 USD |
| | Дизайн головної сторінки та кабінету користув | 4 днів? | 06.11.25, 16:59 | 12.11.25, 16:59 | 12 | UI/UX Designer;Frontend Lead | 10 днів | 10 днів | 0% | 640,00 USD |
| | **Інфраструктура DevOps (Infrastructure)** | **5 днів?** | **12.11.25, 08:00** | **18.11.25, 17:00** | **10** | | **39 днів** | **39 днів** | **0%** | **568,00 USD** |
| | Налаштування середовища (Git, Composer) | 1 день | 12.11.25, 08:00 | 12.11.25, 17:00 | 10 | Backend Lead | 0 днів | 0 днів | 0% | 120,00 USD |
| | Розгортання Docker контейнерів (PHP, MySQL, | 3 днів? | 13.11.25, 08:00 | 17.11.25, 17:00 | 15;10 | DevOps Engineer | 0 днів | 0 днів | 0% | 336,00 USD |
| | Налаштування PhpMyAdmin та веб-сервера | 1 день | 18.11.25, 08:00 | 18.11.25, 17:00 | 16SS+3 днів | DevOps Engineer | 39 днів | 39 днів | 0% | 112,00 USD |
| | **Розробка Backend (Backend Dev)** | **30 днів?** | **18.11.25, 08:00** | **31.12.25, 17:00** | **16;10** | | **10 днів** | **10 днів** | **0%** | **5836,00 USD** |
| | Реалізація структури БД в MySQL | 5 днів | 18.11.25, 08:00 | 24.11.25, 17:00 | 16;10 | Backend Dev 1 (MySQL) | 0 днів | 0 днів | 0% | 480,00 USD |
| | Розробка ядра системи на PHP (Auth, User Prof | 14 днів? | 25.11.25, 08:00 | 15.12.25, 17:00 | 19 | Backend Lead[50%];Backend Dev 1 (MySQL)[... | 0 днів | 0 днів | 0% | 1512,00 USD |
| | Інтеграція з платіжними системами | 6 днів | 27.11.25, 08:00 | 04.12.25, 17:00 | 20SS+2 днів | Backend Dev 2 (Logic)[50%] | 0 днів | 23 днів | 0% | 288,00 USD |
| | Розробка алгоритмів регулювання клімату | 10 днів | 27.11.25, 08:00 | 11.12.25, 17:00 | 20SS+2 днів | Backend Lead[25%];IoT Engineer | 23 днів | 23 днів | 0% | 588,00 USD |
| | Реалізація логістики: розрахунок маршрутів | 5 днів | 12.12.25, 08:00 | 18.12.25, 17:00 | 20SS+3 днів | Logistics Expert;Backend Dev 2 (Logic) | 18 днів | 18 днів | 0% | 800,00 USD |
| | Інтеграція API митних реєстрів та генерація до | 16 днів | 28.11.25, 08:00 | 22.12.25, 17:00 | 20SS+3 днів | Backend Dev 1 (MySQL)[25%];Business Analyst | 16 днів | 16 днів | 0% | 928,00 USD |
| | Розробка модуля фінансової аналітики | 4 днів | 05.12.25, 08:00 | 11.12.25, 17:00 | 21 | Backend Dev 2 (Logic);Business Analyst | 23 днів | 23 днів | 0% | 520,00 USD |
| | Розробка логіки оренди та розрахунку вартості | 5 днів | 19.12.25, 08:00 | 25.12.25, 17:00 | 20SS+3 днів | Backend Dev 2 (Logic) | 0 днів | 0 днів | 0% | 480,00 USD |
| | Інтеграція API для обробки даних з IoT сенсор | 8 днів? | 19.12.25, 08:00 | 31.12.25, 17:00 | 26SS | Backend Dev 1 (MySQL);Backend Lead[25%] | 0 днів | 0 днів | 0% | 240,00 USD |
| | **Розробка Frontend (Frontend Dev)** | **23 днів?** | **27.11.25, 08:00** | **31.12.25, 17:00** | **20SS;13** | | **10 днів** | **10 днів** | **0%** | **3560,00 USD** |
| | Верстка шаблонів Twig (Header, Footer, Forms) | 8 днів? | 27.11.25, 08:00 | 09.12.25, 17:00 | 13;20SS+2... | Frontend Lead | 0 днів | 0 днів | 0% | 960,00 USD |
| | Реалізація Dashboard моніторингу в реальному | 7 днів | 10.12.25, 08:00 | 18.12.25, 17:00 | 29 | Frontend Lead | 18 днів | 18 днів | 0% | 840,00 USD |
| | Стилізація сторінок (CSS + Bootstrap) | 9 днів? | 28.11.25, 08:00 | 11.12.25, 17:00 | 29SS+1 день | Frontend Dev (JS/CSS) | 0 днів | 0 днів | 0% | 720,00 USD |
| | JS логіка (Валідація, Карти, Графіки Chart.js) | 13 днів? | 12.12.25, 08:00 | 31.12.25, 17:00 | 29;31 | Frontend Dev (JS/CSS) | 0 днів | 0 днів | 0% | 1040,00 USD |
| | **Тестування та Запуск (QA & Release)** | **10 днів?** | **02.01.26, 08:00** | **16.01.26, 17:00** | **32;27** | | **0 днів** | **0 днів** | **0%** | **1648,00 USD** |
| | Функціональне та навантажувальне тестуванн | 7 днів? | 02.01.26, 08:00 | 13.01.26, 17:00 | 27;32 | QA Manual;QA Automation | 0 днів | 0 днів | 0% | 952,00 USD |
| | SWOT-аналіз результатів | 1 день | 06.01.26, 08:00 | 06.01.26, 17:00 | 34SS+2 дн... | Business Analyst;Project Manager | 7 днів | 7 днів | 0% | 216,00 USD |
| | Підготовка документації та інструкцій | 3 днів | 14.01.26, 08:00 | 16.01.26, 17:00 | 34 | Project Manager;System Architect | 0 днів | 0 днів | 0% | 480,00 USD |
| | Здача проекту | 0 днів | 16.01.26, 17:00 | 16.01.26, 17:00 | 36 | Project Manager | 0 днів | 0 днів | 0% | 0,00 USD |

Рисунок Е.1 – Таблиця реалізації і залежності проєкту



| Назва | Робота | Профіль | | 30 лист. 25 | | | | | | | | | | 7 груд. 25 | | | | | | | | | | 14 груд. 25 | | | | | | | | | | 21 груд. 25 | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | П | С | Н | П | В | С | Ч | П | С | Н | П | В | С | Ч | П | С | Н | П | В | С | Ч | П | С | Н | П | | | | | | | | |
| *Прототипування інтерфе* | *16 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| *Дизайн головної сторінк* | *32 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| IoT Engineer | 56 годин | | Робота | 8г | 8г | 0г | 0г | 8г | 8г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г |
| *Розробка алгоритмів рег* | *32 годин* | *Плоский* | Робота | *8г* | *8г* | *0г* | *0г* | *8г* | *8г* | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| *Вибір IoT обладнання (D* | *24 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Backend Lead | 132 годин | | Робота | 6г | 6г | 0г | 0г | 6г | 6г | 6г | 6г | 6г | 0г | 0г | 0г | 6г | 6г | 6г | 4г | 0г | 0г | 4г | 0г | 0г | 0г | 2г | 0г | 0г | 2г | | | | | | |
| *Розробка алгоритмів рег* | *20 годин* | *Плоский* | Робота | *2г* | *2г* | *0г* | *0г* | *2г* | *2г* | *2г* | *2г* | *2г* | *0г* | *0г* | *0г* | *2г* | *2г* | *2г* | | | | | | | | | | | | | | | | | | |
| *Проектування БД (Конце* | *32 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| *Інтеграція API для обробо* | *16 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | 2г | 0г | 0г | 2г | | | | | | |
| *Налаштування середови* | *8 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| *Розробка ядра системи н* | *56 годин* | *Плоский* | Робота | *4г* | *4г* | *0г* | *0г* | *4г* | *4г* | *4г* | *4г* | *4г* | *0г* | *0г* | *0г* | *4г* | *4г* | *4г* | *4г* | *0г* | *0г* | *4г* | | | | | | | | | | | | | | |
| Backend Dev 1 (MySQL) | 128 годин | | Робота | 6г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 0г | 4г | 2г | 2г | 2г | 0г | 0г | 2г | 2г | 2г | 2г | 2г | 0г | 0г | 2г | | | | | | |
| *Інтеграція API митних р* | *32 годин* | *Плоский* | Робота | | *2г* | *0г* | *0г* | *2г* | *2г* | *2г* | *2г* | *2г* | *0г* | *0г* | *0г* | *2г* | *2г* | *2г* | *2г* | *0г* | *0г* | *2г* | *2г* | *2г* | *2г* | *2г* | *0г* | *0г* | *2г* | | | | | | |
| *Розробка ядра системи н* | *56 годин* | *Плоский* | Робота | *6г* | *6г* | *0г* | *0г* | *6г* | *6г* | *6г* | *6г* | *6г* | *0г* | *0г* | *0г* | *2г* | | | | | | | | | | | | | | | | | | | | |
| *Інтеграція API для обробо* | *0 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | 0г | | | | | | | | | |
| *Реалізація структури БД* | *40 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| Backend Dev 2 (Logic) | 136 годин | | Робота | 4г | 4г | 0г | 0г | 4г | 4г | 4г | 4г | 8г | 0г | 0г | 0г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | | | | | | |
| *Інтеграція з платіжними* | *24 годин* | *Плоский* | Робота | *4г* | *4г* | *0г* | *0г* | *4г* | *4г* | *4г* | *4г* | | | | | | | | | | | | | | | | | | | | | | | | |
| *Реалізація логістики: роз* | *40 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | 8г | 0г | 0г | 8г | 8г | 8г | 8г | | | | | | | | | | |
| *Розробка логіки оренди* | *40 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | 8г | 0г | 0г | 8г | | | | | | |
| *Розробка модуля фінанс* | *32 годин* | *Плоский* | Робота | | | | | | | | 8г | 0г | 0г | 0г | 8г | 8г | 8г | | | | | | | | | | | | | | | | | | |
| Frontend Lead | 152 годин | | Робота | 8г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 0г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 0г | 0г | 0г | 0г | | | | | | |
| *Дизайн головної сторінк* | *32 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| *Верстка шаблонів Twig (* | *64 годин* | *Плоский* | Робота | *8г* | *8г* | *0г* | *0г* | *8г* | *8г* | *8г* | *8г* | *8г* | *0г* | *0г* | *0г* | *8г* | | | | | | | | | | | | | | | | | | | | |
| *Реалізація Dashboard мо* | *56 годин* | *Плоский* | Робота | | | | | | | | | | | | | | 8г | 8г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | | | | | | | | | | |
| Frontend Dev (JS/CSS) | 176 годин | | Робота | 0г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 0г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | | | | | | |
| *JS логіка (Валідація, Кар* | *104 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 8г | | | | | | |
| *Стилізація сторінок (CSS* | *72 годин* | *Плоский* | Робота | | 8г | 0г | 0г | 8г | 8г | 8г | 8г | 8г | 0г | 0г | 0г | 8г | 8г | 8г | | | | | | | | | | | | | | | | | | |
| DevOps Engineer | 32 годин | | Робота | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | 0г | | | | | | |
| *Налаштування PhpMyAdr* | *8 годин* | *Плоский* | Робота | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

Рисунок Е.2 – Таблиця використання ресурсів

**ДОДАТОК Ж**

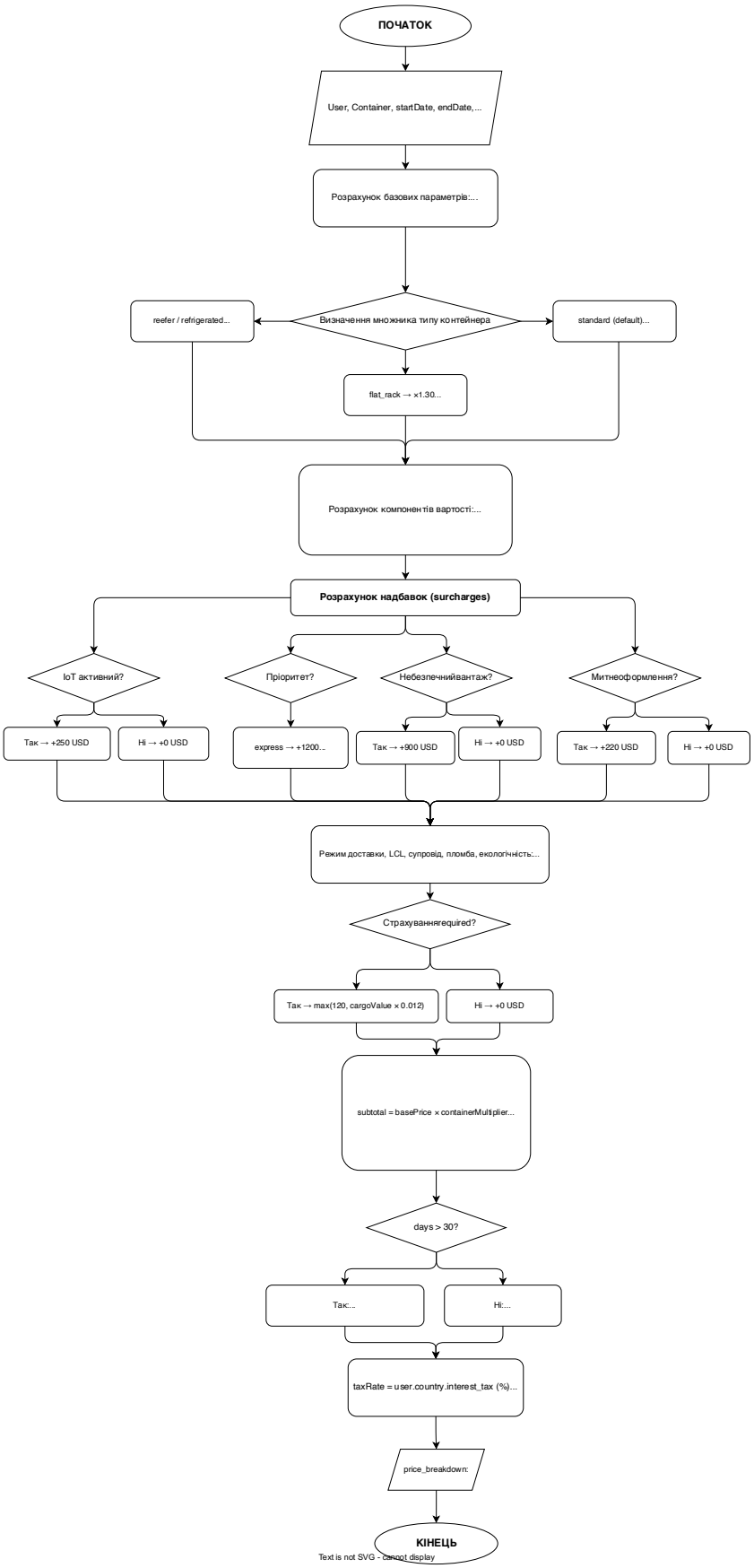



Рис. Ж.1 — Блок-схема алгоритму розрахунку вартості оренди контейнера

**ДОДАТОК З**

# ІНСТРУКЦІЯ КОРИСТУВАЧА

Інструкція користувача призначена для ознайомлення з можливостями веб-платформи Logistics SWay та описує порядок роботи користувача із системою оренди контейнерів, перегляду фінансової інформації та моніторингу стану вантажу за допомогою IoT-датчиків.

Для початку роботи необхідно відкрити веб-браузер та перейти за адресою веб-платформи. Після відкриття сайту користувач потрапляє на головну сторінку системи, де розміщено коротку інформацію про сервіс, основні логістичні послуги, можливості IoT-моніторингу та кнопки для переходу до авторизації або перегляду послуг. Без входу в систему користувач може переглянути сторінки Home, Services та Contact, ознайомитися з напрямами перевезень, типами датчиків і залишити повідомлення через форму зворотного зв’язку.

Для повноцінної роботи з платформою користувачу необхідно створити обліковий запис або увійти до вже наявного акаунта. Для реєстрації потрібно натиснути кнопку Account Login та перейти до форми створення нового користувача. У формі необхідно вказати ім’я, прізвище, електронну пошту, номер телефону, країну та пароль. Після заповнення обов’язкових полів користувач натискає кнопку реєстрації, після чого система перевіряє введені дані, створює обліковий запис і автоматично відкриває особистий кабінет.

Якщо користувач уже має акаунт, для входу в систему потрібно відкрити форму авторизації, ввести електронну пошту та пароль, після чого натиснути кнопку входу. Система перевіряє правильність введених даних і, якщо вони є коректними, перенаправляє користувача до особистого кабінету. У разі введення неправильного пароля система повідомляє про помилку авторизації. Якщо користувач забув пароль, він може скористатися посиланням Forgot your password?, після чого на електронну пошту буде надіслано інструкцію для відновлення доступу.

Після успішної авторизації користувач переходить до головної сторінки особистого кабінету — Operations Dashboard. На цій сторінці відображається загальний стан облікового запису, кількість активних і завершених оренд, непрочитані сповіщення, інформація про профіль, фінансові показники та поточні логістичні

операції. За допомогою лівої панелі навігації користувач може перейти до основних розділів системи: Dashboard, Finance, Rental, Request Rental та Support.

У розділі Dashboard користувач переглядає загальну інформацію про свою діяльність у системі. Тут можна швидко побачити, чи є активні замовлення, чи очікують заявки підтвердження, чи є нові повідомлення від системи, а також чи потрібно доповнити дані профілю. Заповнення профілю є бажаним, оскільки повні контактні дані допомагають швидше обробляти заявки на оренду контейнерів.

Для створення нової заявки на оренду контейнера користувач переходить до розділу Request Rental. На цій сторінці необхідно послідовно заповнити дані замовлення. Спочатку користувач обирає контейнер зі списку доступних варіантів. Для кожного контейнера система показує його серійний номер, тип, габарити, максимальну вагу, поточний порт розташування та наявність IoT-моніторингу. Після цього користувач зазначає маршрут перевезення: порт відправлення, порт призначення, дату початку та дату завершення оренди.

Після вибору маршруту система автоматично враховує відстань між портами, орієнтовний час транспортування та доступність контейнера на обраний період. Якщо між вибраними портами немає відкритого маршруту або контейнер недоступний на зазначені дати, система повідомляє користувача про неможливість створення заявки та пропонує змінити параметри замовлення.

Наступним кроком користувач заповнює інформацію про вантаж. У цьому блоці потрібно вказати тип вантажу, його масу, об’єм, кількість місць і задекларовану вартість. Якщо вантаж є небезпечним, користувач додатково зазначає відповідні параметри, зокрема UN-номер і клас небезпечності. Також користувач може вибрати умови доставки, наприклад port-to-port або door-to-door, тип завантаження FCL або LCL, необхідність страхування, митного оформлення, пломбування або супроводу.

Після заповнення всіх основних полів користувач натискає кнопку Calculate Price. Система виконує попередній розрахунок вартості оренди контейнера та показує детальну розбивку ціни. У розрахунку враховується базова вартість, тип контейнера, тривалість оренди, відстань маршруту, характеристики вантажу, наявність IoT-моніторингу, пріоритет доставки, страхування, митне оформлення та інші додаткові

послуги. Завдяки цьому користувач бачить не лише загальну суму, а й розуміє, з яких складових вона сформована.

Якщо користувача влаштовують умови та розрахована вартість, він підтверджує заявку. Після цього система зберігає замовлення та присвоює йому статус pending_approval. Це означає, що заявка передана на перевірку працівникам платформи. Користувач може переглядати створену заявку в розділі Rental та відстежувати зміну її статусу.

У розділі Rental відображається список усіх заявок користувача. Для кожного замовлення доступна основна інформація: обраний контейнер, маршрут, дати оренди, статус заявки, статус оплати та розрахована вартість. Якщо заявка ще очікує підтвердження, користувач бачить відповідний статус. Після перевірки працівником платформи статус може бути змінено на підтверджений, відхилений або запланований. У разі зміни статусу система надсилає користувачу сповіщення в особистому кабінеті.

Після підтвердження заявки користувачу стає доступна можливість оплати. Для цього необхідно перейти до відповідного замовлення або до розділу Finance. У фінансовому розділі відображаються всі транзакції користувача, суми до сплати, статуси платежів і деталі фінансових операцій. Користувач обирає доступний спосіб оплати та підтверджує транзакцію. Після успішної оплати система оновлює статус платежу та пов’язує його з відповідною орендою.

Коли оренда переходить в активний стан, користувач отримує доступ до IoT-дашборду контейнера. Відкрити його можна через розділ Rental у деталях активного замовлення. На сторінці моніторингу відображаються актуальні показники підключених датчиків: температура, вологість, стан дверей, освітленість, удари, нахил, рівень $CO_2$ та GPS-координати контейнера. Дані оновлюються автоматично, що дає змогу контролювати стан вантажу під час перевезення.

Для зручності аналізу користувач може переглядати графіки зміни показників за вибраний період. Наприклад, можна перевірити, як змінювалася температура всередині контейнера протягом транспортування, чи були різкі удари, чи відкривалися двері, а також чи не відхилявся контейнер від маршруту. Якщо один із показників виходить за допустимі межі, система формує сповіщення для користувача.

Система підтримує сповіщення в особистому кабінеті та через Telegram-бота. Для підключення Telegram користувач переходить до налаштувань профілю, створює одноразовий код прив'язки та надсилає його боту платформи. Після успішного підтвердження Telegram-акаунт буде пов'язаний з обліковим записом користувача. Надалі система зможе надсилати повідомлення про зміну статусу оренди, підтвердження заявки, оплату, а також про критичні показники IoT-датчиків.

У разі виникнення питань користувач може перейти до розділу Support або скористатися сторінкою Contact. У формі звернення потрібно вказати тему повідомлення та описати проблему. Після надсилання звернення система зберігає його та передає працівникам платформи для подальшої обробки. Такий механізм дозволяє користувачу швидко звернутися по допомогу без необхідності використовувати сторонні канали зв'язку.

Під час роботи з платформою користувач повинен уважно перевіряти введені дані перед надсиланням заявки. Особливо важливо правильно зазначати дати оренди, порти відправлення та призначення, параметри вантажу, потребу в страхуванні, митному оформленні та додаткових послугах. Від цих даних залежить коректність розрахунку вартості та можливість виконання перевезення.

Якщо дані IoT-дашборду не відображаються, користувачу потрібно перевірити статус оренди. Моніторинг доступний лише для активних або запланованих оренд із підключеною телеметрією. Якщо заявка ще не підтверджена або оренда не розпочалася, сторінка моніторингу може бути недоступною. Якщо замовлення довго очікує підтвердження, користувачу варто перевірити розділ сповіщень, оскільки система могла запросити уточнення або додаткові дані.

Для завершення роботи користувач повинен вийти із системи за допомогою кнопки Logout. Після цього система завершує сеанс користувача та повертає його на публічну частину веб-платформи або сторінку авторизації. Завершення сеансу є важливим для безпеки, особливо якщо користувач працює з платформою на чужому комп'ютері або в публічній мережі.

Таким чином, веб-платформа Logistics SWay забезпечує користувачу повний цикл роботи з орендою контейнера: від реєстрації та створення заявки до оплати, перегляду статусу перевезення та контролю стану вантажу за допомогою IoT-датчиків. Система спрощує взаємодію з логістичною компанією, робить процес оренди прозорішим і дозволяє користувачу самостійно контролювати основні етапи перевезення.

**https://github.com/DmitryBizCode/VueLarContainer.git - весь код програмного продукту.**

**Код 1 – Основні маршрути веб-застосунку *(routes/web.php)***

**Призначення:** Описує публічні сторінки, особистий кабінет і сторінки, доступні тільки після авторизації.

```
use App\Http\Controllers\DashboardController;
use App\Http\Controllers\FinanceMonitoringController;
use App\Http\Controllers\RentalController;
use App\Http\Controllers\RentalsCenterController;
use App\Http\Controllers\TelegramLinkController;
use App\Http\Controllers\UserNotificationController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;

Route::get('/', fn () => Inertia::render('Marketing/HomePage/Index'))
    ->name('home');

Route::get('/services', fn () => Inertia::render('Marketing/ServicesPage/Index'))
    ->name('services');

Route::get('/contact', fn () => Inertia::render('Marketing/ContactPage/Index'))
    ->name('contact');

Route::post('/contact', [InquiryController::class, 'store'])
    ->middleware('throttle:10,1')
    ->name('contact.submit');

Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware(['auth', 'verified'])
    ->name('dashboard');

Route::get('/finance-monitoring', [FinanceMonitoringController::class, 'index'])
    ->middleware(['auth', 'verified'])
    ->name('finance.monitoring');

Route::get('/rentals-center', [RentalsCenterController::class, 'index'])
    ->middleware(['auth', 'verified'])
    ->name('rentals.center');
```

**Код 2 – Маршрути профілю, сповіщень та оренди *(routes/web.php)***

**Призначення:** Групує дії користувача: профіль, Telegram, сповіщення, створення заявки та IoT-моніторинг.

```
Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])
        ->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])
        ->name('profile.destroy');

    Route::middleware('verified')->group(function () {
        Route::patch('/profile/notification-channels', [ProfileController::class, 'updateNotificationChannels'])
            ->name('profile.notification-channels.update');

        Route::post('/telegram/link-code', [TelegramLinkController::class, 'createLinkCode'])
            ->name('telegram.link-code');

        Route::delete('/telegram/links/{link}', [TelegramLinkController::class, 'destroyLink'])
            ->name('telegram.links.destroy');

        Route::get('/notifications', [UserNotificationController::class, 'index'])
            ->name('notifications.index');
        Route::get('/notifications/unread-count', [UserNotificationController::class, 'unreadCount'])
            ->name('notifications.unread-count');

        Route::get('/rentals/reachable-destinations', [RentalController::class, 'reachableDestinations'])
            ->name('rentals.reachable-destinations');
        Route::get('/rentals/request', [RentalController::class, 'create'])
            ->name('rentals.request.create');
        Route::post('/rentals/request/preview', [RentalController::class, 'preview'])
            ->name('rentals.request.preview');
        Route::post('/rentals/request', [RentalController::class, 'store'])
            ->name('rentals.request.store');
        Route::get('/rentals/{rental}/monitor', [RentalController::class, 'monitor'])
            ->name('rentals.monitor');
        Route::get('/rentals/{rental}/monitor-charts-data', [MonitorChartsController::class, 'index'])
            ->name('rentals.monitor.charts-data');
    });
});
```

**Код 3 – Маршрути адміністративної панелі *(routes/web.php)***

**Призначення:** Відокремлює адміністративні функції від клієнтських і захищає їх middleware admin.

```
Route::prefix('admin')
    ->middleware(['auth', 'verified', 'admin'])
    ->group(function () {
        Route::get('/', [AdminDashboardController::class, 'index'])
            ->name('admin.dashboard');

        Route::get('/inquiries', [AdminInquiryController::class, 'index'])
            ->name('admin.inquiries.index');
        Route::patch('/inquiries/{inquiry}', [AdminInquiryController::class, 'update'])
            ->name('admin.inquiries.update');

        Route::get('/approvals', [AdminRentalController::class, 'approvals'])
```

```
            ->name('admin.approvals');
        Route::get('/rentals',
[AdminRentalController::class, 'index'])
            ->name('admin.rentals.index');
        Route::get('/rentals/{rental}',
[AdminRentalController::class, 'show'])
            ->name('admin.rentals.show');
        Route::patch('/rentals/{rental}/sta
tus',
            [AdminRentalController::class,
'updateStatus'])
            ->name('admin.rentals.status');

        Route::get('/finance',
[AdminFinanceController::class, 'index'])
            ->name('admin.finance.index');
        Route::patch('/finance/rentals/{ren
tal}/payment-status',
            [AdminFinanceController::class,
'updateRentalPaymentStatus'])
            -
>name('admin.finance.rentals.payment-
status');

        Route::resource('containers',
AdminContainerController::class)
            ->names('admin.containers');
        Route::resource('ports',
AdminPortController::class)
            ->names('admin.ports');
        Route::resource('routes',
AdminRouteController::class)
            ->names('admin.routes')
            ->parameters(['routes' =>
'route']);
    });
```

**Код 4 – Маршрути авторизації *(routes/auth.php)***

**Призначення:** Окремо описує реєстрацію, вхід, відновлення пароля, підтвердження email і вихід.

```
Route::middleware('guest')->group(function
() {
    Route::get('register',
[RegisteredUserController::class,
'create'])
        ->name('register');
    Route::post('register',
[RegisteredUserController::class,
'store']);

    Route::get('login',
[AuthenticatedSessionController::class,
'create'])
        ->name('login');
    Route::post('login',
[AuthenticatedSessionController::class,
'store']);

    Route::get('forgot-password',
[PasswordResetLinkController::class,
'create'])
        ->name('password.request');
    Route::post('forgot-password',
[PasswordResetLinkController::class,
'store'])
        ->name('password.email');

    Route::get('reset-password/{token}',
[NewPasswordController::class, 'create'])
        ->name('password.reset');
    Route::post('reset-password',
[NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function
() {
    Route::get('verify-email',
EmailVerificationPromptController::class)
        ->name('verification.notice');
    Route::get('verify-email/{id}/{hash}',
VerifyEmailController::class)
        ->middleware(['signed',
'throttle:6,1'])
        ->name('verification.verify');
    Route::post('email/verification-
notification',
        [EmailVerificationNotificationContr
oller::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');
    Route::post('logout',
[AuthenticatedSessionController::class,
'destroy'])
        ->name('logout');
});
```

**Код 5 – Контролер входу в систему *(app/Http/Controllers/Auth/AuthenticatedSessionController.php)***

**Призначення:** Виконує авторизацію, регенерує сесію, записує подію входу/виходу в журнал активності.

```
class AuthenticatedSessionController
extends Controller
{
    public function __construct(
        private readonly ActivityLogService
$activityLog,
    ) {}

    public function create(): Response
    {
        return
Inertia::render('Auth/Login', [
            'canResetPassword' =>
Route::has('password.request'),
            'status' => session('status'),
        ]);
    }

    public function store(LoginRequest
$request): RedirectResponse
    {
        $request->authenticate();
        $request->session()->regenerate();

        $this->activityLog->logAuth(
            $request->user()->id,
            'logged_in',
            'User logged in',
            $request
        );

        return redirect()-
>intended(route('dashboard', absolute:
false));
    }

    public function destroy(Request
$request): RedirectResponse
    {
        $user = $request->user();

        if ($user) {
            $this->activityLog-
>logAuth($user->id, 'logged_out', 'User
logged out', $request);
        }

        Auth::guard('web')->logout();
        $request->session()->invalidate();
        $request->session()-
>regenerateToken();

        return redirect('/');
    }
}
```

**Код 6 – Валідація та перевірка логіна *(app/Http/Requests/Auth/LoginRequest.php)***

**Призначення:** Перевіряє email і пароль, блокує надмірну кількість спроб входу через RateLimiter.

```
class LoginRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
```

```
            'email' => ['required',
'string', 'email'],
            'password' => ['required',
'string'],
        ];
    }

    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this-
>only('email', 'password'), $this-
>boolean('remember'))) {
            RateLimiter::hit($this-
>throttleKey());

            throw
ValidationException::withMessages([
                'email' =>
trans('auth.failed'),
            ]);
        }

        RateLimiter::clear($this-
>throttleKey());
    }

    public function
ensureIsNotRateLimited(): void
    {
        if (!
RateLimiter::tooManyAttempts($this-
>throttleKey(), 5)) {
            return;
        }

        event(new Lockout($this));
        $seconds =
RateLimiter::availableIn($this-
>throttleKey());

        throw
ValidationException::withMessages([
            'email' =>
trans('auth.throttle', [
                'seconds' => $seconds,
                'minutes' => ceil($seconds
/ 60),
            ]),
        ]);
    }

    public function throttleKey(): string
    {
        return
Str::transliterate(Str::lower($this-
>string('email')).'|'.$this->ip());
    }
}
```

**Код 7 – Реєстрація нового користувача *(app/Http/Controllers/Auth/RegisteredUserController.php)***

**Призначення:** Створює клієнтський акаунт, хешує пароль і одразу авторизує користувача.

```
class RegisteredUserController extends
Controller
{
    public function create(): Response
    {
        return
Inertia::render('Auth/Register', [
            'countries' => Country::query()
                ->orderBy('name')
                ->get(['id', 'name',
'iso_code', 'phone_code']),
        ]);
    }

    public function store(Request
$request): RedirectResponse
    {
        $request->validate([
            'first_name' => ['required',
'string', 'max:50'],
            'last_name' => ['required',
'string', 'max:50'],
            'email' => ['required',
'string', 'lowercase', 'email', 'max:255',
'unique:'.User::class],
            'phone_number' => ['nullable',
'string', 'max:20'],
            'country_id' => ['nullable',
'integer', 'exists:countries,id'],
            'password' => ['required',
'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'first_name' => $request-
>first_name,
            'last_name' => $request-
>last_name,
            'email' => $request->email,
            'phone_number' => $request-
>phone_number,
            'country_id' => $request-
>country_id,
            'password' =>
Hash::make($request->password),
            'role' => 'client',
            'account_status' => 'active',
        ]);

        event(new Registered($user));
        Auth::login($user);

        return redirect(route('dashboard',
absolute: false));
    }
}
```

**Код 8 – Формування даних для сторінки заявки на оренду *(app/Http/Controllers/RentalController.php)***

**Призначення:** Готує маршрути, порти, графік відправлень і останні заявки користувача для Vue-сторінки.

```
public function create(Request $request):
Response
{
    $routesQuery = ShippingRoute::query()
        ->with(['originPort.country',
'destinationPort.country'])
        ->where('route_status', 'open')
        ->orderBy('distance');

    if (!
config('logistics.rental_request_show_all_o
pen_routes', false)) {
        $ids = $this->availabilityService-
>openRouteIdsWithAvailableContainerAtOrigin
();

        if ($ids !== []) {
            $routesQuery->whereIn('id',
$ids);
        } else {
            $routesQuery->whereRaw('1 =
0');
        }
    }

    $routes = $routesQuery->get();
    $ports = Port::query()-
>with('country')->orderBy('name')->get();
    $portSchedules = $this-
>availabilityService-
>portSchedulesWithAvailableContainers();
    $scheduleByPortId =
collect($portSchedules)->keyBy('port_id');

    $myRecentRequests = Rental::query()
        ->with(['container',
'originPort.country',
'destinationPort.country'])
        ->where('user_id', $request-
>user()->id)
        ->latest()
        ->limit(8)
        ->get();
```

```
    return
Inertia::render('Operations/ContainerRental
Request/Index', [
        'routes' => $routes->map(fn
(ShippingRoute $route) => [
            'id' => $route->id,
            'origin_port_id' => $route-
>origin_port_id,
            'destination_port_id' =>
$route->destination_port_id,
            'estimated_days' => $route-
>estimated_days,
            'distance' => (float) $route-
>distance,
            'origin_vessel_departure_at' =>
                $scheduleByPortId-
>get($route-
>origin_port_id)['vessel_departure_at'] ??
null,
            'label' => sprintf('%s, %s ->
%s, %s',
                $route->originPort?->name
?? 'N/A',
                $route->originPort?-
>country?->name ?? 'N/A',
                $route->destinationPort?-
>name ?? 'N/A',
                $route->destinationPort?-
>country?->name ?? 'N/A'
            ),
        ]),
        'ports' => $ports->map(fn (Port
$port) => [
            'id' => $port->id,
            'name' => $port->name,
            'city' => $port->city,
            'country_name' => $port-
>country?->name,
            'label' => sprintf('%s, %s',
$port->name, $port->country?->name ??
'N/A'),
        ]),
        'my_recent_requests' =>
$myRecentRequests,
    ]);
}
```

**Код 9 – Попередній розрахунок заявки на оренду *(app/Http/Controllers/RentalController.php)***

**Призначення:** Перевіряє форму, обраний маршрут і можливість виконання перевезення до збереження заявки.

```
public function preview(Request $request):
JsonResponse
{
    $validated = $request->validate([
        'route_mode' => ['required',
'in:route,ports'],
        'route_id' => ['nullable',
'integer', 'exists:routes,id'],
        'origin_port_id' => ['nullable',
'integer', 'exists:ports,id'],
        'destination_port_id' =>
['nullable', 'integer', 'exists:ports,id',
'different:origin_port_id'],
        'container_id' => ['nullable',
'integer', 'exists:containers,id'],
        'start_date' => ['required',
'date', 'after_or_equal:today'],
        'end_date' => ['required', 'date',
'after:start_date'],
        'cargo_types' => ['required',
'array', 'min:1'],
        'cargo_types.*' => ['string'],
        'requested_weight' => ['nullable',
'numeric', 'min:0'],
        'cargo_volume_cbm' => ['nullable',
'numeric', 'min:0'],
        'package_count' => ['nullable',
'integer', 'min:1'],
        'cargo_value' => ['nullable',
'numeric', 'min:0'],
        'priority' => ['required',
'in:normal,urgent,express'],
        'delivery_mode' => ['required',
'in:port_to_port,door_to_port,port_to_door,
door_to_door'],
        'loading_type' => ['required',
'in:fcl,lcl'],
        'sustainability_pref' =>
['required',
'in:standard,eco_optimized,low_emission'],
        'insurance_required' =>
['required', 'boolean'],
        'requires_customs_clearance' =>
['required', 'boolean'],
        'hazardous_material' =>
['required', 'boolean'],
        'requires_escort' => ['required',
'boolean'],
        'seal_required' => ['required',
'boolean'],
        'routing_priority' => ['nullable',
'string', 'in:speed,cost,balanced'],
    ]);

    $startDate =
CarbonImmutable::parse($validated['start_da
te']);
    $endDate =
CarbonImmutable::parse($validated['end_date
']);
    $routeContext = $this-
>availabilityService-
>resolveRouteContext($validated);

    if (! ($routeContext['path_found'] ??
true)) {
        return response()->json([
            'message' => 'No open shipping
route connects the selected ports. Choose
another pair or contact operations.',
        ], 422);
    }

    $routeLegs =
is_array($routeContext['route_legs'] ??
null)
        ? $routeContext['route_legs']
        : [];

    $plan = $this->feasibility->buildPlan(
        $routeLegs,
        $startDate,
        $validated['routing_priority'] ??
null
    );

    if (! ($plan['can_create_rental'] ??
true)) {
        return response()->json([
            'message' =>
$plan['warnings'][0] ?? 'Selected routing
cannot be fulfilled.',
            'route_plan' => $plan,
        ], 422);
    }
}
```

**Код 10 – Підбір контейнера та виклик сервісу ціни *(app/Http/Controllers/RentalController.php)***

**Призначення:** Знаходить доступний контейнер і передає параметри замовлення у RentalPricingService.

```
$availableContainers = $this-
>availabilityService-
>findAvailableContainers(
    $startDate,
    $endDate,
    $routeContext['origin_port_id'],
    isset($validated['requested_weight']) ?
(float) $validated['requested_weight'] :
null,
    $validated['cargo_types']
);

$selectedForPricing = null;
if (! empty($validated['container_id'])) {
    $selectedForPricing =
$availableContainers->firstWhere('id',
(int) $validated['container_id']);
}
```

```
if (! $selectedForPricing instanceof
Container && $availableContainers-
>isNotEmpty()) {
    $selectedForPricing =
$availableContainers->first();
}

$priceBreakdown = null;
$estimatedPrice = 0.0;

if ($selectedForPricing instanceof
Container) {
    $user = User::query()->with('country')-
>findOrFail($request->user()->id);

    $priceBreakdown = $this-
>pricingService->calculate(
        $user,
        $selectedForPricing,
        $startDate,
        $endDate,
        (float) $routeContext['distance'],
        (int)
$routeContext['estimated_days'],
        $validated['cargo_types'],
        isset($validated['requested_weight'
]) ? (float) $validated['requested_weight']
: null,
        isset($validated['cargo_volume_cbm'
]) ? (float) $validated['cargo_volume_cbm']
: null,
        isset($validated['package_count'])
? (int) $validated['package_count'] : null,
        isset($validated['cargo_value']) ?
(float) $validated['cargo_value'] : null,
        (string) ($validated['priority'] ??
'normal'),
        (string)
($validated['delivery_mode'] ??
'port_to_port'),
        (string)
($validated['loading_type'] ?? 'fcl'),
        (string)
($validated['sustainability_pref'] ??
'standard'),
        (bool)
($validated['insurance_required'] ??
false),
        (bool)
($validated['hazardous_material'] ??
false),
        (bool)
($validated['requires_customs_clearance']
?? false),
        (bool)
($validated['requires_escort'] ?? false),
        (bool) ($validated['seal_required']
?? false)
    );

    $priceBreakdown['route_legs'] =
$routeContext['route_legs'] ?? [];
    $priceBreakdown['routing_mode'] =
$routeContext['routing_mode'] ?? null;
    $estimatedPrice = (float)
$priceBreakdown['estimated_total'];
}
```

**Код 11 – Збереження заявки з блокуванням від дублювання *(app/Http/Controllers/RentalController.php)***

**Призначення:** Захищає контейнер від подвійного бронювання через транзакцію і lockForUpdate().

```
public function store(StoreRentalRequest
$request): RedirectResponse
{
    $validated = $request->validated();
    $startDate =
CarbonImmutable::parse($validated['start_da
te']);
    $endDate =
CarbonImmutable::parse($validated['end_date
']);
    $routeContext = $this-
>availabilityService-
>resolveRouteContext($validated);

    $availableContainers = $this-
>availabilityService-
>findAvailableContainers(
        $startDate,
        $endDate,
        $routeContext['origin_port_id'],
        isset($validated['requested_weight'
]) ? (float) $validated['requested_weight']
: null,
        $validated['cargo_types']
    );

    $selectedContainer =
$availableContainers->firstWhere('id',
(int) $validated['container_id']);

    if (! $selectedContainer instanceof
Container) {
        return back()->withErrors([
            'container_id' => 'Selected
container is no longer available for this
period.',
        ]);
    }

    $user = User::query()->with('country')-
>findOrFail($request->user()->id);
    $priceBreakdown = $this-
>pricingService->calculate(/* parameters
*/);

    $rental = DB::transaction(function ()
use ($request, $validated, $routeContext,
        $startDate, $endDate,
$priceBreakdown) {

        $container = Container::query()
            ->whereKey((int)
$validated['container_id'])
            ->lockForUpdate()
            ->firstOrFail();

        $overlapExists = Rental::query()
            ->where('container_id',
$container->id)
            ->whereIn('status',
['pending_approval', 'approved',
'scheduled', 'in_progress'])
            ->where('start_date', '<=',
$endDate)
            ->where('end_date', '>=',
$startDate)
            ->exists();

        if ($overlapExists) {
            throw
ValidationException::withMessages([
                'container_id' =>
'Container was just booked for this
period.',
            ]);
        }

        return Rental::query()->create([
            'user_id' => $request->user()-
>id,
            'container_id' => $container-
>id,
            'route_id' =>
$routeContext['route_id'],
            'origin_port_id' =>
$routeContext['origin_port_id'],
            'destination_port_id' =>
$routeContext['destination_port_id'],
            'start_date' => $startDate,
            'end_date' => $endDate,
            'rental_days' => $startDate-
>diffInDays($endDate),
            'cargo_types' =>
$validated['cargo_types'],
            'price' =>
$priceBreakdown['estimated_total'],
            'price_breakdown' =>
$priceBreakdown,
            'status' => 'pending_approval',
            'payment_status' => 'pending',
        ]);
    });
```

```
    return redirect()-
>route('rentals.center')->with('success',
'Rental request submitted.');
}
```

**Код 12 – Сервіс розрахунку вартості оренди**
***(app/Services/RentalPricingService.php)***

**Призначення:** Основний backend-алгоритм, який формує деталізовану ціну за всіма складовими.

```
class RentalPricingService
{
    public function calculate(
        User $user,
        Container $container,
        CarbonImmutable $startDate,
        CarbonImmutable $endDate,
        float $distanceKm,
        int $estimatedTransitDays,
        array $cargoTypes = [],
        ?float $requestedWeight = null,
        ?float $cargoVolumeCbm = null,
        ?int $packageCount = null,
        ?float $cargoValue = null,
        string $priority = 'normal',
        string $deliveryMode =
'port_to_port',
        string $loadingType = 'fcl',
        string $sustainabilityPref =
'standard',
        bool $insuranceRequired = false,
        bool $hazardousMaterial = false,
        bool $requiresCustomsClearance =
false,
        bool $requiresEscort = false,
        bool $sealRequired = false
    ): array {
        $days = max(1, $startDate-
>diffInDays($endDate));
        $distance = max(0, $distanceKm);
        $basePrice = 5000.00;
        $containerMultiplier = $this-
>containerTypeMultiplier($container->type);

        $distanceComponent = $distance *
0.10;
        $dailyComponent = $days * 20.00;
        $cargoComponent =
count($cargoTypes) * 75.00;
        $weightComponent =
($requestedWeight && $requestedWeight > 0)
            ? ($requestedWeight * 0.02)
            : 0.00;
        $volumeComponent = ($cargoVolumeCbm
&& $cargoVolumeCbm > 0)
            ? ($cargoVolumeCbm * 18.00)
            : 0.00;
        $packageComponent = ($packageCount
&& $packageCount > 0)
            ? ($packageCount * 4.00)
            : 0.00;

        $iotSurcharge = $container-
>iot_active ? 250.00 : 0.00;
        $transitUrgencySurcharge = $days <
$estimatedTransitDays ? 800.00 : 0.00;

        $prioritySurcharge = match
(strtolower($priority)) {
            'express' => 1200.00,
            'urgent' => 550.00,
            default => 0.00,
        };

        $hazardousSurcharge =
$hazardousMaterial ? 900.00 : 0.00;
        $customsHandlingFee =
$requiresCustomsClearance ? 220.00 : 0.00;
    }
}
```

**Код 13 – Підсумовування вартості та повернення JSON-структури**
***(app/Services/RentalPricingService.php)***

**Призначення:** Повертає не тільки загальну суму, а й детальну розбивку для відображення на фронтенді.

```
$deliveryModeFee = match
(strtolower($deliveryMode)) {
    'door_to_door' => 600.00,
    'door_to_port', 'port_to_door' =>
320.00,
    default => 0.00,
};

$lclHandlingFee = strtolower($loadingType)
=== 'lcl' ? 180.00 : 0.00;
$escortFee = $requiresEscort ? 450.00 :
0.00;
$sealFee = $sealRequired ? 75.00 : 0.00;

$sustainabilityFee = match
(strtolower($sustainabilityPref)) {
    'eco_optimized' => 140.00,
    'low_emission' => 260.00,
    default => 0.00,
};

$insuranceCost = $insuranceRequired
    ? max(120.00, (float) ($cargoValue ??
0) * 0.012)
    : 0.00;

$subtotal = ($basePrice *
$containerMultiplier)
    + $distanceComponent
    + $dailyComponent
    + $cargoComponent
    + $weightComponent
    + $volumeComponent
    + $packageComponent
    + $iotSurcharge
    + $transitUrgencySurcharge
    + $prioritySurcharge
    + $hazardousSurcharge
    + $customsHandlingFee
    + $deliveryModeFee
    + $lclHandlingFee
    + $escortFee
    + $sealFee
    + $sustainabilityFee
    + $insuranceCost;

$longTermDiscount = $days > 30 ? ($subtotal
* 0.10) : 0.00;
$discountedTotal = max(0, $subtotal -
$longTermDiscount);
$taxRate = (float) ($user->country?-
>interest_tax ?? 0.00);
$taxAmount = $discountedTotal * ($taxRate /
100);
$finalTotal = $discountedTotal +
$taxAmount;

return [
    'days' => $days,
    'currency' => 'USD',
    'base_price' => round($basePrice, 2),
    'container_multiplier' =>
$containerMultiplier,
    'distance_component' =>
round($distanceComponent, 2),
    'daily_component' =>
round($dailyComponent, 2),
    'cargo_component' =>
round($cargoComponent, 2),
    'iot_surcharge' => round($iotSurcharge,
2),
    'priority_surcharge' =>
round($prioritySurcharge, 2),
    'hazardous_surcharge' =>
round($hazardousSurcharge, 2),
    'insurance_cost' =>
round($insuranceCost, 2),
    'long_term_discount' =>
round($longTermDiscount, 2),
    'tax_rate' => round($taxRate, 2),
    'tax_amount' => round($taxAmount, 2),
    'estimated_total' => round($finalTotal,
2),
    'cargo_types' =>
array_values($cargoTypes),
];
```

**Код 14 – Коефіцієнт типу контейнера**

***(app/Services/RentalPricingService.php)***

**Призначення:** Окремо задає коефіцієнт для різних типів контейнерів.

```
private function
containerTypeMultiplier(?string $type):
float
{
    return match (strtolower((string)
$type)) {
        'high_cube' => 1.20,
        'open_top' => 1.25,
        'flat_rack' => 1.30,
        'refrigerated', 'reefer' => 1.40,
        default => 1.00,
    };
}
```

**Код 15 – Пошук відкритих маршрутів з доступними контейнерами**

***(app/Services/ContainerAvailabilityService.php)***

**Призначення:** Дає змогу показувати користувачу тільки ті маршрути, де у порту відправлення є доступний контейнер.

```
public function
openRouteIdsWithAvailableContainerAtOrigin(
): array
{
    $ids = Route::query()
        ->where('route_status', 'open')
        ->whereExists(function ($q) {
            $q->from('containers')
                -
>whereColumn('containers.current_port_id',
'routes.origin_port_id')
                -
>where('containers.current_status',
'available');
        })
        ->pluck('id')
        ->map(fn ($v) => (int) $v)
        ->values()
        ->all();

    return $ids;
}
```

**Код 16 – Порти з контейнерами і найближчим відправленням судна**

***(app/Services/ContainerAvailabilityService.php)***

**Призначення:** Звіряє наявність контейнерів з графіком суден і формує доступні стартові порти.

```
public function
portSchedulesWithAvailableContainers():
array
{
    $forecastDays = (int)
config('logistics.vessel_forecast_days',
30);
    $now = CarbonImmutable::now();
    $portIds = $this-
>portIdsWithAvailableContainerAtPort();

    $timeLoadDays = (int)
config('logistics.time_load_days', 2);
    $loadingCutoff = $now-
>addDays($timeLoadDays);
    $result = [];

    foreach ($portIds as $portId) {
        $departure = $this->vesselSchedule-
>nextDepartureWindowAtPort(
            $portId,
            $now,
            $forecastDays
        );

        if ($departure !== null &&
$departure->gt($loadingCutoff)) {
            $result[] = [
                'port_id' => $portId,
                'vessel_departure_at' =>
$departure->toIso8601String(),
            ];
        }
    }

    return $result;
}

public function
portIdsWithAvailableContainerAtPort():
array
{
    return Container::query()
        ->where('current_status',
'available')
        ->whereNotNull('current_port_id')
        ->distinct()
        ->pluck('current_port_id')
        ->map(fn ($v) => (int) $v)
        ->values()
        ->all();
}
```

**Код 17 – Визначення контексту маршруту**

***(app/Services/ContainerAvailabilityService.php)***

**Призначення:** Підтримує два режими: готовий маршрут або автоматичний пошук шляху між портами.

```
public function resolveRouteContext(array
$payload): array
{
    $routingMode = $this-
>resolveGraphWeightMode($payload);
    $routeMode = $payload['route_mode'] ??
'route';

    if ($routeMode === 'route' && !
empty($payload['route_id'])) {
        $route = Route::query()->find((int)
$payload['route_id']);

        if ($route === null) {
            return $this-
>emptyContext($routingMode);
        }

        $leg = $this-
>legFromRouteModel($route);
        return $this-
>assembleContextFromLegs([$leg],
$routingMode, false);
    }

    $originPortId = !
empty($payload['origin_port_id'])
        ? (int) $payload['origin_port_id']
        : null;

    $destinationPortId = !
empty($payload['destination_port_id'])
        ? (int)
$payload['destination_port_id']
        : null;

    if ($routeMode === 'ports' &&
$originPortId && $destinationPortId) {
        $direct = Route::query()
            ->where('route_status', 'open')
            ->where('origin_port_id',
$originPortId)
            ->where('destination_port_id',
$destinationPortId)
            ->orderBy($routingMode ===
'cost' ? 'distance' : 'estimated_days')
            ->first();

        if ($direct) {
            return $this-
>assembleContextFromLegs([
                $this-
>legFromRouteModel($direct)
            ], $routingMode, false);
        }

        $path = $this->pathfinder-
>shortestPath($originPortId,
$destinationPortId, $routingMode);
```

```
        return $path ? $this->assembleContextFromLegs($path['legs'], $routingMode, true)
            : $this->emptyContext($routingMode);
    }

    return $this->emptyContext($routingMode);
}
```

**Код 18 – Графовий пошук маршруту між портами**
***(app/Services/RoutePathfinderService.php)***

**Призначення:** Використовується для пошуку досяжних портів і побудови найкращого маршруту.

```
public function reachablePortIds(int $originPortId): array
{
    $graph = $this->buildGraph('balanced');
    $visited = [];
    $queue = [$originPortId];

    while ($queue !== []) {
        $current = array_shift($queue);
        if (isset($visited[$current])) {
            continue;
        }

        $visited[$current] = true;

        foreach ($graph[$current] ?? [] as $edge) {
            $next = (int) $edge['destination_port_id'];
            if (! isset($visited[$next])) {
                $queue[] = $next;
            }
        }
    }

    unset($visited[$originPortId]);

    return array_map('intval', array_keys($visited));
}

public function shortestPath(int $originPortId, int $destinationPortId, string $mode = 'balanced'): ?array
{
    $graph = $this->buildGraph($mode);
    $dist = [$originPortId => 0.0];
    $prev = [];
    $queue = [$originPortId => 0.0];

    while ($queue !== []) {
        asort($queue);
        $current = array_key_first($queue);
        unset($queue[$current]);

        if ($current === $destinationPortId) {
            break;
        }

        foreach ($graph[$current] ?? [] as $edge) {
            $next = (int) $edge['destination_port_id'];
            $alt = $dist[$current] + (float) $edge['weight'];
            if (! isset($dist[$next]) || $alt < $dist[$next]) {
                $dist[$next] = $alt;
                $prev[$next] = ['port' => $current, 'edge' => $edge];
                $queue[$next] = $alt;
            }
        }
    }

    if (! isset($dist[$destinationPortId])) {
        return null;
    }
}
```

**Код 19 – Криптографічний IoT-аудит**
***(app/Services/IotAuditChainService.php)***

**Призначення:** Кожен новий IoT-запис містить хеш попереднього запису, тому ланцюг не можна непомітно змінити.

```
class IotAuditChainService
{
    public const EVENT_ACTUATOR_UPDATED = 'actuator_updated';
    public const EVENT_SENSOR_TOGGLED = 'sensor_toggled';
    public const EVENT_SIMULATION_TICK = 'simulation_tick';

    public function append(
        int $containerId,
        string $eventType,
        array $payload,
        ?int $rentalId = null,
        ?int $userId = null
    ): IotAuditChain {
        return DB::transaction(function () use ($containerId, $eventType, $payload, $rentalId, $userId) {
            $prev = IotAuditChain::query()
                ->where('container_id', $containerId)
                ->orderByDesc('sequence')
                ->first();

            $sequence = $prev ? $prev->sequence + 1 : 1;
            $prevHash = $prev?->row_hash;
            $now = now();
            $payloadJson = json_encode($payload, JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE);
            $rowHash = hash('sha256',
                ($prevHash ?? '').'|'.$sequence.'|'.$payloadJson.'|'.$now->toIso8601String()
            );

            return IotAuditChain::query()->create([
                'container_id' => $containerId,
                'rental_id' => $rentalId,
                'user_id' => $userId,
                'event_type' => $eventType,
                'payload' => $payload,
                'sequence' => $sequence,
                'prev_hash' => $prevHash,
                'row_hash' => $rowHash,
            ]);
        });
    }
}
```

**Код 20 – Отримання записів IoT-аудиту для оренди**
***(app/Services/IotAuditChainService.php)***

**Призначення:** Повертає останні події IoT-ланцюга для відображення в моніторингу.

```
public function forRental(int $rentalId, int $containerId, int $limit = 50, ?int $beforeId = null)
{
    $query = IotAuditChain::query()
        ->where('container_id', $containerId)
        ->where('rental_id', $rentalId)
        ->with('user:id,first_name,last_name,email')
        ->orderByDesc('sequence');

    if ($beforeId !== null) {
        $query->where('id', '<', $beforeId);
    }

    return $query->limit($limit)->get();
}
```

**Код 21 – Сервіс підготовки даних IoT-графіків**
***(app/Services/MonitorChartsService.php)***

**Призначення:** Готує дані для карток сенсорів і графіків на сторінці моніторингу.

```
class MonitorChartsService
{
    public function payload(Rental $rental, Request $request): array
    {
        $rental->loadMissing(['container.sensors.sensorType', 'container.currentPort.country']);

        $range = $this->resolveRange($request);
        $sensorKeys = $this->sensorKeysForRental($rental);

        $metrics = Metric::query()
            ->where('rental_id', $rental->id)
            ->where('container_id', $rental->container_id)
            ->whereBetween('recorded_at', [$range['from'], $range['to']])
            ->whereIn('type', $sensorKeys)
            ->orderBy('recorded_at')
            ->get();

        return [
            'range' => [
                'from' => $range['from']->toIso8601String(),
                'to' => $range['to']->toIso8601String(),
            ],
            'sensors' => $this->buildSensorCards($rental, $metrics),
            'charts' => $this->buildCharts($metrics),
            'last_point_at' => optional($metrics->last())->recorded_at?->toIso8601String(),
        ];
    }
}
```

**Код 22 – Симуляція показників датчиків**
***(app/Services/SimulationService.php)***

**Призначення:** Генерує телеметрію для активних контейнерів і записує подію в IoT-аудит.

```
public function tick(?int $containerId = null): int
{
    $query = Container::query()
        ->with(['sensors.sensorType'])
        ->where('iot_active', true)
        ->whereHas('rentals', function ($q) {
            $q->whereIn('status', ['scheduled', 'in_progress'])
                ->where('is_telemetry_active', true);
        });

    if ($containerId !== null) {
        $query->whereKey($containerId);
    }

    $processed = 0;

    $query->chunkById(50, function ($containers) use (&$processed) {
        foreach ($containers as $container) {
            $rental = $container->rentals()
                ->whereIn('status', ['scheduled', 'in_progress'])
                ->where('is_telemetry_active', true)
                ->latest()
                ->first();

            if (! $rental) {
                continue;
            }

            $state = $this->nextState($container, $rental);
            $this->persistMetrics($container, $rental, $state);
            $this->audit->append(
                $container->id,
                IotAuditChainService::EVENT_SIMULATION_TICK,
                ['state' => $state],
                $rental->id
            );

            $processed++;
        }
    });

    return $processed;
}
```

**Код 23 – Команда запуску симуляції IoT**
***(app/Console/Commands/SimulationWorkerCommand.php)***

**Призначення:** Запускає фоновий процес, який регулярно оновлює IoT-показники.

```
class SimulationWorkerCommand extends Command
{
    protected $signature = 'simulation:worker
        {--once : Run one pass and exit}
        {--container= : Tick only one container}
        {--interval=5 : Seconds between passes}';

    protected $description = 'Run IoT simulation worker for active rentals';

    public function handle(SimulationService $simulation): int
    {
        $interval = max(1, (int) $this->option('interval'));
        $containerId = $this->option('container') ? (int) $this->option('container') : null;

        do {
            $processed = $simulation->tick($containerId);
            $this->info('Simulation tick processed containers: '.$processed);

            if ($this->option('once')) {
                break;
            }

            sleep($interval);
        } while (true);

        return self::SUCCESS;
    }
}
```

**Код 24 – Підключення Telegram-бота до акаунта**
***(app/Http/Controllers/TelegramLinkController.php)***

**Призначення:** Видає одноразовий код для прив'язки Telegram і дозволяє користувачу від'єднати бот.

```
class TelegramLinkController extends Controller
{
    public function __construct(
        private readonly TelegramLinkCodeService $linkCodes,
    ) {}

    public function createLinkCode(Request $request): JsonResponse
    {
        $user = $request->user();
        $issued = $this->linkCodes->issueForUser($user);
```

```
        return response()->json([
            'code' => $issued['plain'],
            'expires_at' =>
$issued['expires_at']->toIso8601String(),
            'bot' => [
                'username' => (string)
config('services.telegram.bot_username',
''),
            ],
        ]);
    }

    public function destroyLink(Request
$request, UserTelegramLink $link):
JsonResponse
    {
        $user = $request->user();

        if ((int) $link->user_id !== (int)
$user->id) {
            abort(403);
        }

        $link->delete();
        return response()->json(['ok' =>
true]);
    }
}
```

**Код 25 – Журналювання входу та системних дій *(app/Services/ActivityLogService.php)***

**Призначення:** Фіксує важливі дії користувачів для аудиту та адміністративного контролю.

```
class ActivityLogService
{
    public function logAuth(int $userId,
string $action, string $description,
Request $request): void
    {
        $this->log([
            'user_id' => $userId,
            'action' => $action,
            'model_name' => 'Auth',
            'model_id' => $userId,
            'new_values' => ['description'
=> $description],
            'ip_address' => $request->ip(),
            'device_type' => $this-
>deviceType($request),
            'browser' => $this-
>browser($request),
        ]);
    }

    public function log(array $payload):
void
    {
        ActivityLog::query()->create([
            'user_id' =>
$payload['user_id'] ?? null,
            'action' => $payload['action'],
            'model_name' =>
$payload['model_name'] ?? null,
            'model_id' =>
$payload['model_id'] ?? null,
            'old_values' =>
$payload['old_values'] ?? null,
            'new_values' =>
$payload['new_values'] ?? null,
            'ip_address' =>
$payload['ip_address'] ?? null,
            'device_type' =>
$payload['device_type'] ?? null,
            'browser' =>
$payload['browser'] ?? null,
        ]);
    }
}
```

**Код 26 – Модель оренди та зв’язки з іншими таблицями *(app/Models/Rental.php)***

**Призначення:** Показує центральну сутність системи, яка зв’язує користувача, контейнер, маршрут, оплату та IoT-дані.

```
class Rental extends Model
{
    protected $fillable = [
        'user_id', 'container_id',
'route_id',
        'origin_port_id',
'destination_port_id',
        'start_date', 'end_date',
'rental_days',
        'cargo_types', 'price',
'price_breakdown',
        'status', 'payment_status',
'is_telemetry_active',
        'rejection_reason',
'cancellation_reason',
    ];

    protected $casts = [
        'start_date' => 'datetime',
        'end_date' => 'datetime',
        'cargo_types' => 'array',
        'price_breakdown' => 'array',
        'price' => 'decimal:2',
        'is_telemetry_active' => 'boolean',
    ];

    public function user(): BelongsTo
    {
        return $this-
>belongsTo(User::class);
    }

    public function container(): BelongsTo
    {
        return $this-
>belongsTo(Container::class);
    }

    public function route(): BelongsTo
    {
        return $this-
>belongsTo(ShippingRoute::class,
'route_id');
    }

    public function metrics(): HasMany
    {
        return $this-
>hasMany(Metric::class);
    }

    public function canAccessIotMonitor():
bool
    {
        return in_array($this->status,
['approved', 'scheduled', 'in_progress',
'completed'], true)
            && $this->is_telemetry_active;
    }
}
```

**Код 27 – Модель контейнера *(app/Models/Container.php)***

**Призначення:** Описує контейнер як об’єкт оренди з власником, портом, сенсорами і сервісною історією.

```
class Container extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'serial_number', 'type', 'width',
'length', 'height',
        'max_weight', 'iot_active',
'current_status',
        'owner_id', 'current_port_id',
    ];

    protected $casts = [
        'width' => 'decimal:2',
        'length' => 'decimal:2',
        'height' => 'decimal:2',
        'max_weight' => 'decimal:2',
        'iot_active' => 'boolean',
    ];

    public function owner(): BelongsTo
    {
```

```
        return $this->belongsTo(Owner::class);
    }

    public function currentPort(): BelongsTo
    {
        return $this->belongsTo(Port::class, 'current_port_id');
    }

    public function rentals(): HasMany
    {
        return $this->hasMany(Rental::class);
    }

    public function sensors(): HasMany
    {
        return $this->hasMany(ContainerSensor::class);
    }

    public function maintenances(): HasMany
    {
        return $this->hasMany(Maintenance::class);
    }
}
```

**Код 28 – Модель телеметричної метрики**
***(app/Models/Metric.php)***

**Призначення:** Зберігає показники IoT-датчиків з типом, значенням, одиницею вимірювання і часовою міткою.

```
class Metric extends Model
{
    public $timestamps = false;

    protected $fillable = [
        'container_id',
        'rental_id',
        'type',
        'value',
        'unit',
        'meta',
        'recorded_at',
        'metrics_partition_key',
    ];

    protected $casts = [
        'value' => 'decimal:4',
        'meta' => 'array',
        'recorded_at' => 'datetime',
    ];

    public function container(): BelongsTo
    {
        return $this->belongsTo(Container::class);
    }

    public function rental(): BelongsTo
    {
        return $this->belongsTo(Rental::class);
    }
}
```

**Код 29 – Міграція таблиці оренд**
***(database/migrations/...create_rentals_table.php)***

**Призначення:** Фізична структура центральної таблиці, де зберігаються заявки на оренду.

```
Schema::create('rentals', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->foreignId('container_id')->constrained()->restrictOnDelete();
    $table->foreignId('route_id')->nullable()->constrained('routes')->nullOnDelete();
    $table->foreignId('origin_port_id')->nullable()->constrained('ports')->nullOnDelete();
    $table->foreignId('destination_port_id')->nullable()->constrained('ports')->nullOnDelete();
    $table->timestamp('start_date');
    $table->timestamp('end_date');
    $table->integer('rental_days');
    $table->json('cargo_types')->nullable();
    $table->decimal('requested_weight', 12, 2)->nullable();
    $table->decimal('cargo_volume_cbm', 10, 2)->nullable();
    $table->integer('package_count')->nullable();
    $table->decimal('cargo_value', 15, 2)->nullable();
    $table->decimal('price', 15, 2);
    $table->json('price_breakdown')->nullable();
    $table->string('status', 50)->default('pending_approval');
    $table->string('payment_status', 50)->default('pending');
    $table->boolean('is_telemetry_active')->default(false);
    $table->text('rejection_reason')->nullable();
    $table->text('cancellation_reason')->nullable();
    $table->timestamps();

    $table->index(['user_id', 'status']);
    $table->index(['container_id', 'start_date', 'end_date']);
    $table->index(['payment_status', 'status']);
});
```

**Код 30 – Міграція IoT-метрик**
***(database/migrations/...create_metrics_table.php)***

**Призначення:** Таблиця для великого обсягу телеметрії з індексами і партиціонуванням.

```
Schema::create('metrics', function (Blueprint $table) {
    $table->id();
    $table->foreignId('container_id')->constrained()->cascadeOnDelete();
    $table->foreignId('rental_id')->nullable()->constrained()->nullOnDelete();
    $table->string('type', 64);
    $table->decimal('value', 12, 4)->nullable();
    $table->string('unit', 100)->nullable();
    $table->json('meta')->nullable();
    $table->timestampTz('recorded_at');
    $table->bigInteger('metrics_partition_key')->default(0);

    $table->index(['container_id', 'type', 'recorded_at']);
    $table->index(['rental_id', 'recorded_at']);
});

DB::statement('ALTER TABLE metrics PARTITION BY LIST (metrics_partition_key)');
```

**Код 31 – Vue-форма заявки на оренду: стан форми**
***(resources/js/Pages/Operations/ContainerRentalRequest/Index.vue)***

**Призначення:** На фронтенді зберігає всі поля заявки та обраний контейнер.

```
const form = useForm({
    route_mode: 'route',
    route_id: '',
    origin_port_id: '',
    destination_port_id: '',
    container_id: '',
```

```
    start_date: '',
    end_date: '',
    cargo_types: [],
    requested_weight: '',
    cargo_volume_cbm: '',
    package_count: '',
    cargo_value: '',
    priority: 'normal',
    delivery_mode: 'port_to_port',
    loading_type: 'fcl',
    sustainability_pref: 'standard',
    insurance_required: false,
    requires_customs_clearance: false,
    hazardous_material: false,
    requires_escort: false,
    seal_required: true,
    routing_priority: '',
});

const preview = ref(null);
const loadingPreview = ref(false);
const selectedContainer = computed(() => {
    return
preview.value?.available_containers?.find(
        (container) => container.id ===
Number(form.container_id)
    );
});
```

**Код 32 – Vue: попередній розрахунок ціни**
***(resources/js/Pages/Operations/ContainerRentalRequest/Index.vue)***

**Призначення:** Надсилає форму на backend, отримує доступні контейнери та розбивку ціни.

```
const calculatePreview = async () => {
    loadingPreview.value = true;
    preview.value = null;

    try {
        const { data } = await
axios.post(route('rentals.request.preview')
, {
            route_mode: form.route_mode,
            route_id: form.route_id ||
null,
            origin_port_id:
form.origin_port_id || null,
            destination_port_id:
form.destination_port_id || null,
            container_id: form.container_id
|| null,
            start_date: form.start_date,
            end_date: form.end_date,
            cargo_types: form.cargo_types,
            requested_weight:
form.requested_weight || null,
            cargo_volume_cbm:
form.cargo_volume_cbm || null,
            package_count:
form.package_count || null,
            cargo_value: form.cargo_value
|| null,
            priority: form.priority,
            delivery_mode:
form.delivery_mode,
            loading_type:
form.loading_type,
            sustainability_pref:
form.sustainability_pref,
            insurance_required:
form.insurance_required,
            requires_customs_clearance:
form.requires_customs_clearance,
            hazardous_material:
form.hazardous_material,
            requires_escort:
form.requires_escort,
            seal_required:
form.seal_required,
            routing_priority:
form.routing_priority || null,
        });

        preview.value = data;
        if (! form.container_id &&
data.available_containers?.length) {
            form.container_id =
data.available_containers[0].id;
        }
    } catch (error) {
        preview.value = null;
        alert(error.response?.data?.message
?? 'Unable to calculate price.');
    } finally {
        loadingPreview.value = false;
    }
};
```

**Код 33 – Vue: надсилання заявки на оренду**
***(resources/js/Pages/Operations/ContainerRentalRequest/Index.vue)***

**Призначення:** Після підтвердження клієнтом створює заявку і очищає форму.

```
const submit = () => {
    form.post(route('rentals.request.store'
), {
        preserveScroll: true,
        onSuccess: () => {
            preview.value = null;
            form.reset(
                'route_id',
                'origin_port_id',
                'destination_port_id',
                'container_id',
                'start_date',
                'end_date',
                'cargo_types',
                'requested_weight',
                'cargo_volume_cbm',
                'package_count',
                'cargo_value'
            );
        },
    });
};

watch(
    () => [form.route_mode, form.route_id,
form.origin_port_id,
form.destination_port_id],
    () => {
        preview.value = null;
        form.container_id = '';
    }
);
```

**Код 34 – Vue-форма входу користувача**
***(resources/js/Pages/Auth/Login.vue)***

**Призначення:** Використовує Inertia useForm для надсилання email і пароля на Laravel-маршрут login.

```
const form = useForm({
    email: '',,
    password: '',
    remember: false,
});

const submit = () => {
    form.post(route('login'), {
        onFinish: () =>
form.reset('password'),
    });
};

<template>
    <GuestLayout>
        <Head title="Log in" />

        <form @submit.prevent="submit">
            <InputLabel for="email"
value="Email" />
            <TextInput
                id="email"
                type="email"
                class="mt-1 block w-full"
                v-model="form.email"
                required
                autofocus
                autocomplete="username"
            />
            <InputError class="mt-2"
:message="form.errors.email" />
```

```
            <InputLabel for="password"
value="Password" />
            <TextInput
                id="password"
                type="password"
                class="mt-1 block w-full"
                v-model="form.password"
                required
                autocomplete="current-
password"
            />

            <PrimaryButton :class="{
'opacity-25': form.processing }"
                :disabled="form.processing"
>
                Log in
            </PrimaryButton>
        </form>
    </GuestLayout>
</template>
```

**Код 35 – Vue: відображення списку оренд** ***(resources/js/Pages/Operations/RentalsCenter/Index.vue)***

**Призначення:** Показує оренди користувача, статуси, ціну і перехід до IoT-моніторингу.

```
const props = defineProps({
    rentals: Object,
    filters: Object,
    statusOptions: Array,
    paymentStatusOptions: Array,
});

const statusBadgeClass = (status) => ({
    pending_approval: 'bg-amber-100 text-
amber-800',
    approved: 'bg-emerald-100 text-emerald-
800',
    scheduled: 'bg-sky-100 text-sky-800',
    in_progress: 'bg-blue-100 text-blue-
800',
    completed: 'bg-slate-100 text-slate-
700',
    rejected: 'bg-rose-100 text-rose-800',
    cancelled: 'bg-zinc-100 text-zinc-700',
}[status] ?? 'bg-gray-100 text-gray-700');

const canOpenMonitor = (rental) => {
    return rental.can_view_iot_monitor &&
rental.container_iot_active;
};
```

```
<template>
    <OperationsLayout title="Rentals
Center">
        <table class="min-w-full divide-y
divide-slate-200">
            <tbody class="divide-y divide-
slate-100 bg-white">
                <tr v-for="rental in
rentals.data" :key="rental.id">
                    <td>{{
rental.container_serial }}</td>
                    <td>{{ rental.origin }}
-> {{ rental.destination }}</td>
                    <td>
                        <span
:class="statusBadgeClass(rental.status)">
                            {{
rental.status }}
                        </span>
                    </td>
                    <td>${{ rental.price
}}</td>
                    <td>
                        <Link v-
if="canOpenMonitor(rental)"
                            :href="route('r
entals.monitor', rental.id)">
                            IoT monitor
                        </Link>
                    </td>
                </tr>
            </tbody>
        </table>
    </OperationsLayout>
</template>
```

**Код 36 – Vue: завантаження даних графіків моніторингу** ***(resources/js/Pages/Operations/RentalMonitor/Index.vue)***

**Призначення:** Оновлює IoT-дашборд кожні 15 секунд без перезавантаження сторінки.

```
const charts = ref(null);
const loading = ref(false);
const selectedRange = ref('24h');

const loadCharts = async () => {
    loading.value = true;

    try {
        const { data } = await axios.get(
            route('rentals.monitor.charts-
data', props.rental.id),
            { params: { range:
selectedRange.value } }
        );

        charts.value = data;
    } catch (error) {
        console.error(error);
        charts.value = null;
    } finally {
        loading.value = false;
    }
};

onMounted(() => {
    loadCharts();
    intervalId =
window.setInterval(loadCharts, 15000);
});

onBeforeUnmount(() => {
    if (intervalId) {
        window.clearInterval(intervalId);
    }
});
```

**Код 37 – Docker Compose: основні сервіси** ***(docker-compose.yml)***

**Призначення:** Контейнеризує Laravel, Nginx, PostgreSQL і Redis в одному середовищі.

```
services:
  php:
    build:
      context: .
      dockerfile: Dockerfile
    volumes:
      - ./:/var/www/html
    depends_on:
      - postgres
      - redis
    networks:
      - app-network

  nginx:
    image: nginx:alpine
    ports:
      - "${NGINX_HTTP_PORT:-80}:80"
    volumes:
      - ./:/var/www/html
      -
./docker/nginx/default.conf:/etc/nginx/conf
.d/default.conf
    depends_on:
      - php
    networks:
      - app-network

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      -
postgres_data:/var/lib/postgresql/data
    networks:
      - app-network
```

```
  redis:
    image: redis:alpine
    networks:
      - app-network
```

**Код 38 – Docker Compose: черга, frontend і симуляція *(docker-compose.yml)***

**Призначення:** Окремими контейнерами запускає frontend, Redis-чергу, Telegram long-polling і IoT-симуляцію.

```
  node:
    image: node:20-alpine
    working_dir: /var/www/html
    volumes:
      - ./:/var/www/html
    command: sh -c "npm install && npm run
dev -- --host 0.0.0.0"
    ports:
      - "5173:5173"
    networks:
      - app-network

  queue:
    build:
      context: .
      dockerfile: Dockerfile
    command: php artisan queue:work redis -
-sleep=3 --tries=3
    volumes:
      - ./:/var/www/html
    depends_on:
      - php
      - redis
    networks:
      - app-network

  telegram:
    build:
      context: .
      dockerfile: Dockerfile
    command: php artisan telegram:poll
    volumes:
      - ./:/var/www/html
    depends_on:
      - php
      - redis
    networks:
      - app-network

  simulation:
    build:
      context: .
      dockerfile: Dockerfile
    command: php artisan simulation:worker
--interval=5
    volumes:
      - ./:/var/www/html
    depends_on:
      - php
      - postgres
    networks:
      - app-network
```

**Код 39 – Dockerfile для Laravel-застосунку *(Dockerfile)***

**Призначення:** Готує PHP-FPM контейнер із розширеннями для PostgreSQL, черг і Laravel.

```
FROM php:8.2-fpm-alpine AS base

RUN apk add --no-cache     bash     git
curl     libpq-dev     oniguruma-dev
libzip-dev     zip     unzip

RUN docker-php-ext-install     pdo
pdo_pgsql     mbstring     zip     bcmath
pcntl

COPY --from=composer:2 /usr/bin/composer
/usr/bin/composer

WORKDIR /var/www/html

COPY composer.json composer.lock ./

RUN composer install --no-interaction --
prefer-dist --no-dev --optimize-autoloader

COPY . .

RUN chown -R www-data:www-data storage
bootstrap/cache

CMD ["php-fpm"]
```

**Код 40 – Команди першого запуску проєкту *(README.md)***

**Призначення:** Набір команд для локального розгортання, міграцій, сідерів, карти маршрутів і IoT-симуляції.

```
cp .env.example .env

docker compose up -d --build

docker compose exec php php artisan
key:generate

docker compose exec php php artisan migrate
--force

docker compose exec php php artisan db:seed

# Якщо потрібно повністю очистити базу і
named volumes:
docker compose down -v --remove-orphans

docker compose up -d --build

docker compose exec php php artisan
migrate:fresh --seed --force

# Запуск побудови морських ліній для карти:
docker compose exec php php artisan
routes:build-sea-path --only-missing

# Ручний запуск IoT-симуляції:
docker compose exec php php artisan
schedule:work
# або
php artisan simulation:worker --interval=5
```